

图像滤波器及其应用

讲师: Hao WANG
AI Thrust, 信息枢纽

Image filters and its Applications

Instructor: Hao WANG
AI Thrust, Information Hub

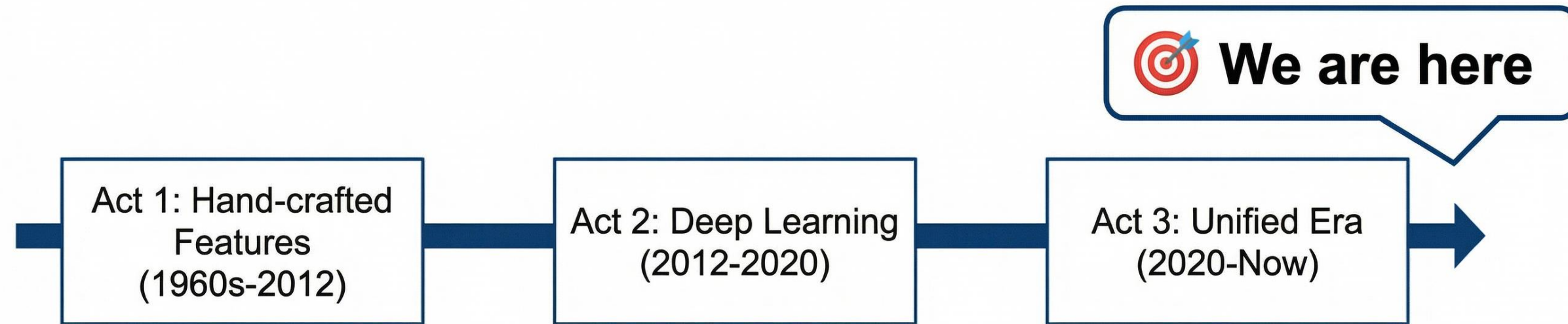


上周回顾

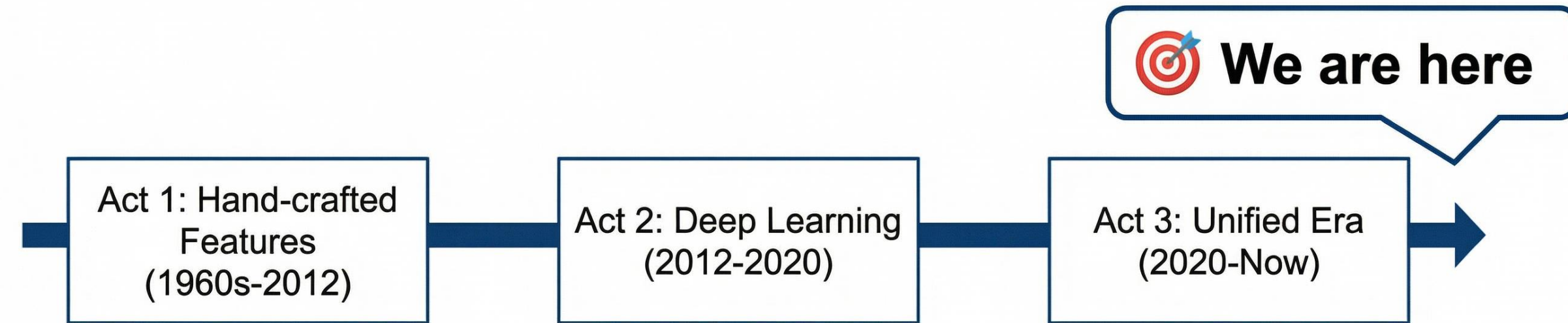
Recap of Last Week



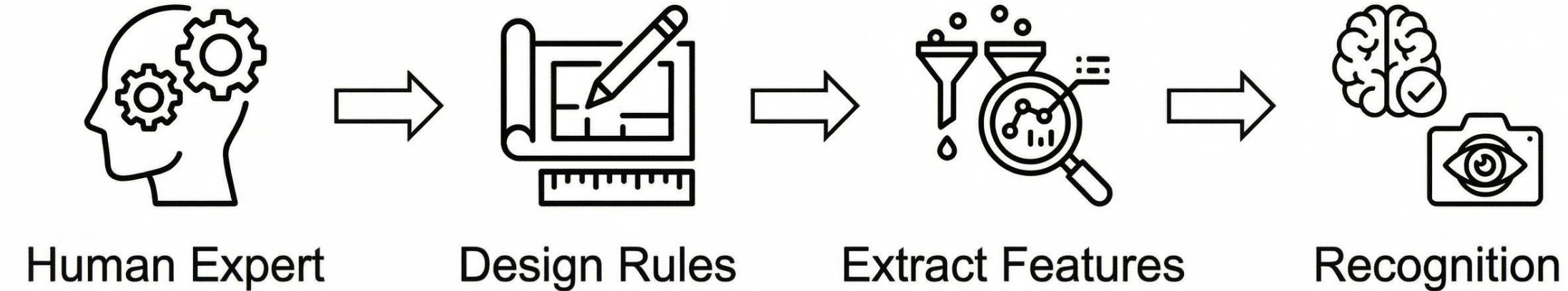
Three Eras of CV



Three Eras of CV



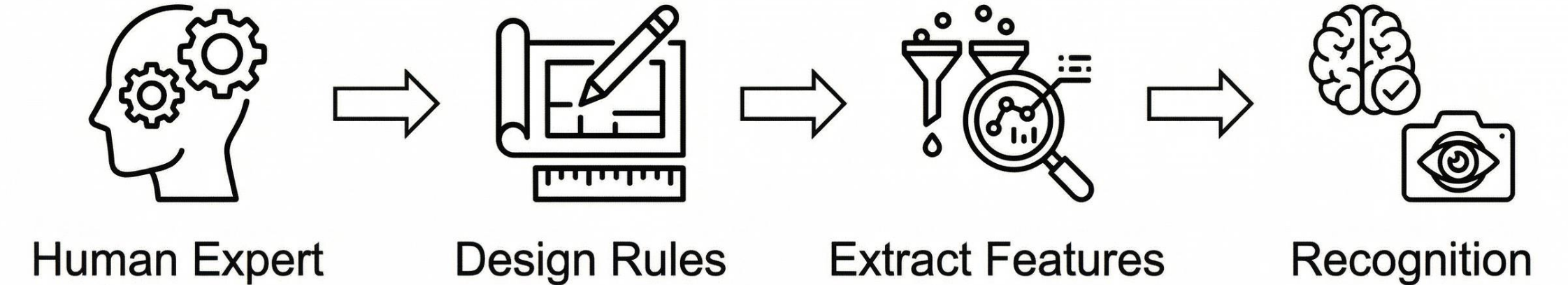
Hand-crafted Approach



Philosophy: Humans tell machines

Examples: SIFT, HOG, Haar

Hand-crafted Approach



Philosophy: Humans tell machines

Examples: SIFT, HOG, Haar

Limitations

- ✗ Task-specific
- ✗ Poor generalization
- ✗ Hard to design
- ✗ Performance ceiling

Breakthrough needed...

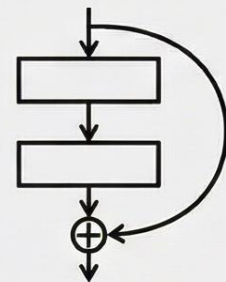
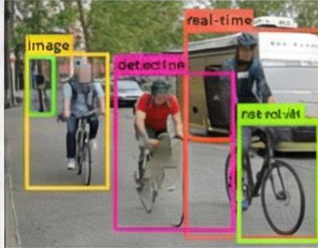
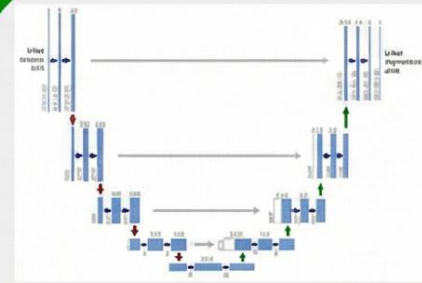
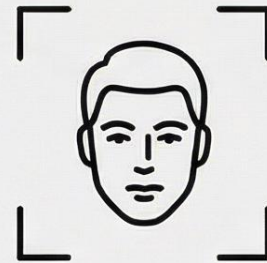
Limitations

- ✗ Task-specific
- ✗ Poor generalization
- ✗ Hard to design
- ✗ Performance ceiling

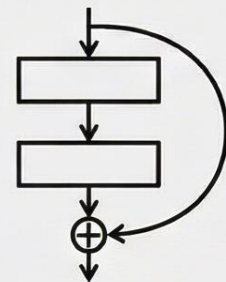
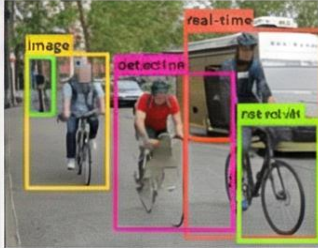
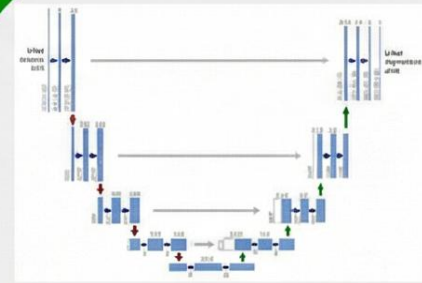
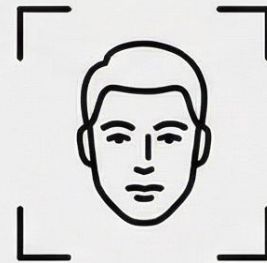
Breakthrough needed...



DL Achievements (2012-2020)

 ResNet: Classification: 95%+	 YOLO: Detection: Real-time	 U-Net: Segmentation: Pixel-level
 OpenPose: Pose: Joints	 FaceNet: Face: Beyond human	+ Depth, Video, 3D...

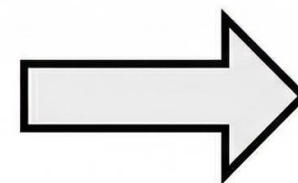
DL Achievements (2012-2020)

 ResNet: Classification: 95%+	 YOLO: Detection: Real-time	 U-Net: Segmentation: Pixel-level
 OpenPose: Pose: Joints	 FaceNet: Face: Beyond human	+ Depth, Video, 3D...

Act 2 Limitations

- ✗ One model, one task
- ✗ Understanding $\neq$ Generation
- ✗ Massive labeled data
- ✗ Lacks generalization

Can we unify them?

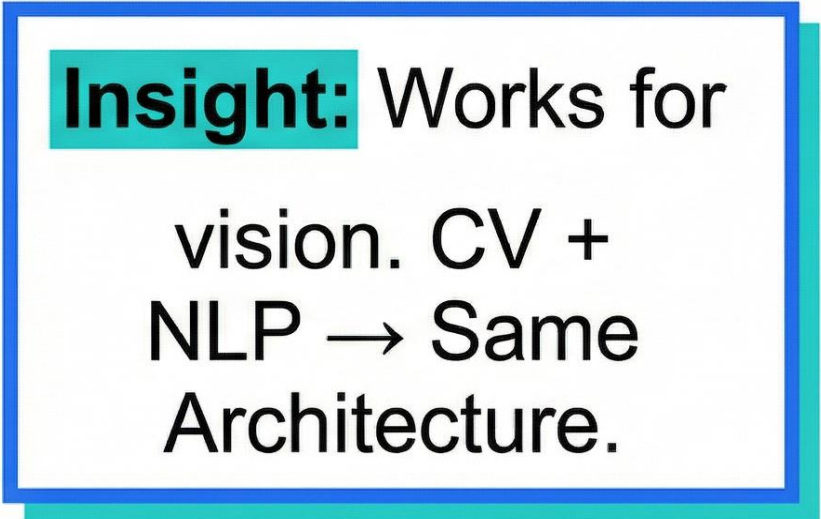
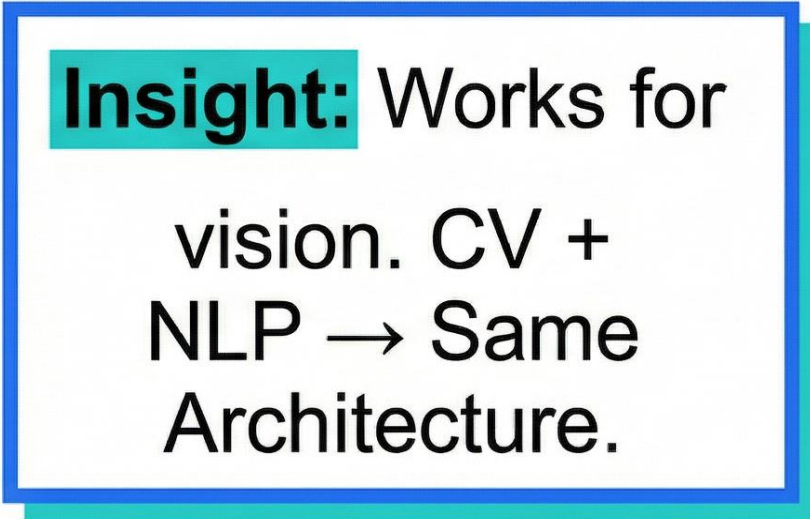


Act 2 Limitations

- ✗ One model, one task
- ✗ Understanding $\neq$ Generation
- ✗ Massive labeled data
- ✗ Lacks generalization

Can we unify them?





Four Unifying Trends

1 Multimodal
Text + Vision

2 Zero-shot
No training

3 Visual Dialog
Reasoning

4 World Model
Unified

Four Unifying Trends

1 Multimodal
Text + Vision

2 Zero-shot
No training

3 Visual Dialog
Reasoning

4 World Model
Unified



Image = Matrix



=

255	200	255
200	50	200
255	200	255

Each pixel = number
(grayscale) or 3
numbers (color)

Range: 0-255

Everything is
numbers.

Image = Matrix



=

255	200	255
200	50	200
255	200	255

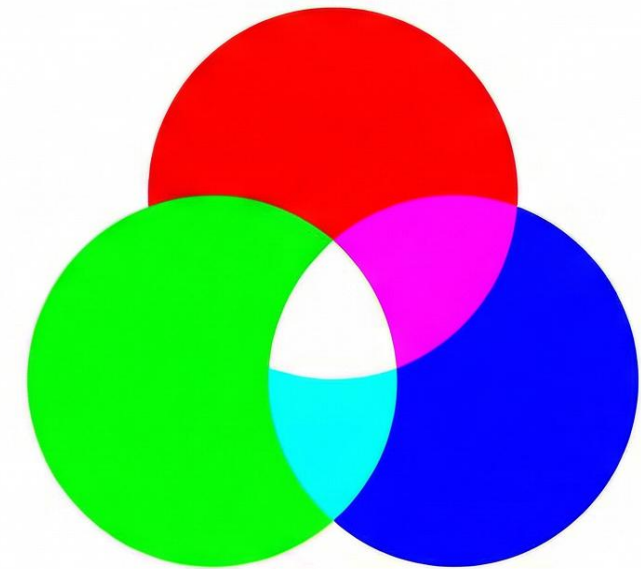
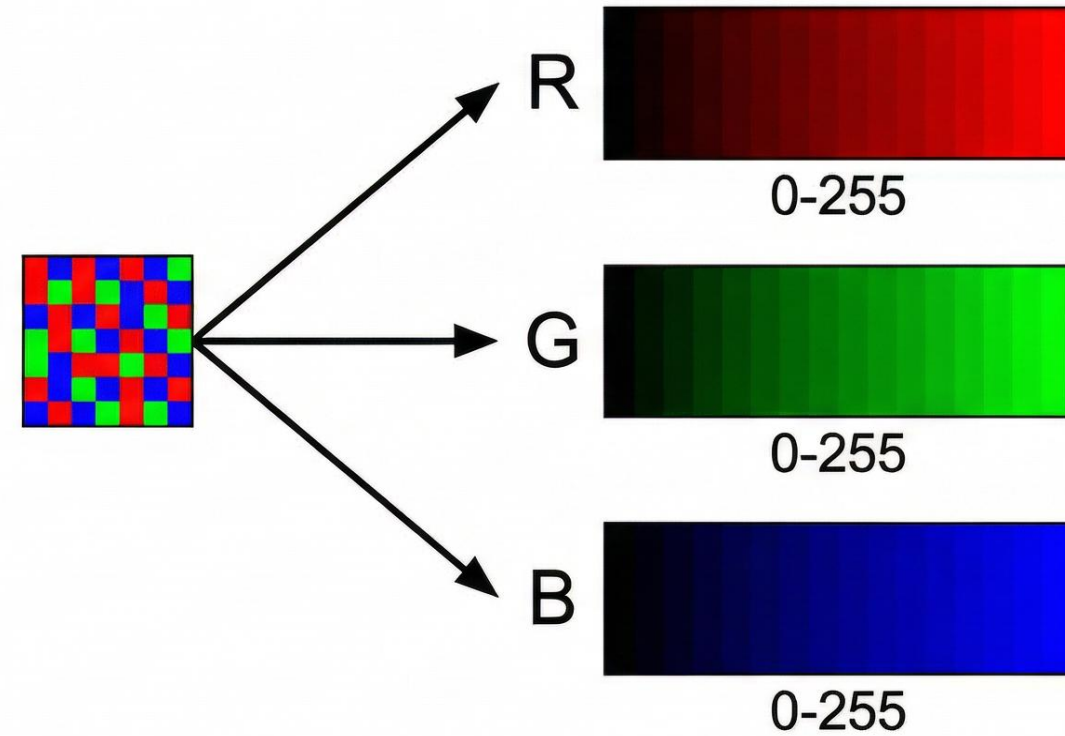
Each pixel = number
(grayscale) or 3
numbers (color)

Range: 0-255

Everything is
numbers.

Color = 3 Channels (RGB)

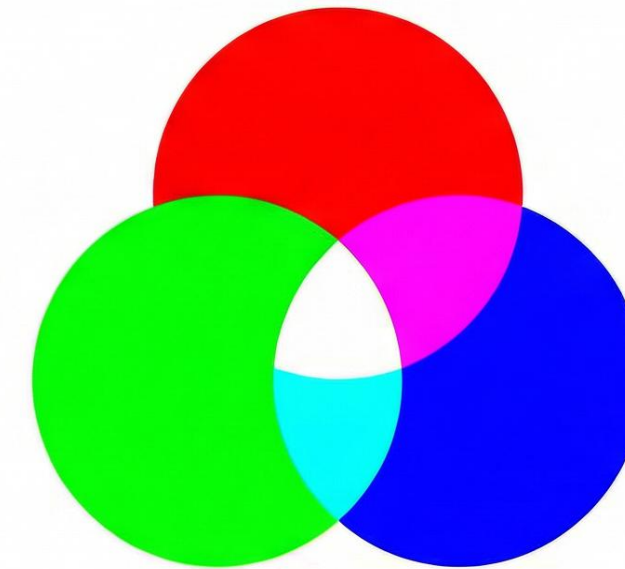
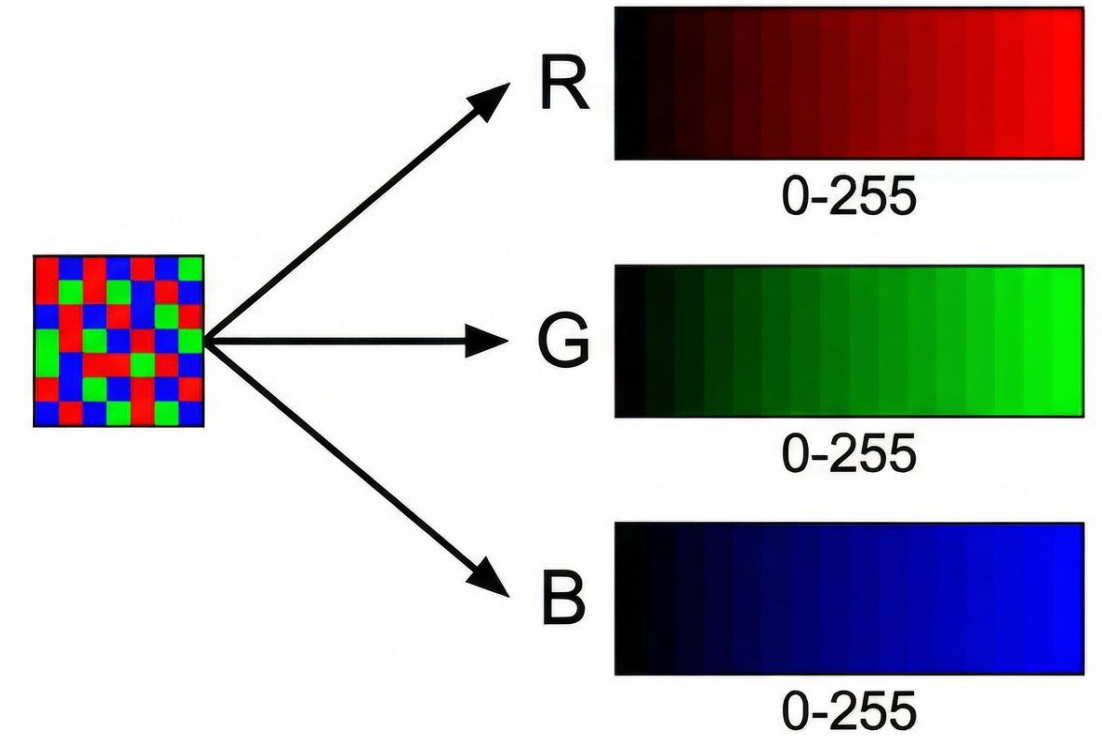
Color = 3 Channels
Each pixel = (R, G, B)
Each: 0-255



→ Red + Green + Blue → All colors

Color = 3 Channels (RGB)

Color = 3 Channels
Each pixel = (R, G, B)
Each: 0-255



→ Red + Green + Blue → All colors

Quiz

为什么 **Transformer** 被认为是现代人工智能的基础模型？ 以下哪一项最能解释其成功的主要原因？

- A. 它天然支持文本、图像和音频等多模态输入。 B. 相比先前的架构，它能更好地建模长程依赖关系。 C. 随着数据、参数和算力的增加，它展现出强大且可预测的扩展行为。 D. 其注意力机制与人类认知过程高度相似。

Quiz

Why is the **Transformer** considered a foundational model of modern AI? Which of the following best explains the *primary reason* for its success?

- A. It naturally supports multimodal inputs such as text, images, and audio.
- B. It models long-range dependencies better than previous architectures.
- C. It exhibits strong and predictable scaling behavior with more data, parameters, and compute.
- D. Its attention mechanism closely resembles human cognitive processes.



今日计划

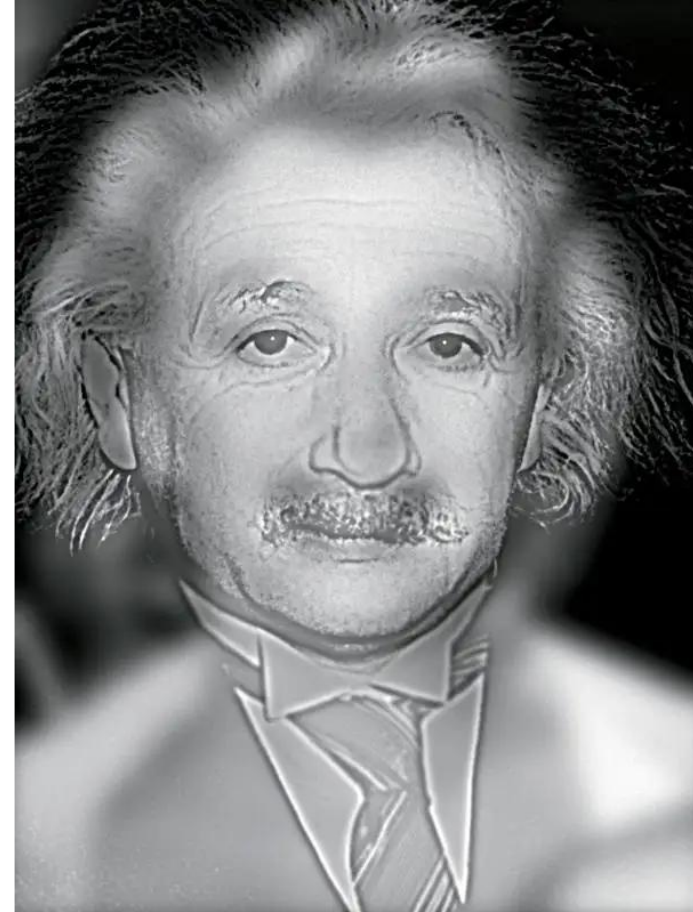
图像滤波器及其应用！

Today's Plan

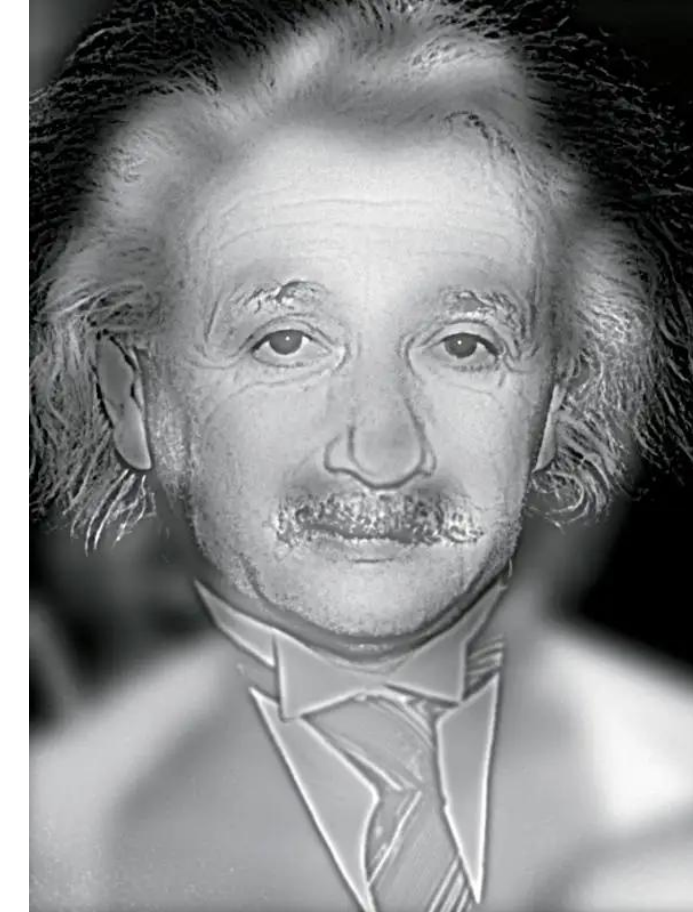
Image filters and its applications!



这个人是谁？



Who is this person?



图像滤波

- 图像滤波： 在每个位置计算局部邻域的函数
- 非常重要！
 - 增强图像
 - 去噪、调整尺寸等。
 - 从图像中提取信息
 - 纹理、边缘、显著点等。
 - 检测模式
 - 模板匹配
 - 未来内容： 卷积神经网络！



Image filtering

- Image filtering: compute function of local neighborhood at each position
- Really important!
 - Enhance images
 - Denoise, resize, etc.
 - Extract information from images
 - Texture, edges, distinctive points, etc.
 - Detect patterns
 - Template matching
 - Future content: Convolutional Neural Networks!



示例：盒式滤波器

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

“均值滤波器”：每个像素在邻域中都被同等重视。

Example: box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

“Mean Filter”: Every pixel in the neighborhood is treated with **equal importance**.

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

$$[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

来源：S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

$$h[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot] f$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$[\cdot, \cdot]$

	0	10							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

来源: S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot, \cdot]$$

	0	10	20						

$$[\cdot, \cdot] h = \sum_{k,l} g[k,l] f[m+k, n+l]$$

图片来源: S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20						

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot, \cdot]$$

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

来源: S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot]g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot]f$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot, \cdot]$$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

来源：S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot, \cdot]$$

	0	10	20	30	30				

$$[\cdot, \cdot] h = \sum_{k,l} g[k,l] f[m+k, n+l]$$

来源：S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

$$[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				
						?			

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

来源：S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

$$h[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				
						?			

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot] f$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot, \cdot]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$[\cdot, \cdot] h = \sum_{k,l} g[k,l] f[m+k, n+l]$$

图片来源: S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

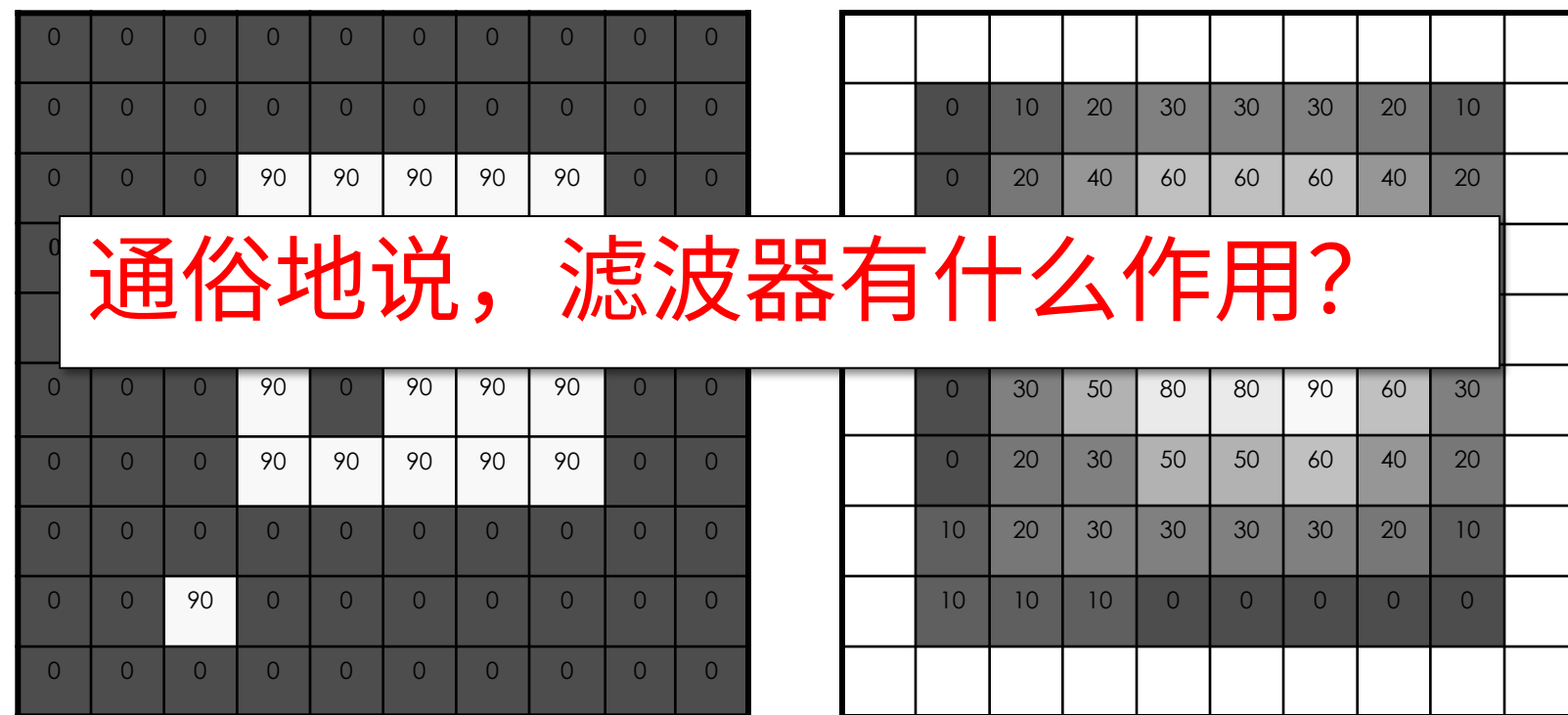
Credit: S. Seitz

图像滤波

$$[\cdot, \cdot] g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot] f$

$[\cdot, \cdot]$



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

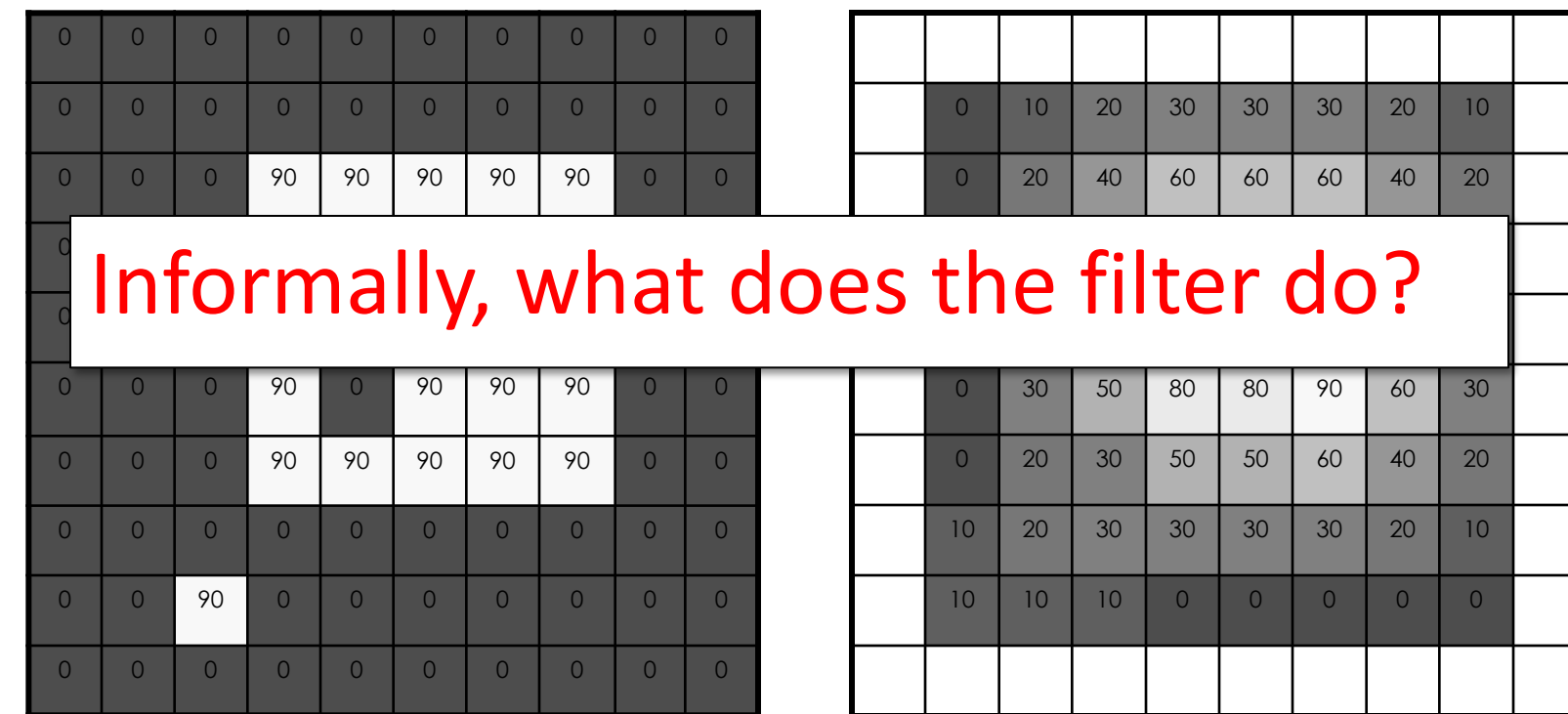
来源：S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Credit: S. Seitz

盒式滤波器

它有什么作用？

- 将每个像素替换为其邻域的平均值
- 实现平滑效果（去除尖锐特征）

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g$$

Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[\cdot, \cdot]$$

使用盒式滤波器进行平滑



Smoothing with box filter



线性滤波器练习



原图

0	0	0
0	1	0
0	0	0

?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

?



线性滤波器练习



原图

0	0	0
0	1	0
0	0	0



滤波后（无
变化）

来源：D. Lowe

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Source: D. Lowe

线性滤波器练习



原始

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?



线性滤波器练习



原图

0	0	0
0	0	1
0	0	0



向左平移
1 像素

来源：D. Lowe

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Source: D. Lowe

线性滤波器练习



原图

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(注意滤波器之和为 1)

?

Practice with linear filters



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(Note that filter sums to 1)

?

线性滤波器练习



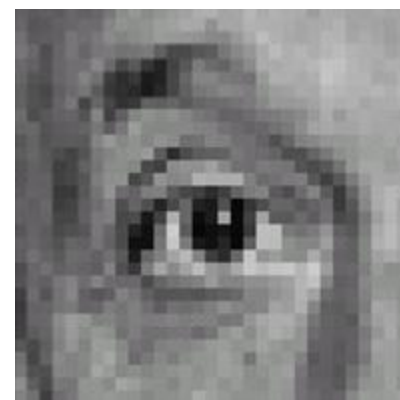
原图

0	0	0
0	2	0
0	0	0

— $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

锐化滤波器 - 强调与局部平均值的差异



来源：D. Lowe

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

— $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

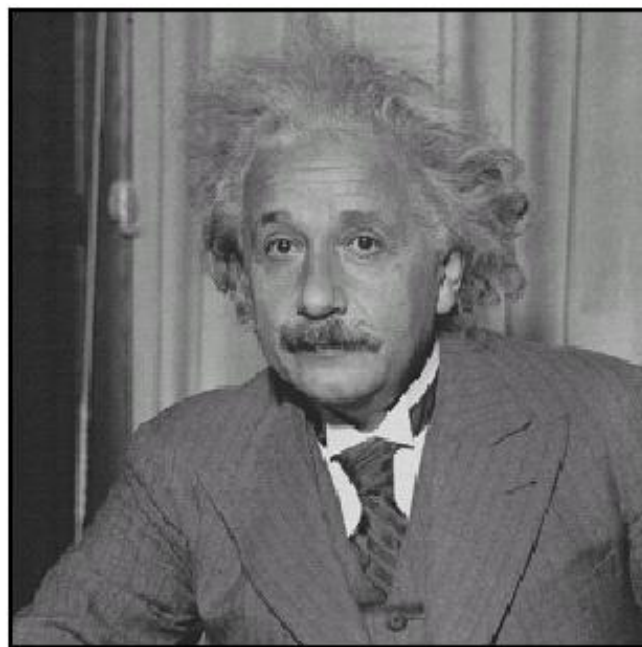
Sharpening filter
- Accentuates differences with local average



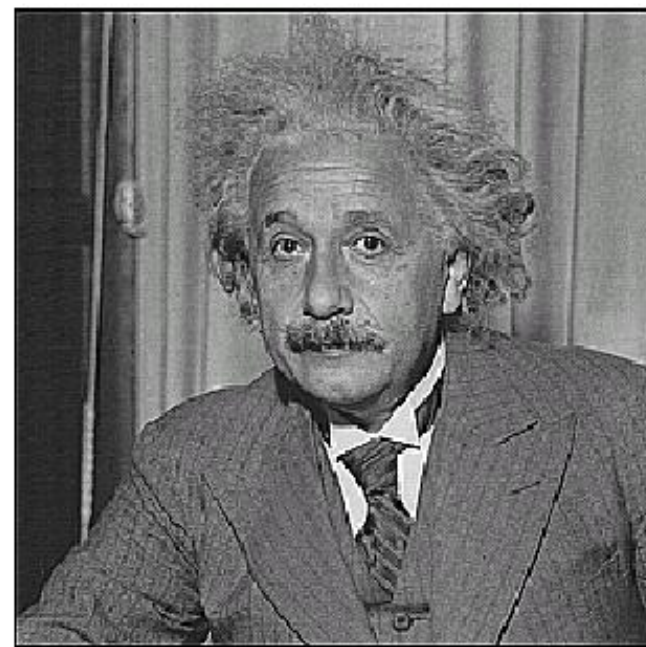
Source: D. Lowe



锐化



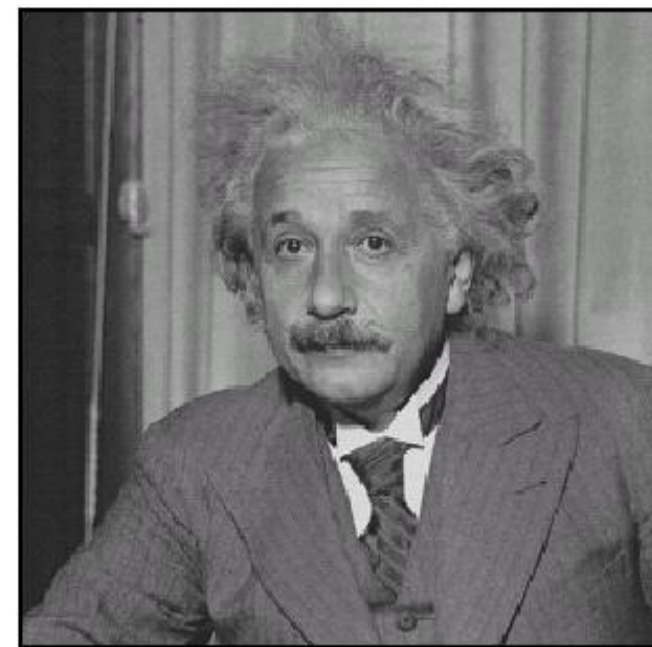
before



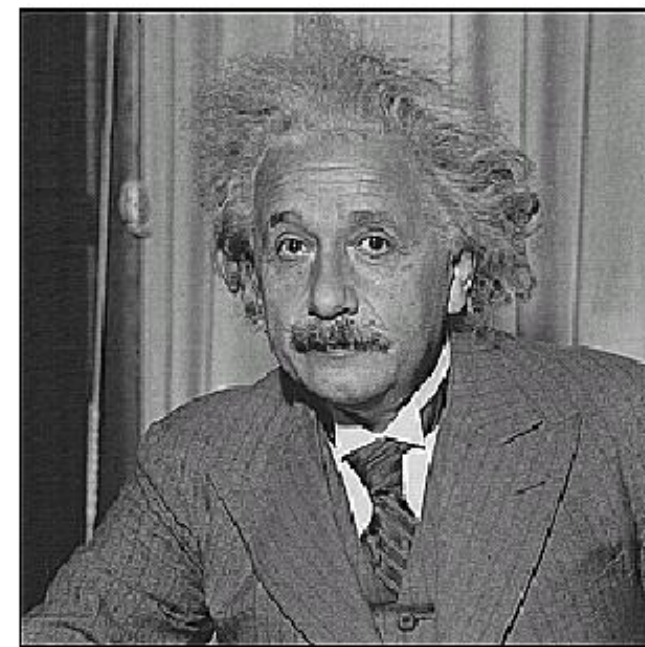
after

来源：D. Lowe

Sharpening



before



after

Source: D. Lowe



线性滤波器的关键性质

线性:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

平移不变性: 无论像素位置如何, 行为相同

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

任何线性平移不变算子都可以表示为卷积



Key properties of linear filters

Linearity:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

Shift invariance: same behavior regardless of pixel location

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

Any linear, shift-invariant operator can be represented as a convolution



更多性质

- 交换律: $a * b = b * a$
 - 从概念上讲, 滤波器与信号 (图像) 没有区别
- 结合律: $a * (b * c) = (a * b) * c$ • 通常依次应用多个滤波器:
 $((a * b_1) * b_2) * b_3$ • 这等价于应用一个滤波器: $a * (b_1 * b_2 * b_3)$
- 对加法满足分配律: $a * (b + c) = (a * b) + (a * c)$
- 标量可提取出来: $ka * b = a * kb = k(a * b)$
- 恒等性: 单位脉冲 $e = [0, 0, 1, 0, 0]$, $a * e = a$

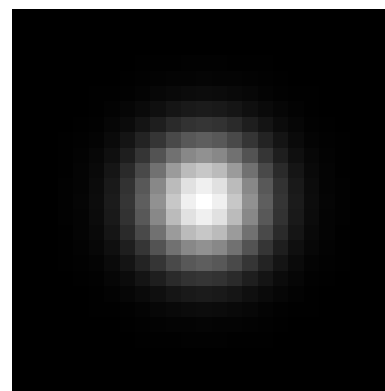
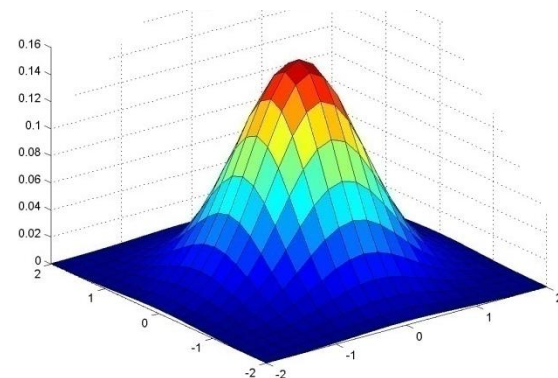
More properties

- Commutative: $a * b = b * a$
 - Conceptually no difference between filter and signal (image)
- Associative: $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k(a * b)$
- Identity: unit impulse $e = [0, 0, 1, 0, 0]$,
 $a * e = a$

重要滤波器：高斯

- 相邻像素基于邻近度的权重贡献

它使用加权平均，其中 **中心像素的影响最大**，并且周围像素的影响随着它们离中心越远而减小。



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

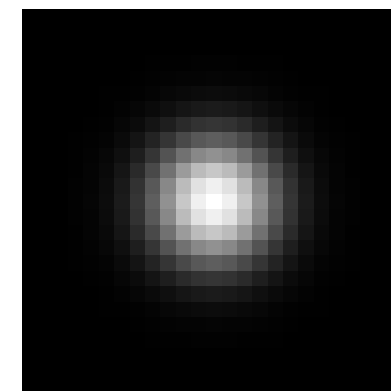
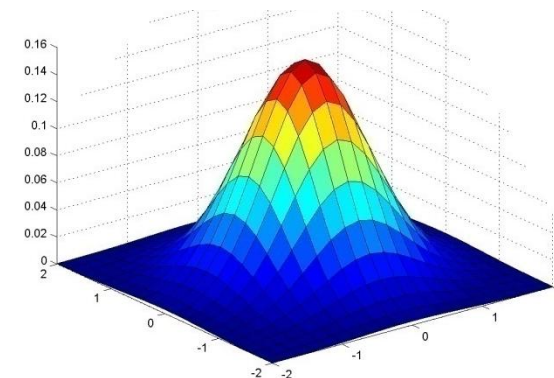
5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness

It uses a weighted average where the **center pixel has the most influence**, and the influence of surrounding pixels decreases as they get further from the center.

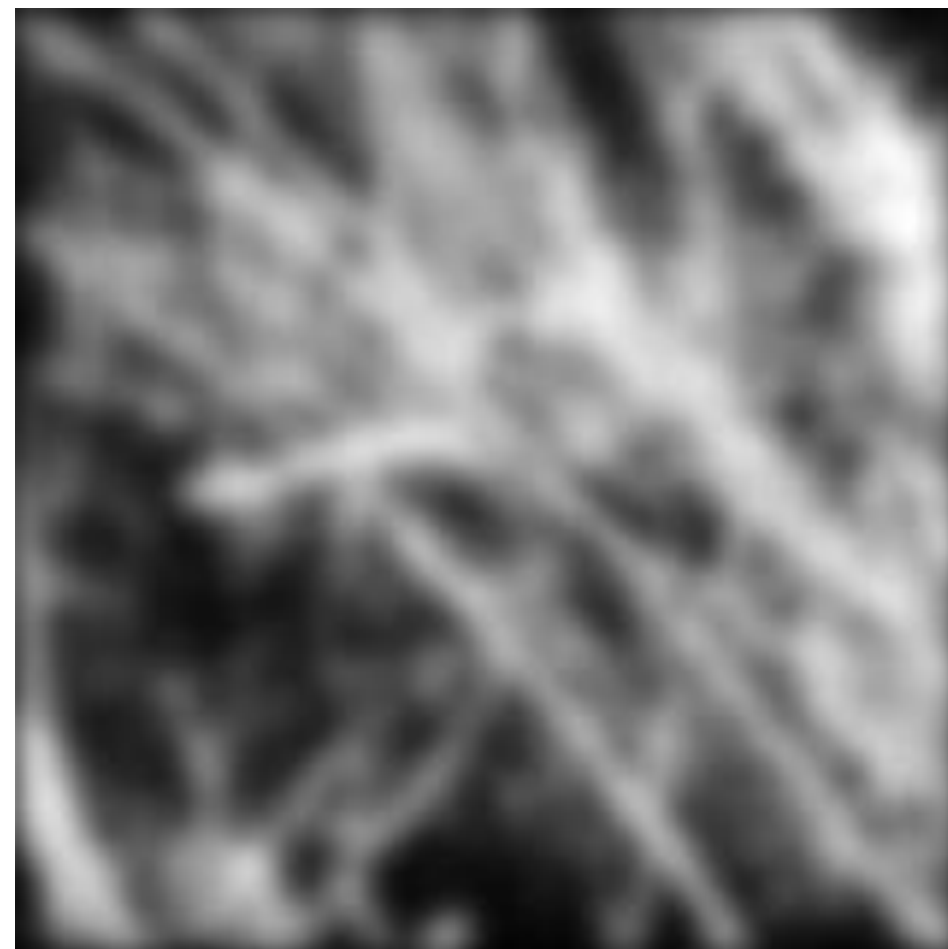


0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

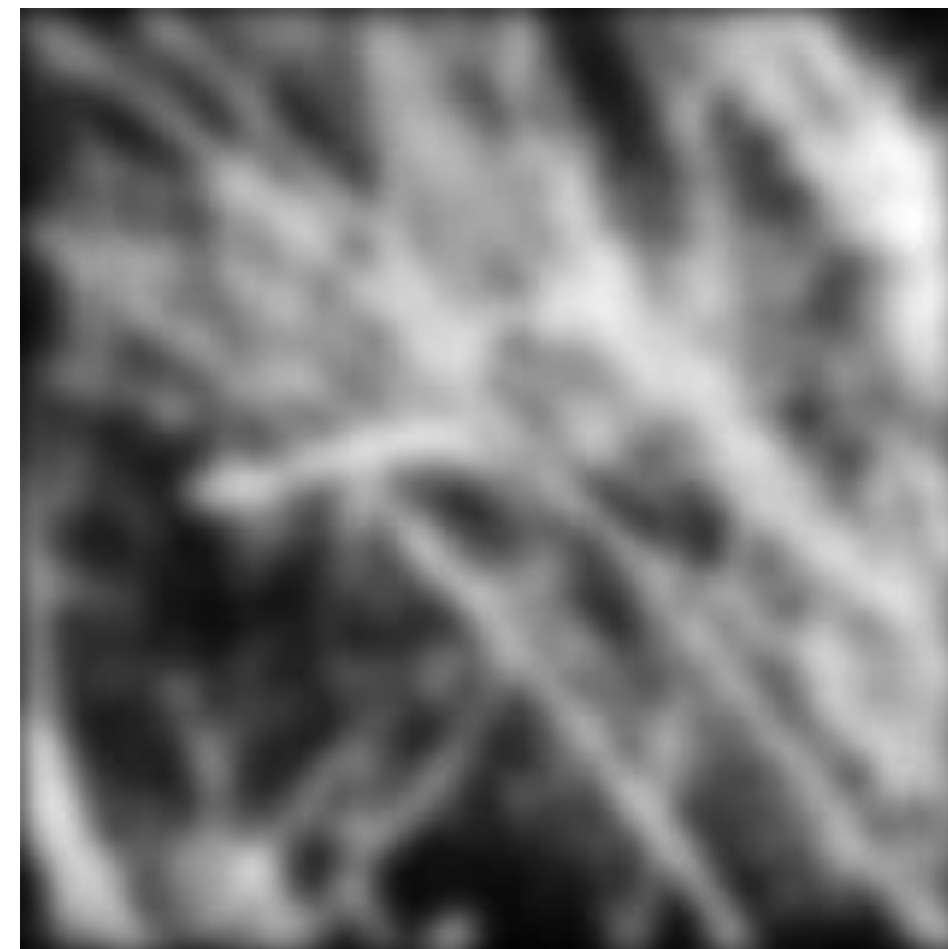
5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

使用高斯滤波器进行平滑



Smoothing with Gaussian filter



高斯滤波器

- 从图像中移除“高-频”分量（低通滤波器）

更自然

- 图像变得更平滑
- 与自身卷积得到另一个高斯函数—因此可以用小宽度核进行平滑，重复操作，得到与较大宽度核相同的结果—用宽度为 σ 的高斯核卷积两次，等同于用宽度为 $\sigma \sqrt{2}$ 的核卷积一次

更灵活

- 可分离核
 - 可分解为两个一维高斯函数的乘积

更高效

Gaussian filters

- Remove “**high-frequency**” components from the image (low-pass filter)

More natural

- Images become more smooth

- Convolution with self is another Gaussian

- So can smooth with small-width kernel, repeat, and get same result as larger-width kernel would have
 - Convolving two times with Gaussian kernel of width σ is same as convolving once with kernel of width $\sigma\sqrt{2}$

More Flexible

- *Separable* kernel

- Factors into product of two 1D Gaussians

More Efficient



高斯滤波器的可分离性

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Separability of the Gaussian filter

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

可分离性示例

$$O(M \cdot N \cdot m \cdot n)$$

二维滤波（仅中心位置）

1	2	1
2	4	2
1	2	1

 *

2	3	3
3	5	5
4	4	6

$$O(M \cdot N \cdot (m + n))$$

该滤波器可分解为一维滤波器的乘积：

1	2	1
2	4	2
1	2	1

 =

1
2
1

 x

1	2	1
---	---	---

沿行执行滤波：

1	2	1
---	---	---

 *

2	3	3
3	5	5
4	4	6

 =

	11	
	18	
	18	

接着沿剩余的列进行滤波：

1
2
1

 *

	11	
	18	
	18	

 =

	65	

来源：K. Grauman



Separability example

$$O(M \cdot N \cdot m \cdot n)$$

2D filtering (center location only)

1	2	1
2	4	2
1	2	1

 *

2	3	3
3	5	5
4	4	6

$$O(M \cdot N \cdot (m + n))$$

The filter factors into a product of 1D filters:

1	2	1
2	4	2
1	2	1

 =

1
2
1

 x

1	2	1
---	---	---

Perform filtering along rows:

1	2	1
---	---	---

 *

2	3	3
3	5	5
4	4	6

 =

	11	
	18	
	18	

Followed by filtering along the remaining column:

1
2
1

 *

	11	
	18	
	18	

 =

	65	

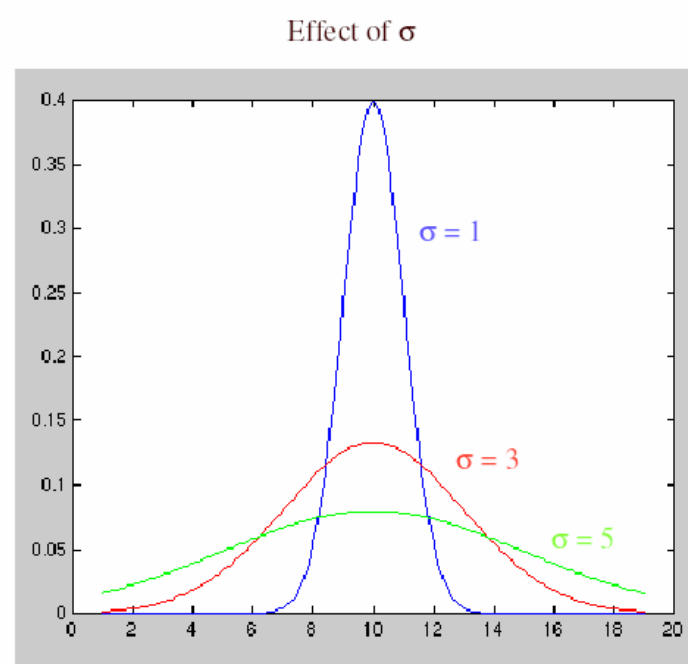
Source: K. Grauman



实际事项

滤波器应该多大?

- 边缘处的值应接近零
- 高斯核的经验法则：将核半宽设置为 $\geq 3\sigma$

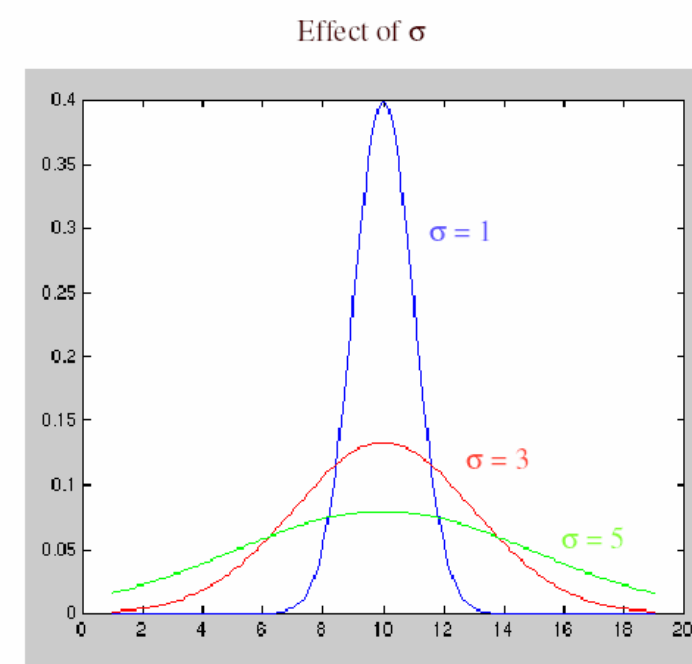


如果 $\sigma = 1$ ，你的卷积核半径应为 3，使得卷积核总大小为 7×7 （每侧 3 个像素 + 中心像素）。

Practical matters

How big should the filter be?

- Values at edges should be near zero
- **Rule of thumb for Gaussian: set kernel half-width to $\geq 3\sigma$**



If $\sigma = 1$, your kernel radius should be 3, making the total kernel size 7×7 (3 pixels on each side + the center pixel).

实际问题

- 靠近边缘时怎么办呢？
 - 滤波窗口超出图像边缘
 - 需要进行外推
 - 方法：
 - 裁剪滤波器（黑色）：将图像边界外的所有像素视为零（黑色）。
 - 环绕：将图像视为环面，右边缘与左边缘相连。
 - 复制边缘：将边缘最后一个像素的值向外扩展。
 - 沿边缘反射：沿边界镜像图像像素以填充空白区域。



来源：S. Marschner



Practical matters

- What about near the edge?
 - The filter window falls off the edge of the image
 - Need to extrapolate
 - Methods:
 - **Clip filter (black)**: Treat all pixels outside the image boundary as zero (black).
 - **Wrap around**: Treat the image as if it were a torus, where the right edge connects to the left edge.
 - **Copy edge**: Extend the value of the last pixel at the edge outward.
 - **Reflect across edge**: Mirror the image pixels across the boundary to fill the empty space.



Source: S. Marschner



实际问题

- 方法（Python）：
 - 裁剪滤波器（黑色）：`convolve2d(f, g, boundary='fill', 0)`
 - 环绕：`convolve2d(f, g, boundary='wrap')`
 - 沿边缘反射：`convolve2d(f, g, boundary='symm')`

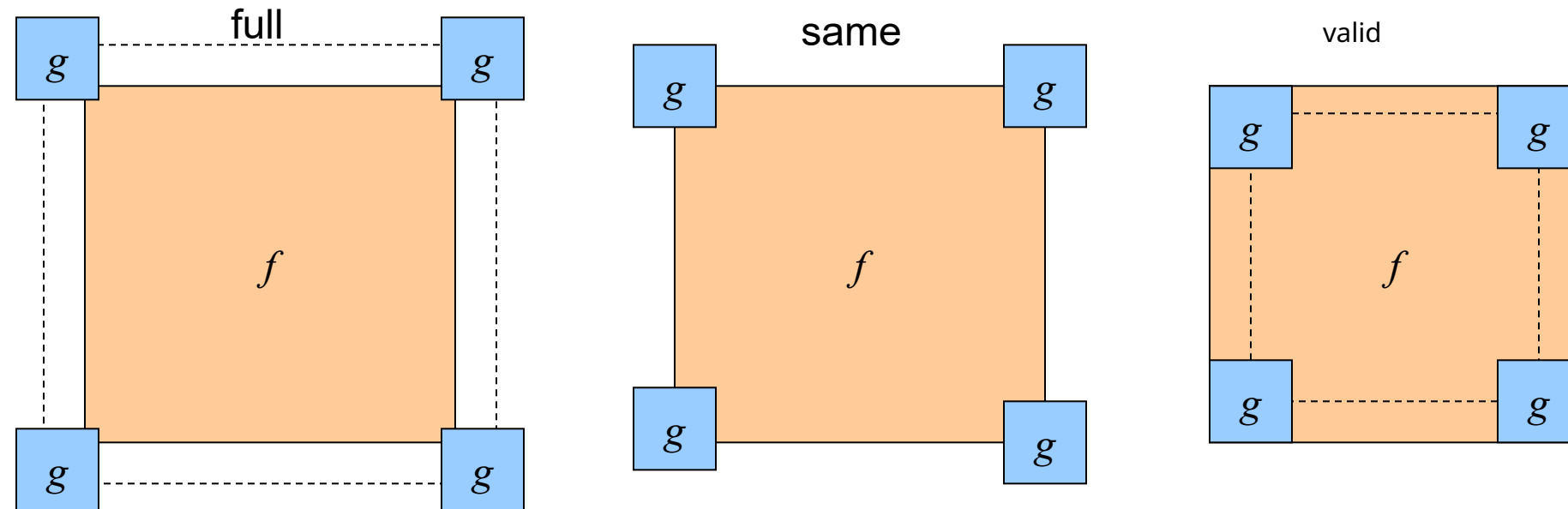
Practical matters

- Methods (Python):
 - clip filter (black): `convolve2d(f, g, boundary='fill', 0)`
 - wrap around: `convolve2d(f, g, boundary='wrap')`
 - reflect across edge: `convolve2d(f, g, boundary='symm')`



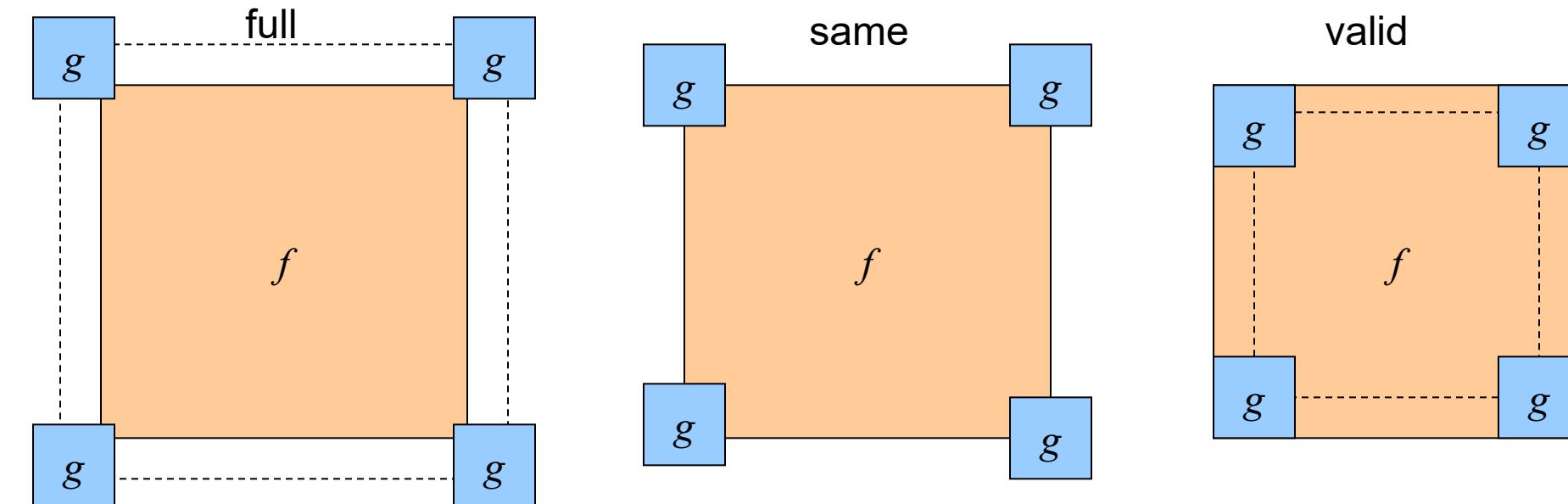
实际问题

- 输出的尺寸是多少？
- Python: `convolve2d(g, f, mode)`
 - *mode* = 'full': 输出大小为 *f* 和 *g* 的大小之和
 - *mode* = 'same': 输出大小与 *f* 相同
 - *mode* = 'valid': 输出大小为 *f* 和 *g* 的大小之差

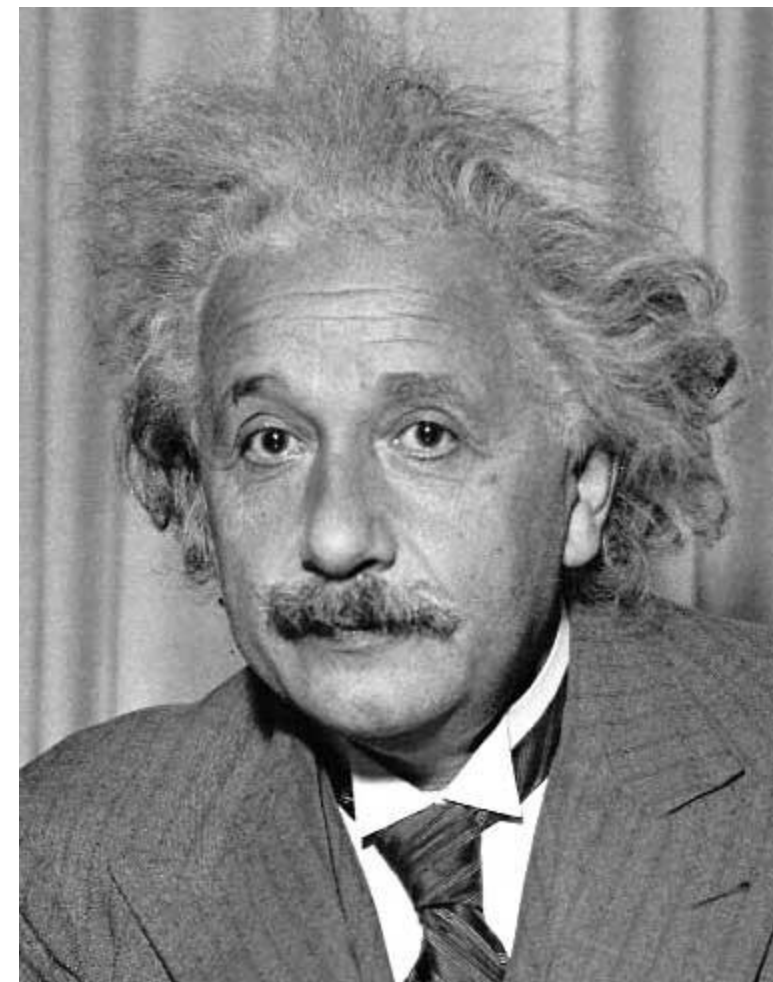


Practical matters

- What is the size of the output?
- Python: `convolve2d(g, f, mode)`
 - *mode* = 'full': output size is sum of sizes of *f* and *g*
 - *mode* = 'same': output size is same as *f*
 - *mode* = 'valid': output size is difference of sizes of *f* and *g*

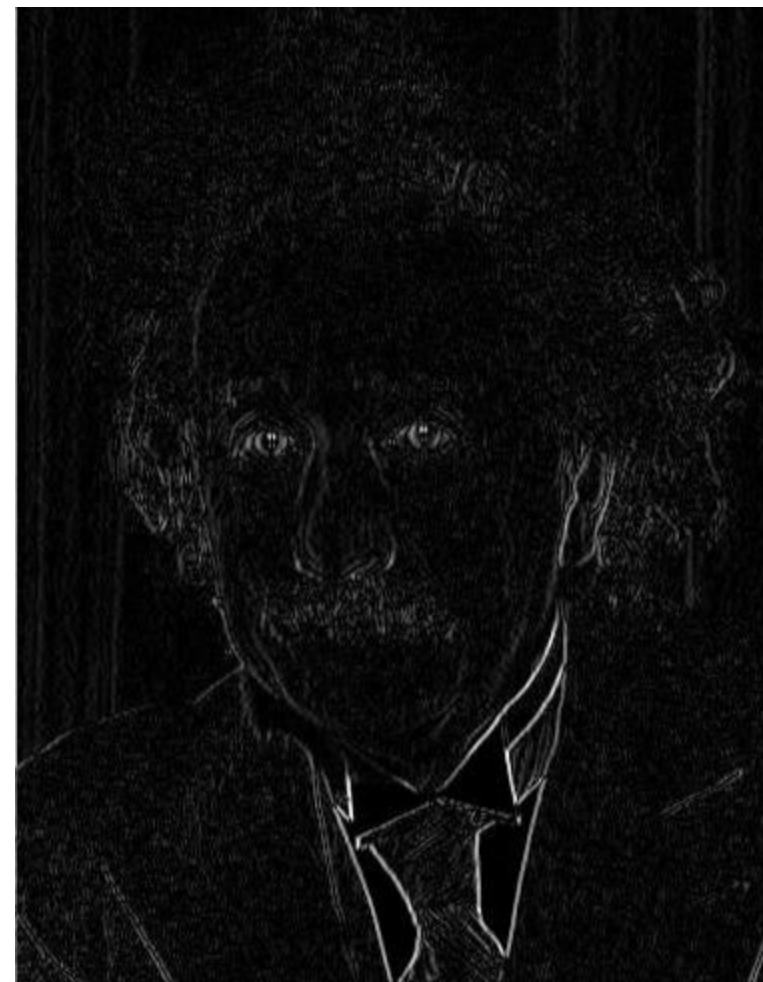


其他滤波器



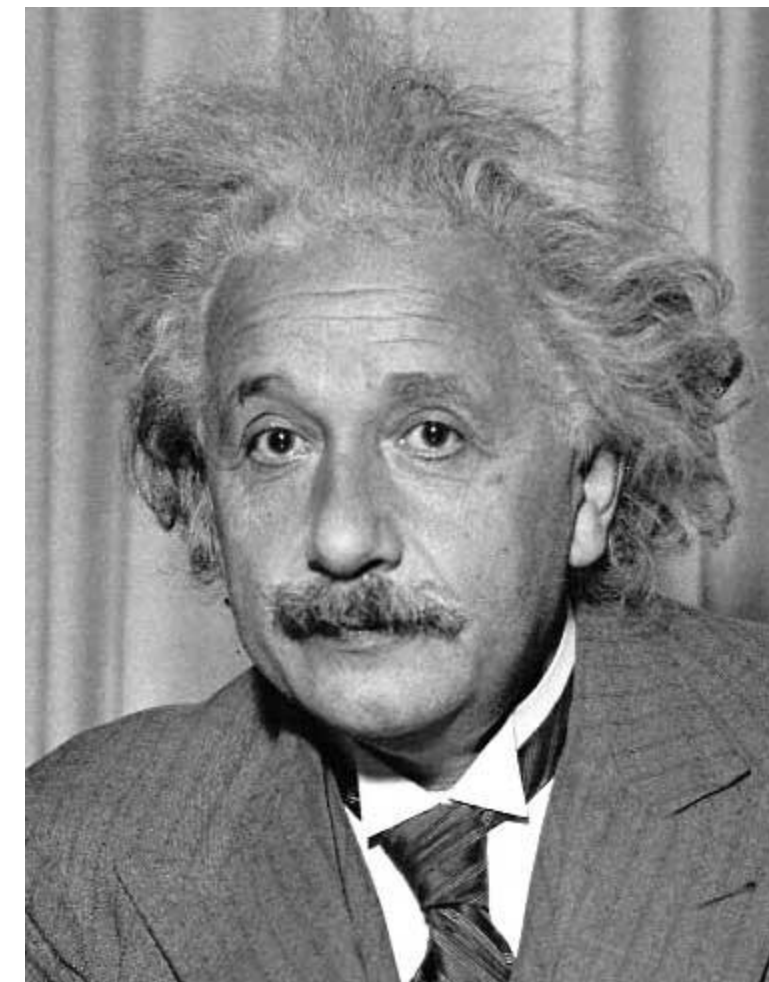
1	0	-1
2	0	-2
1	0	-1

Sobel



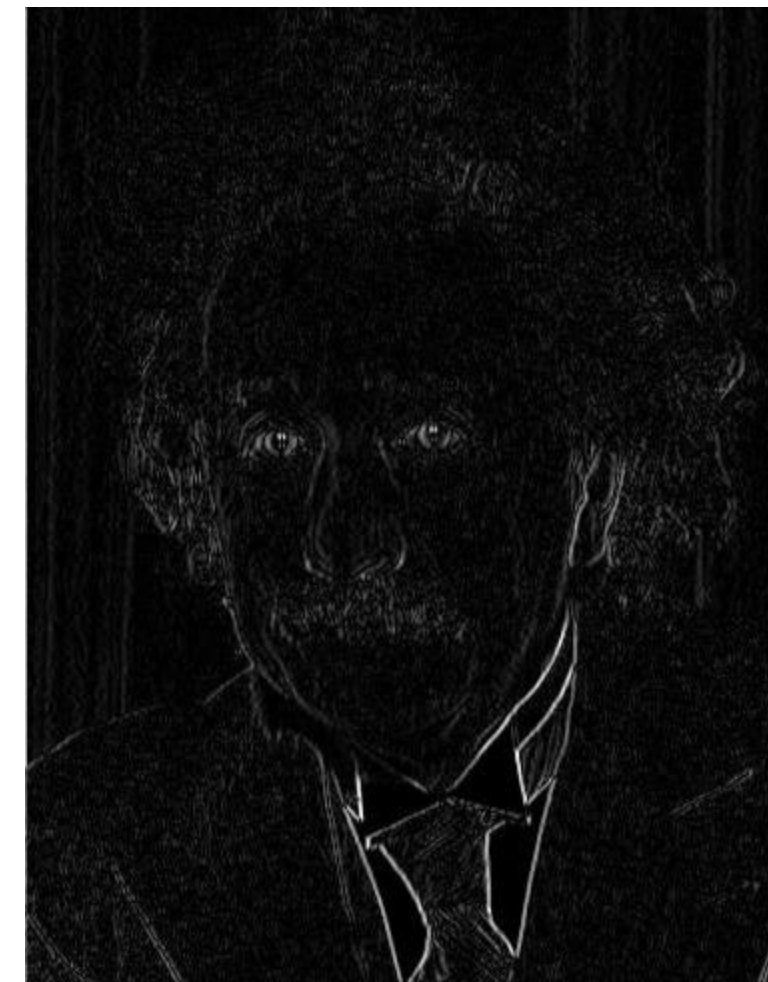
垂直边缘
(绝对值)

Other filters



1	0	-1
2	0	-2
1	0	-1

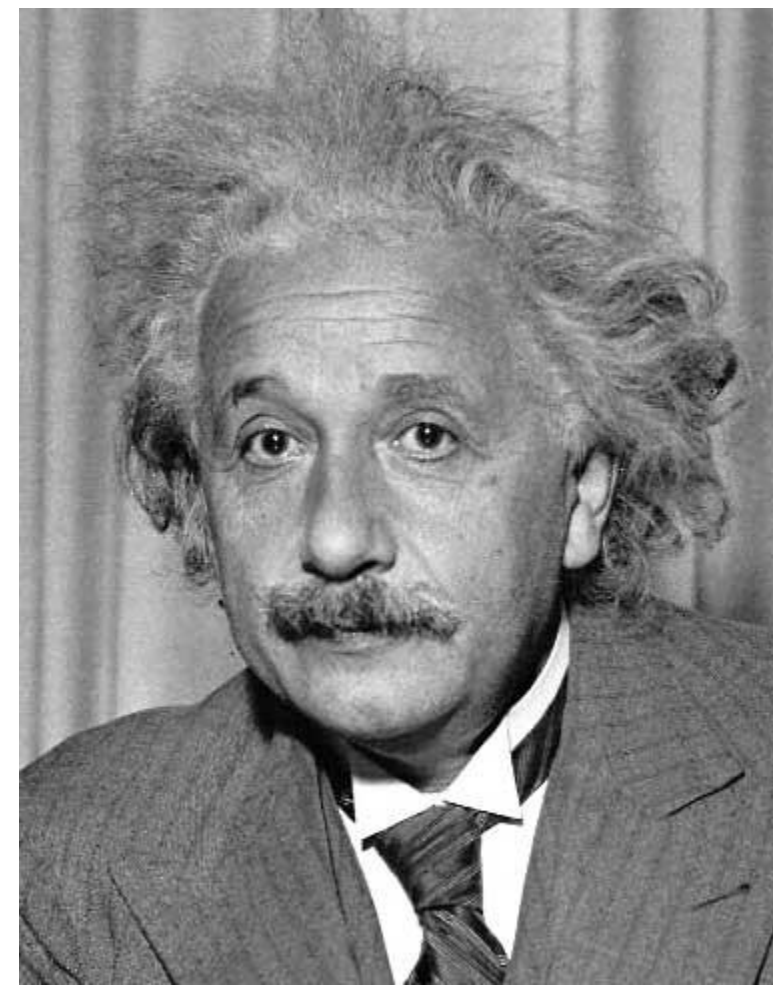
Sobel



Vertical Edge
(absolute value)



其他滤波器



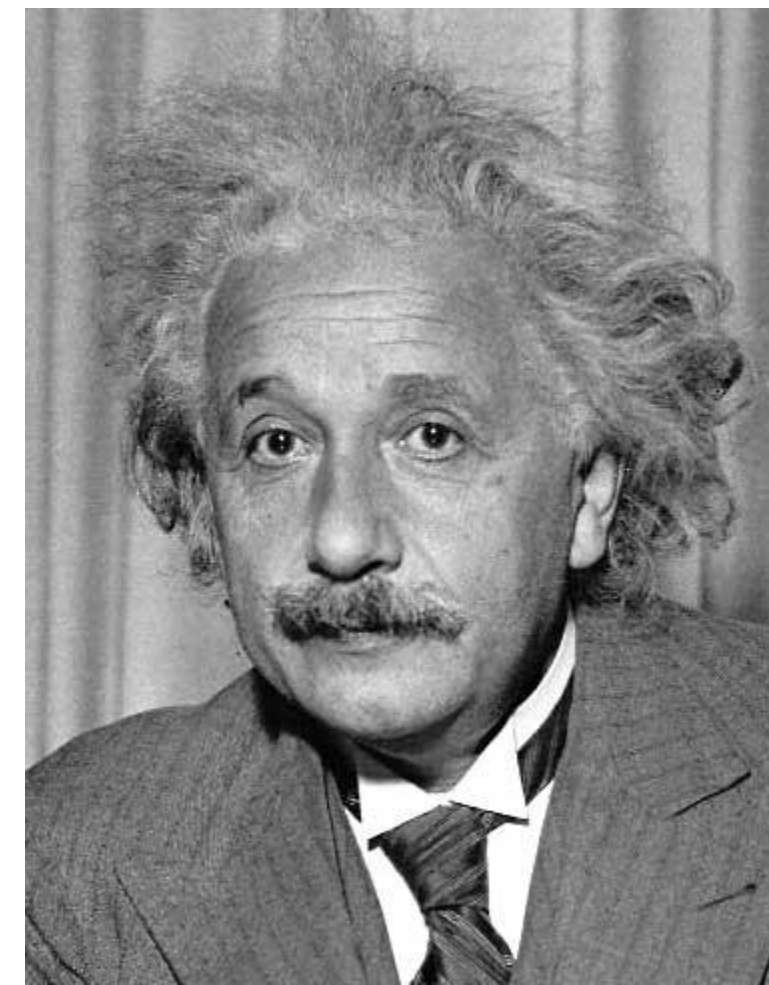
1	2	1
0	0	0
-1	-2	-1

Sobel



水平边缘（绝对值）

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel

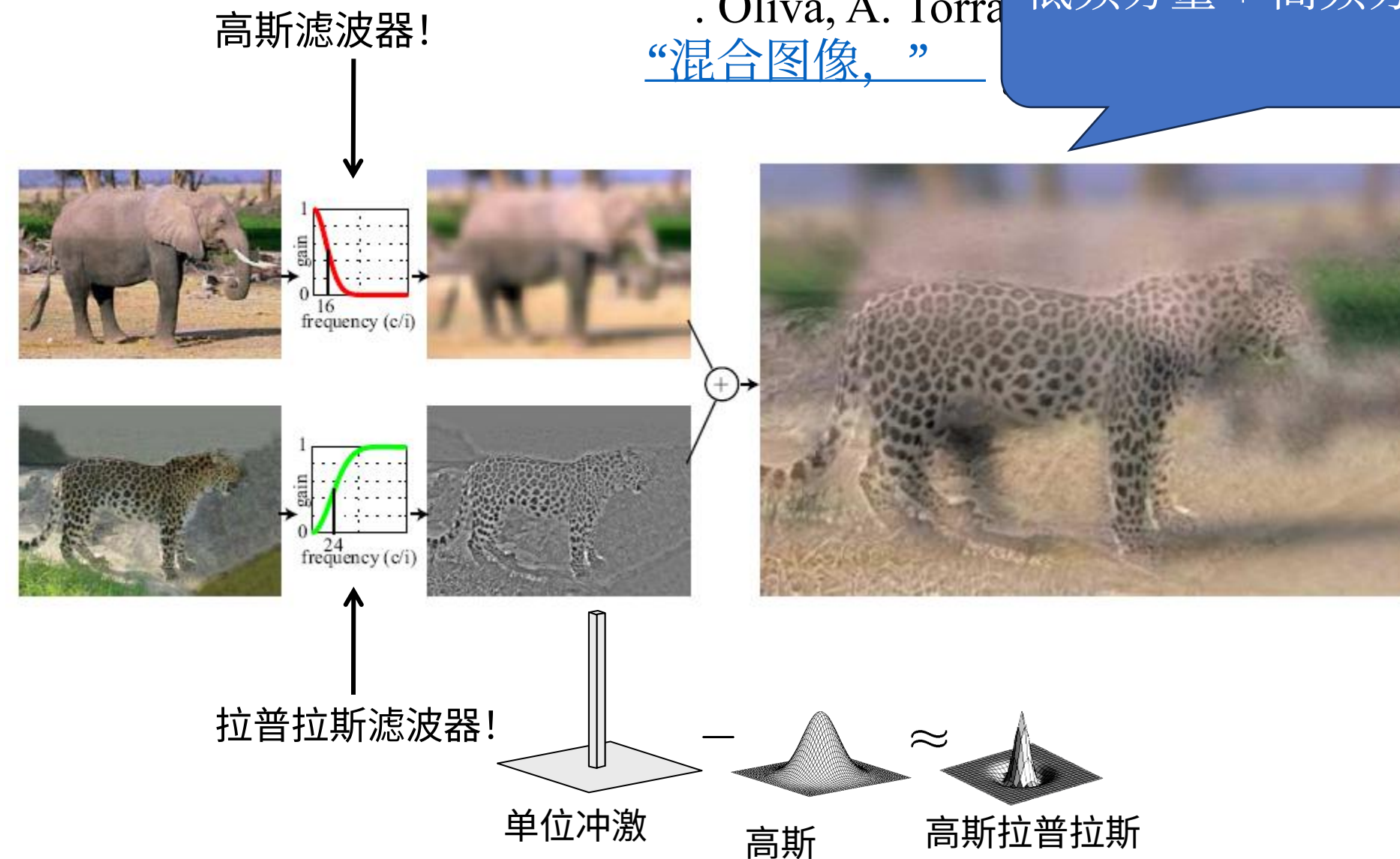


Horizontal Edge
(absolute value)

应用：混合图像^A

. Oliva, A. Torralba
“混合图像,”

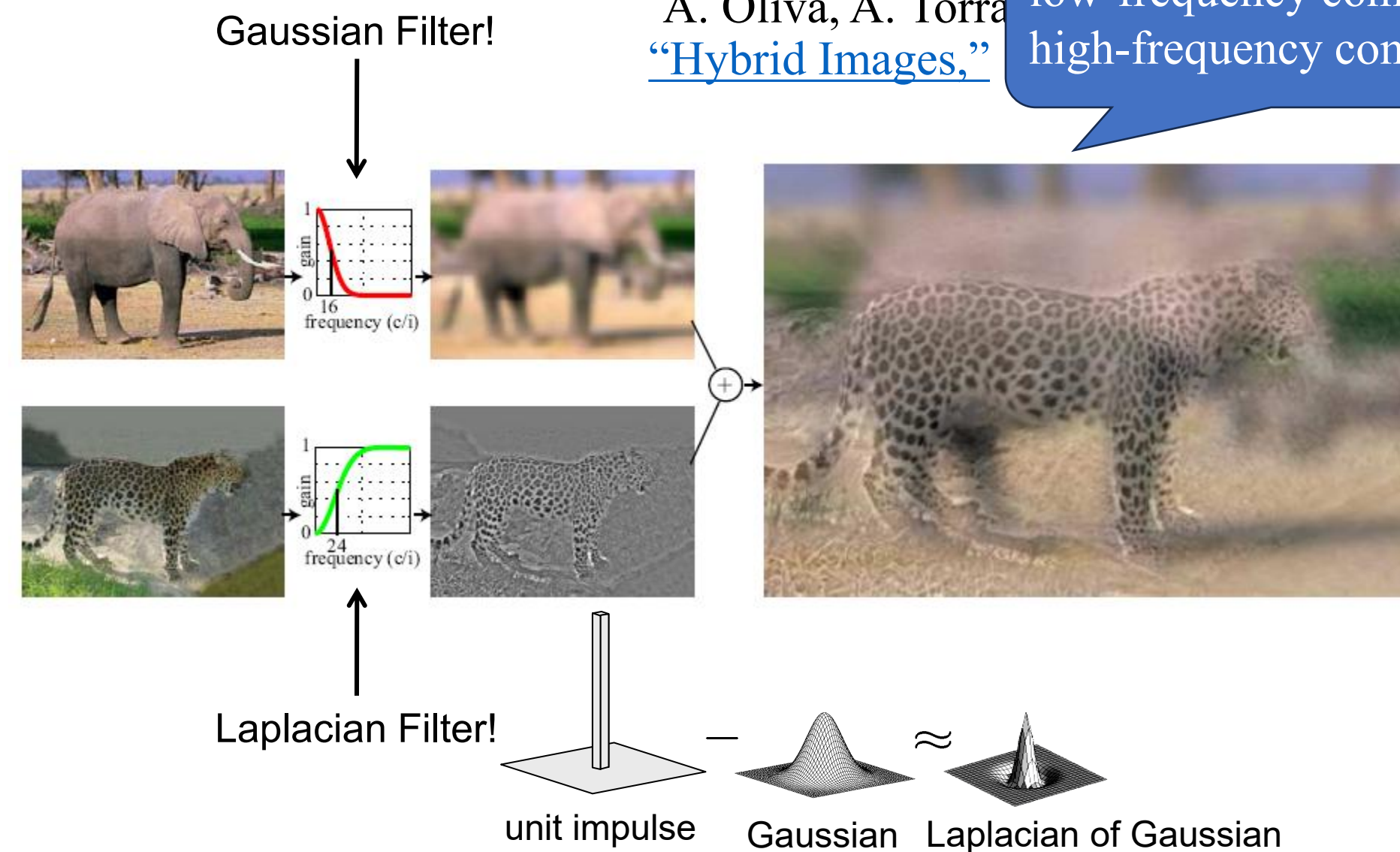
低频分量 + 高频分量



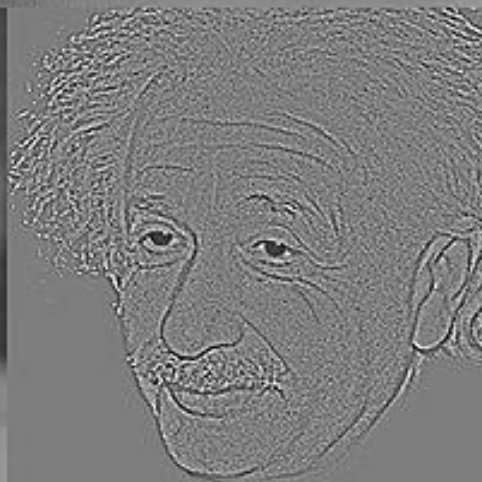
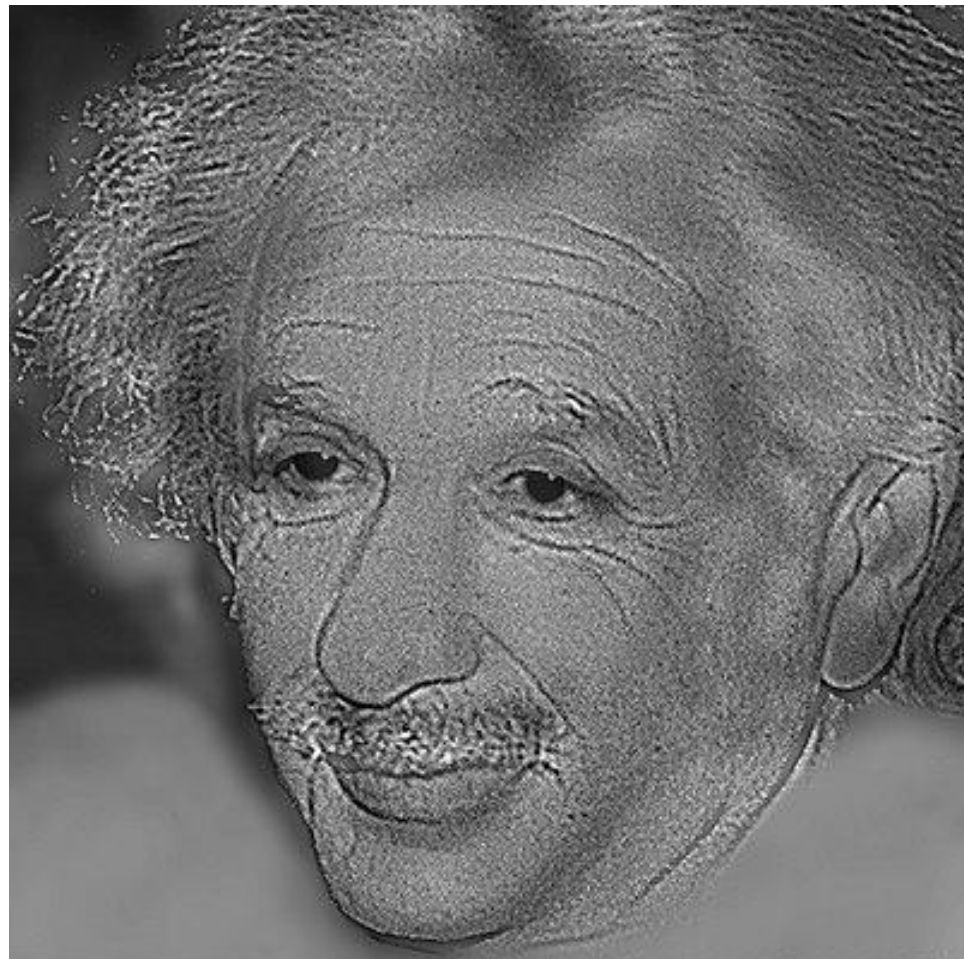
Application: Hybrid Images

A. Oliva, A. Torralba
“Hybrid Images,”

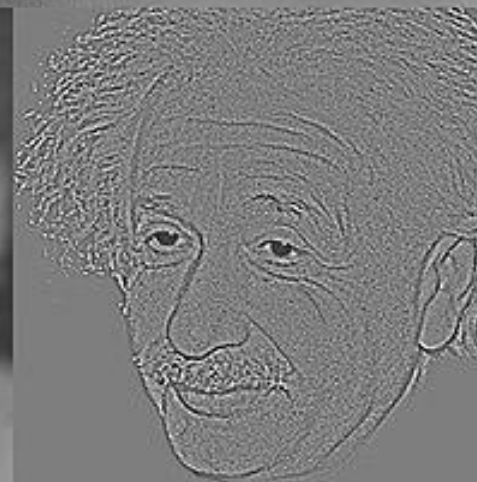
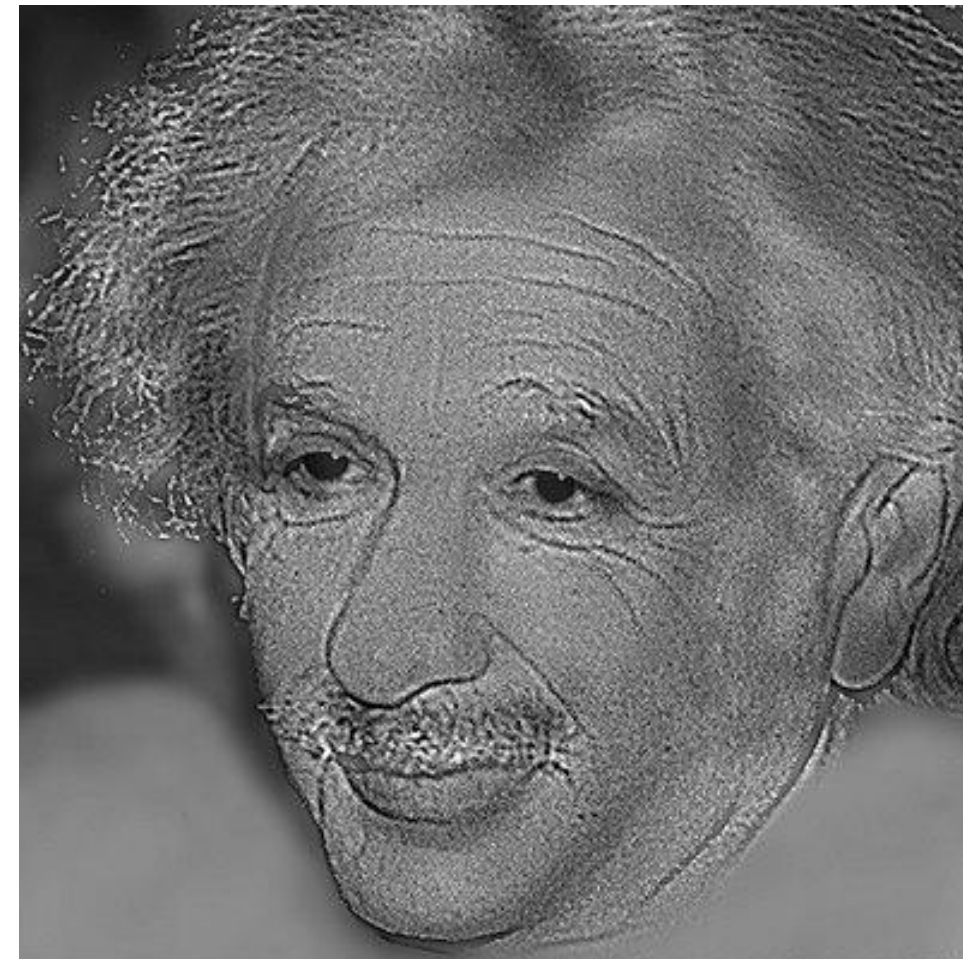
low-frequency components +
high-frequency components







https://en.wikipedia.org/wiki/Hybrid_image



https://en.wikipedia.org/wiki/Hybrid_image

其他滤波器应用

- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔



Other Applications of Filters

- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid



其他滤波器应用

- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔

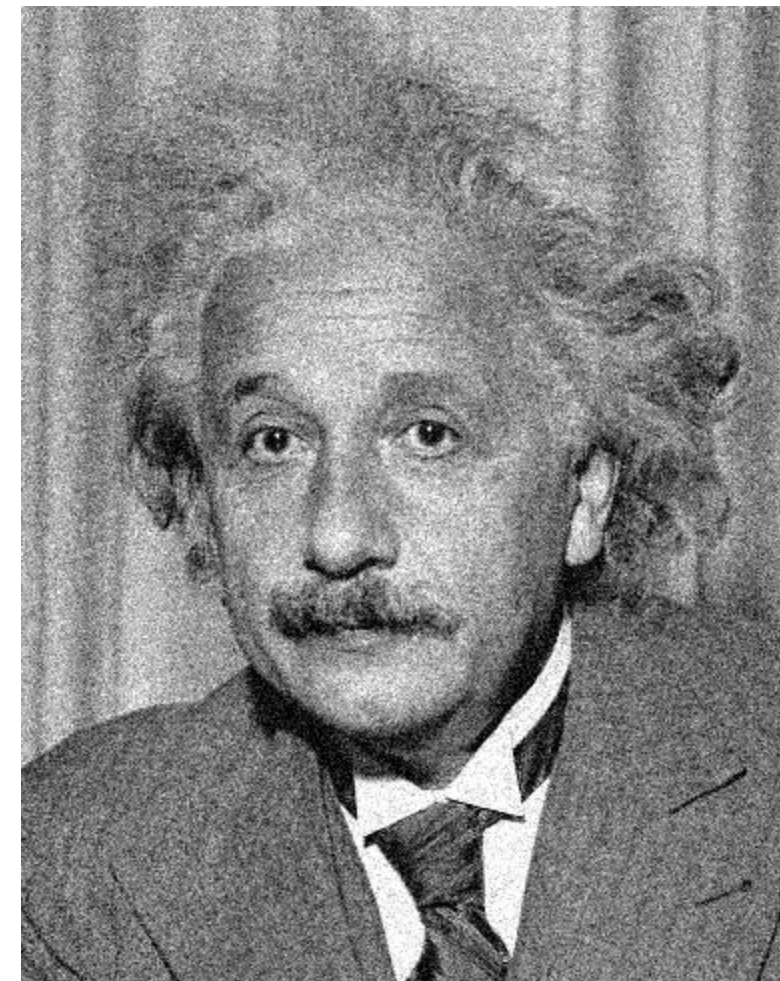


Other Applications of Filters

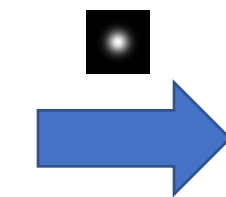
- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid



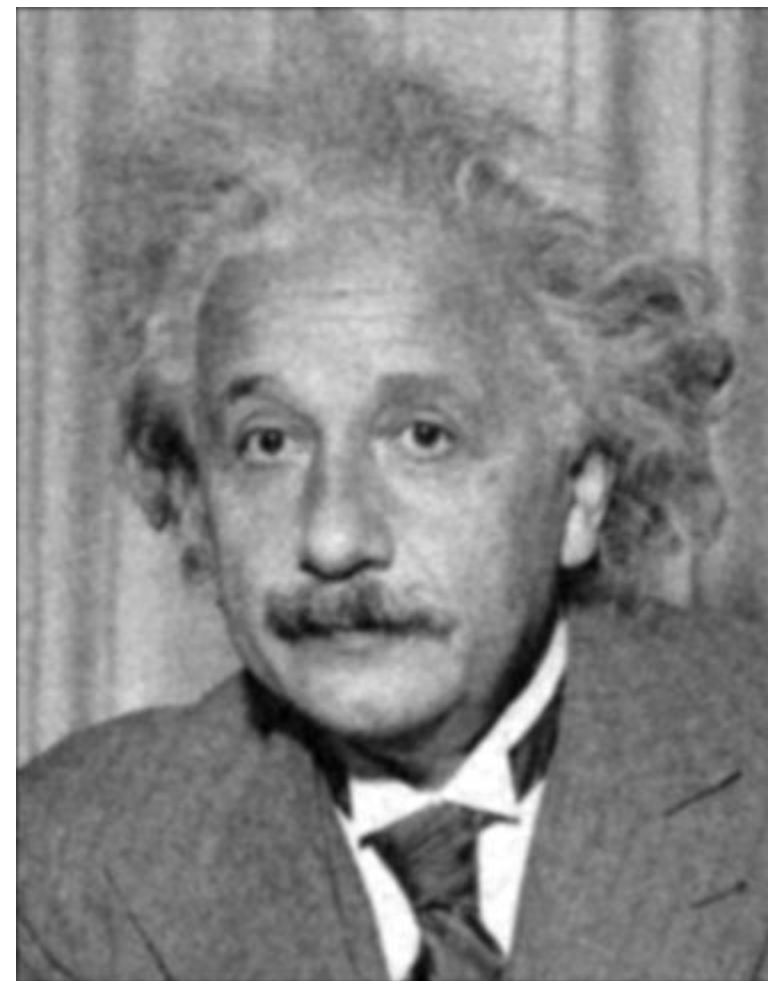
去噪



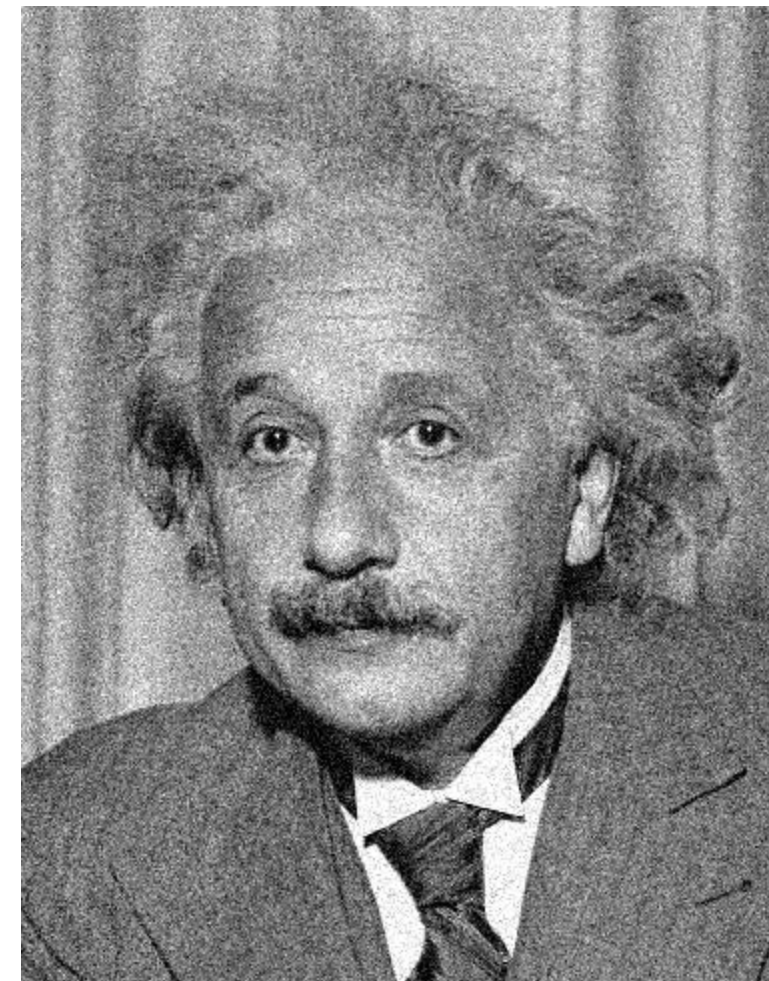
加性高斯噪声



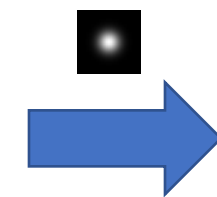
高斯滤波器



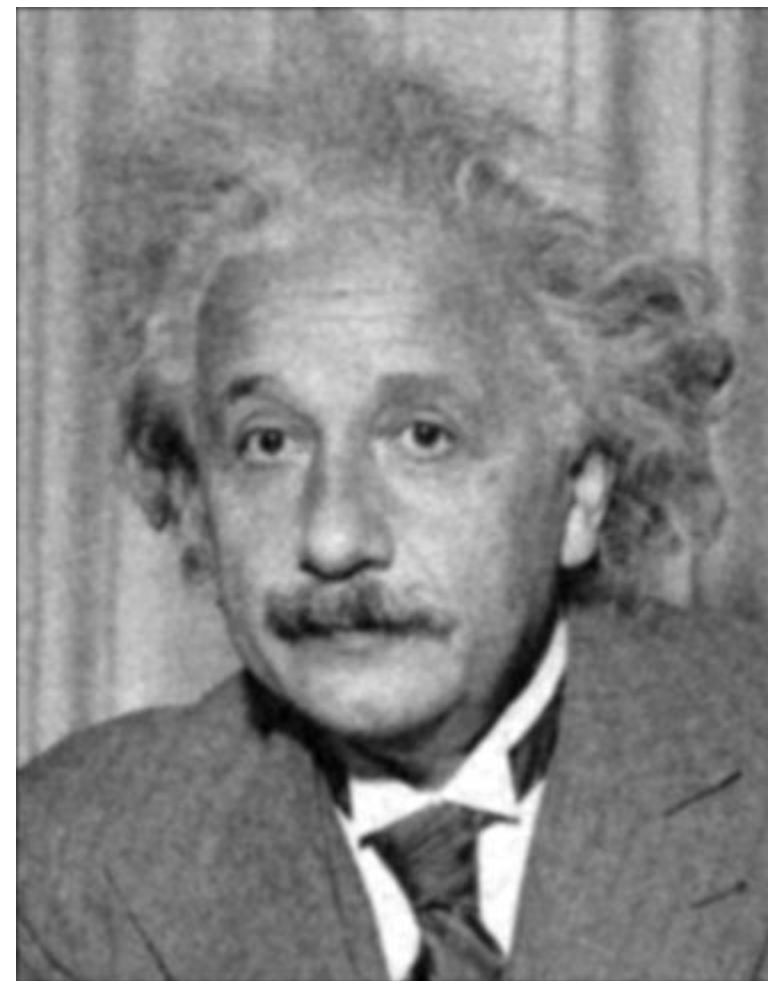
Denoising



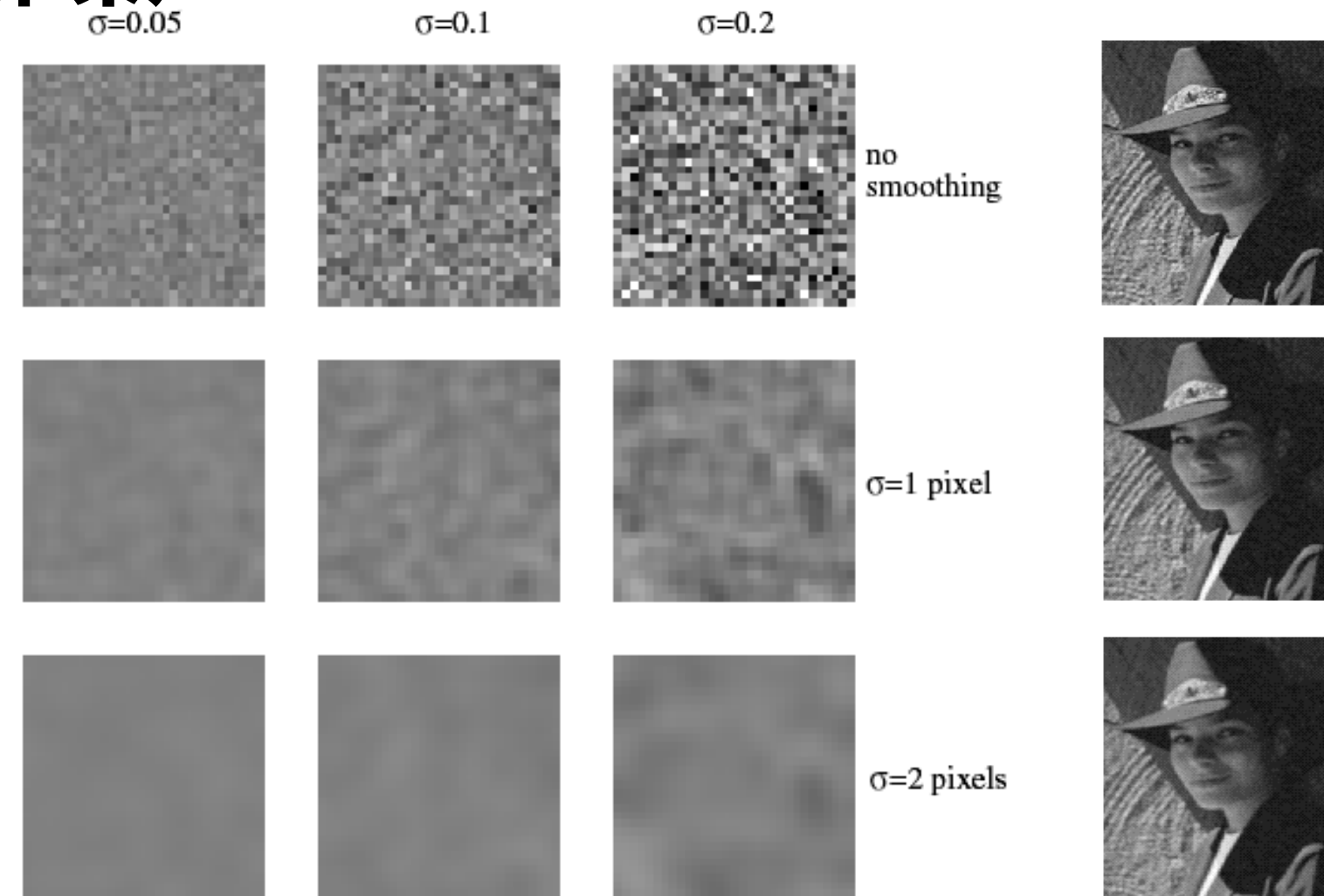
Additive Gaussian Noise



Gaussian
Filter



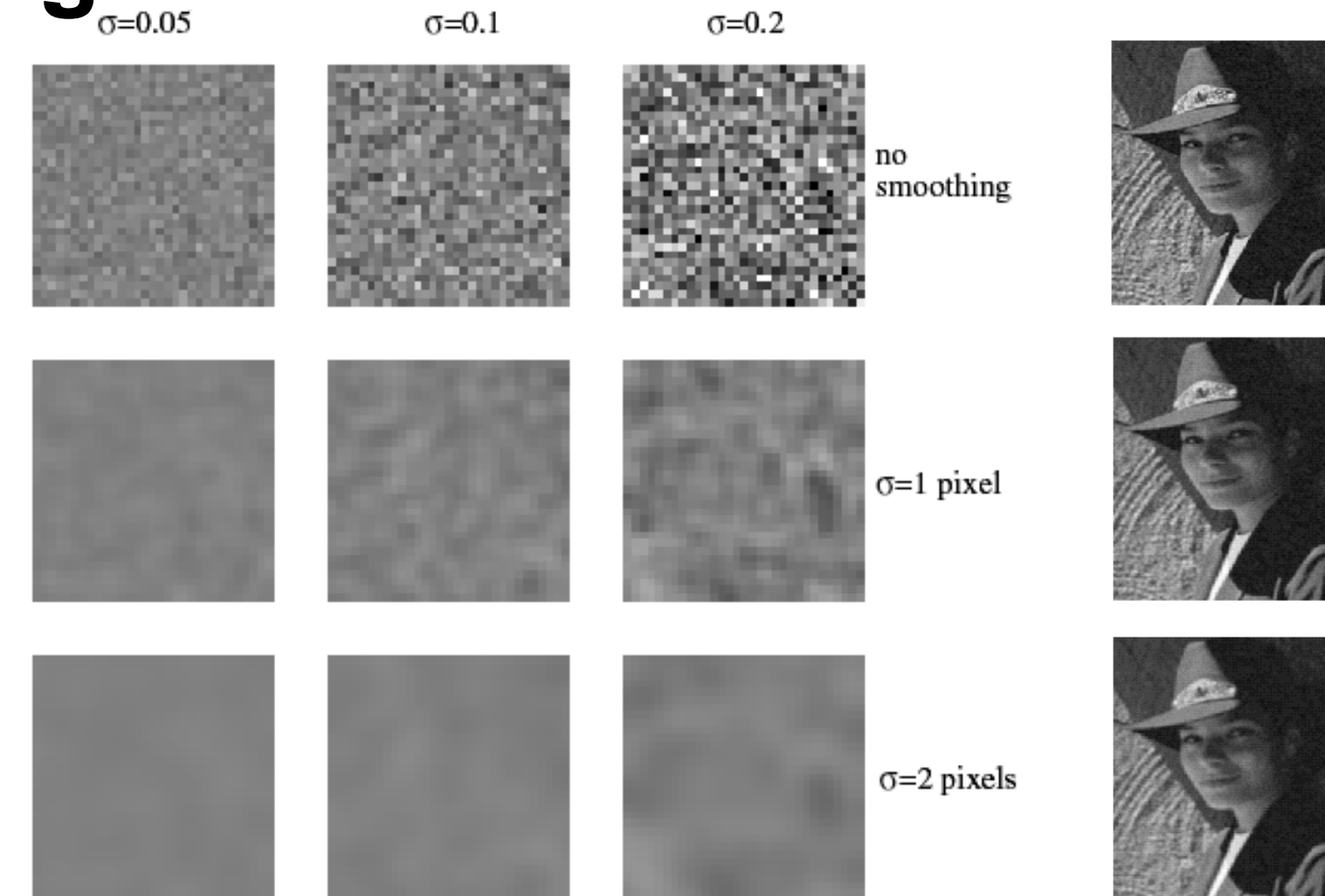
降低高斯噪声



使用较大标准差进行平滑可以抑制噪声，但也会模糊图像

来源：S. Lazebnik

Reducing Gaussian noise



Smoothing with larger standard deviations suppresses noise, but also blurs the image

Source: S. Lazebnik

通过高斯平滑降低椒盐噪声（脉冲噪声）

Reducing salt-and-pepper noise (impulse noise) by Gaussian smoothing

3x3



5x5



7x7



3x3



5x5

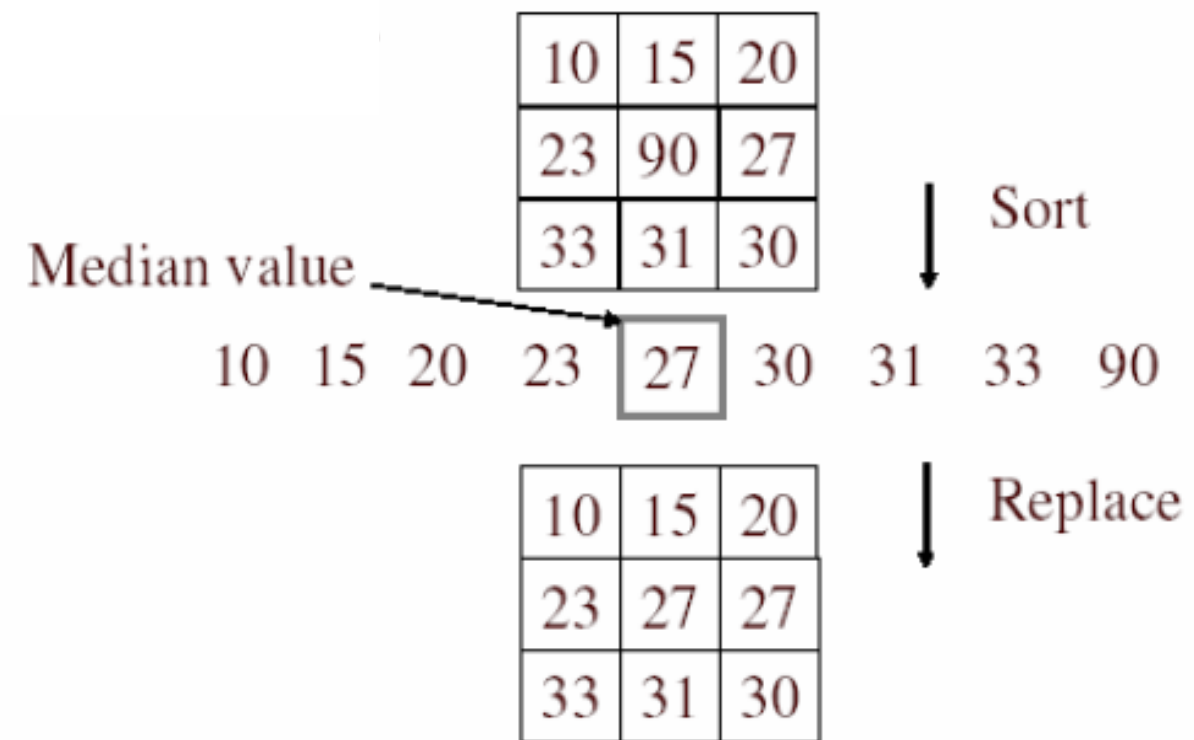


7x7



替代思路：中值滤波

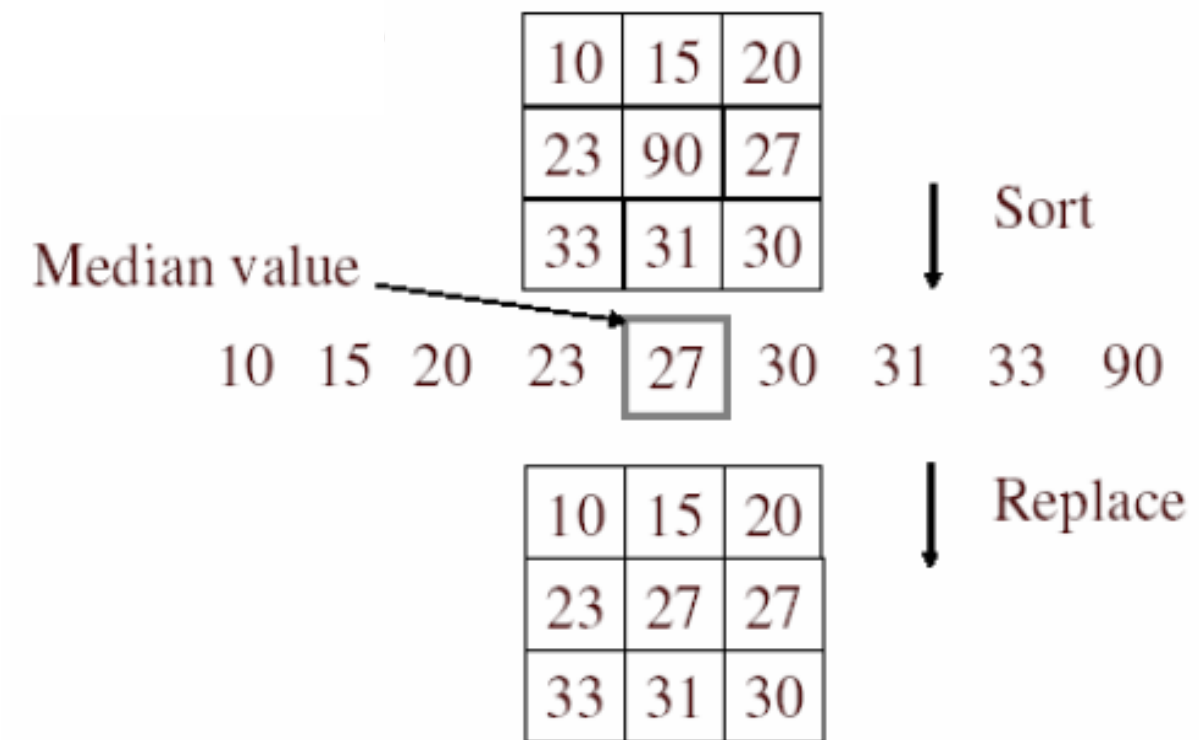
- 中值滤波器通过选择窗口内的中值强度来对窗口进行操作



- 中值滤波是线性的吗？

Alternative idea: Median filtering

- A **median filter** operates over a window by selecting the median intensity in the window

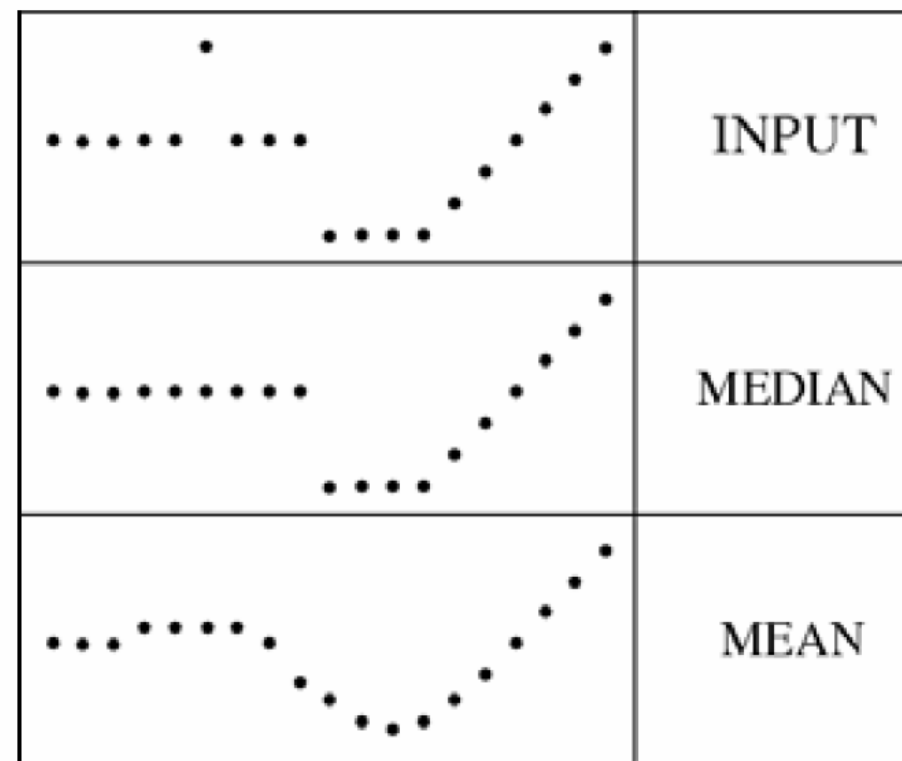


- Is median filtering linear?

中值滤波

- 中值滤波相比高斯滤波有什么优势?
- 对离群值的鲁棒性

filters have width 5 :

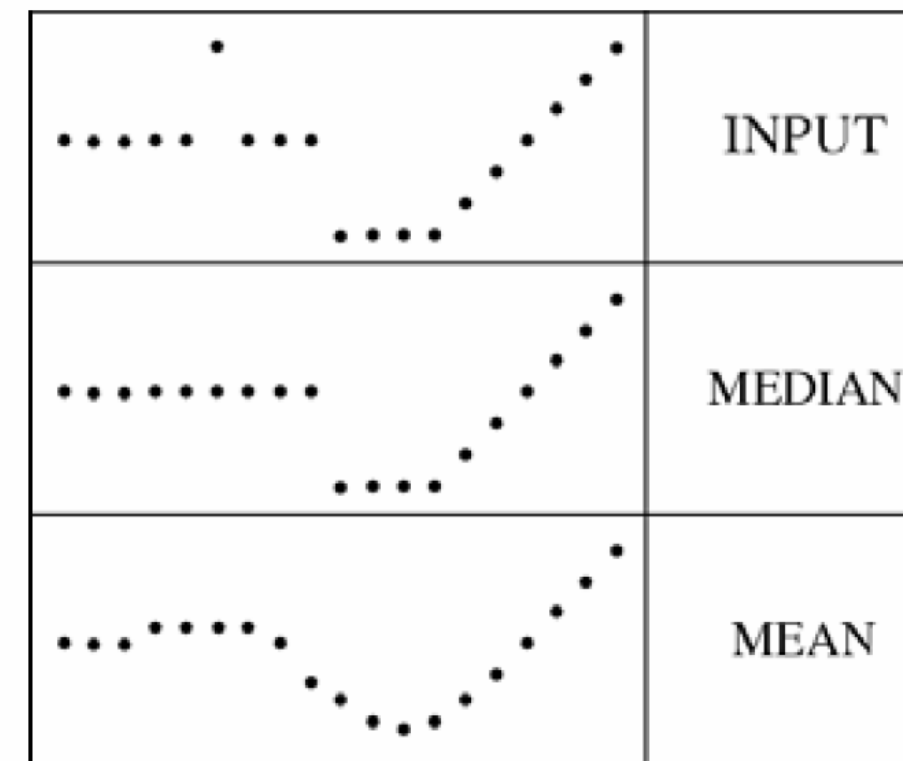


来源：K. Grauman

Median filter

- What advantage does median filtering have over Gaussian filtering?
 - Robustness to outliers

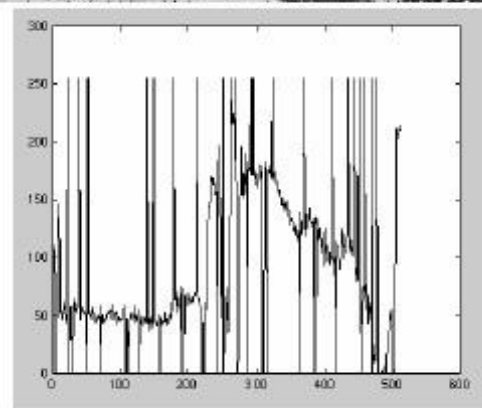
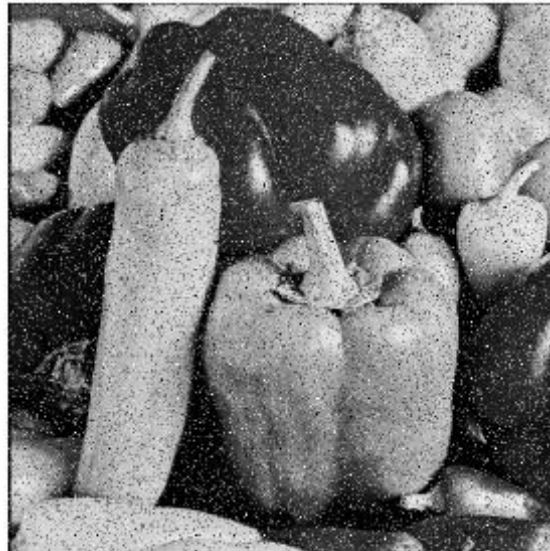
filters have width 5 :



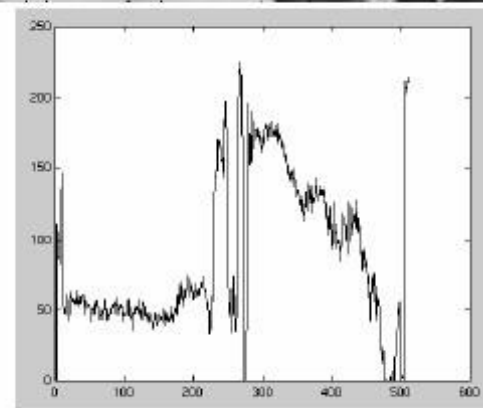
Source: K. Grauman

中值滤波

椒盐噪声



中值滤波后

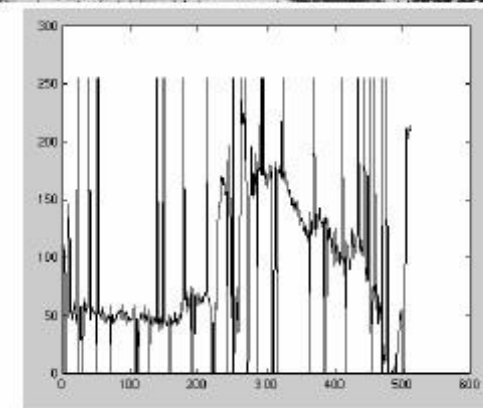
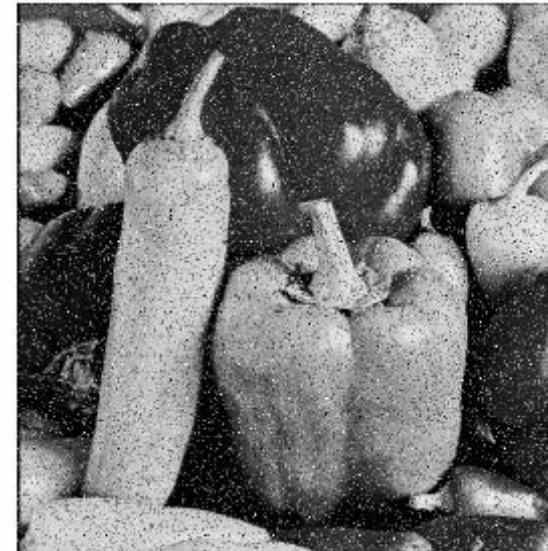


Python: `scipy.ndimage.median_filter (image, size)`

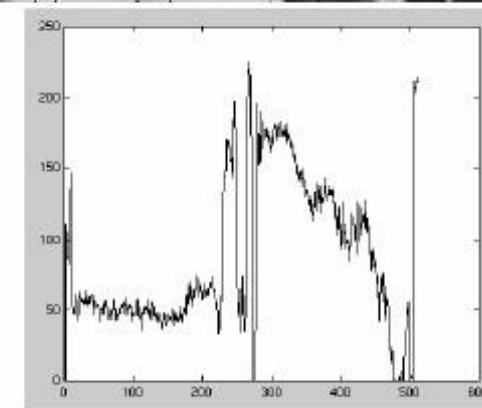
来源: M. Hebert

Median filter

Salt-and-pepper noise



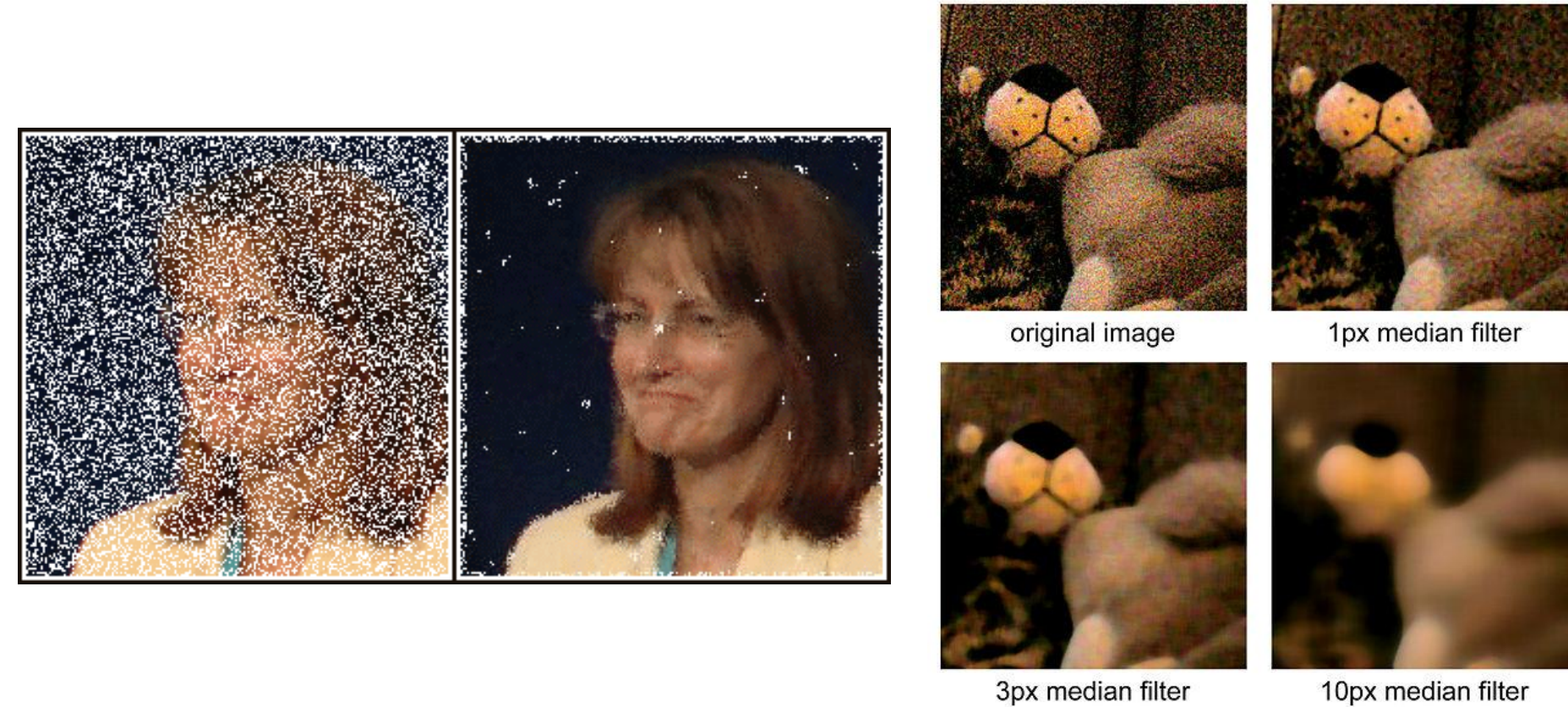
Median filtered



Python: `scipy.ndimage.median_filter (image, size)`

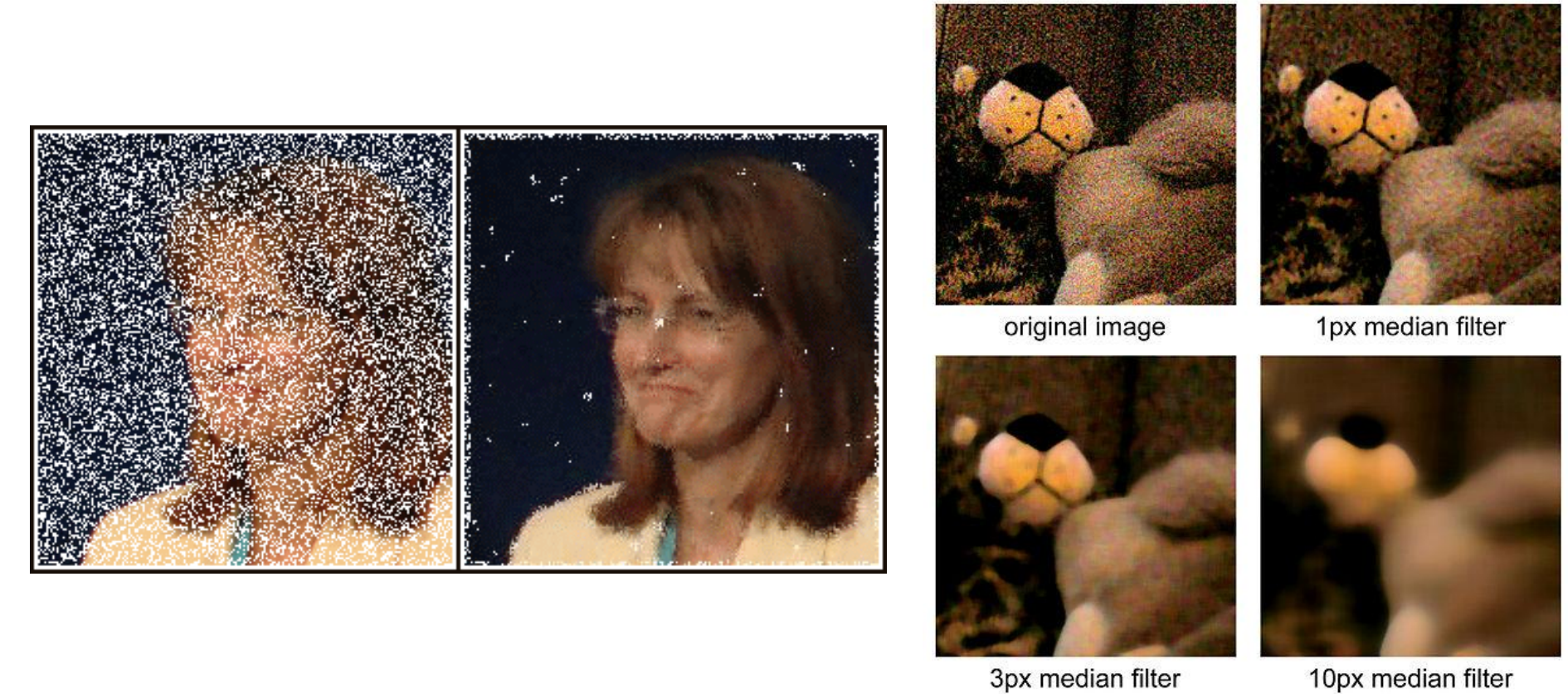
Source: M. Hebert

中值滤波示例



<http://en.wikipedia.org/wiki/File:Medianfilterp.png>
http://en.wikipedia.org/wiki/File:Median_filter_example.jpg

Median Filtered Examples



<http://en.wikipedia.org/wiki/File:Medianfilterp.png>
http://en.wikipedia.org/wiki/File:Median_filter_example.jpg

中值滤波与高斯滤波

3x3

5x5 7x7

高斯



中值



Median vs. Gaussian filtering

3x3

5x5

7x7

Gaussian



Median



其他滤波器选择

- 加权中值（离中心越远的像素权重越小）
- 截尾均值（忽略少数最亮和最暗像素后取平均）
- 双边滤波（根据空间距离和强度差异进行加权）

`cv2.bilateralFilter(size, sigma_color, sigma_spatial)`



双边滤波

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

Diagram illustrating the components of the bilateral filter equation:

- $\frac{1}{W_p}$: Normalization Factor
- $G_{\sigma_s}(\|p - q\|)$: Space Weight (represented by a Gaussian kernel image)
- $G_{\sigma_r}(|I_p - I_q|)$: Range Weight (represented by a range kernel graph)

Other filter choices

- Weighted median (pixels further from center count less)
- Clipped mean (average, ignoring few brightest and darkest pixels)
- Bilateral filtering (weight by spatial distance *and* intensity difference)

`cv2.bilateralFilter(size, sigma_color, sigma_spatial)`



Bilateral filtering

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

Diagram illustrating the components of the bilateral filter equation:

- $\frac{1}{W_p}$: Normalization Factor
- $G_{\sigma_s}(\|p - q\|)$: Space Weight (represented by a Gaussian kernel image)
- $G_{\sigma_r}(|I_p - I_q|)$: Range Weight (represented by a range kernel graph)

大纲：滤波的一些应用

- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔



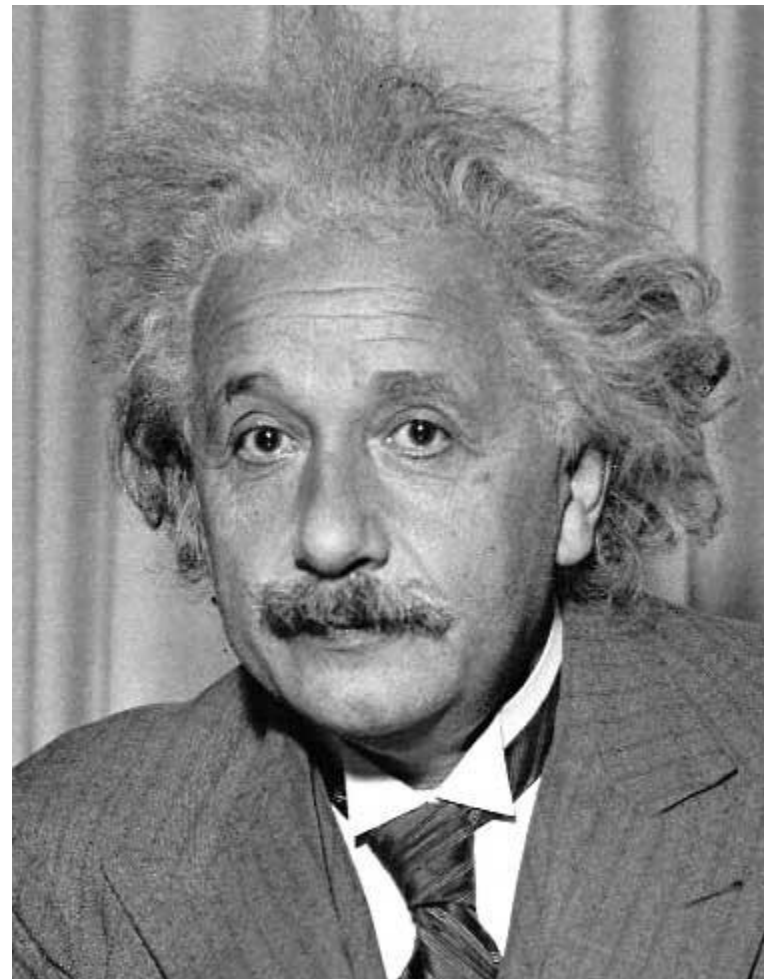
Outline: some applications of filtering

- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid



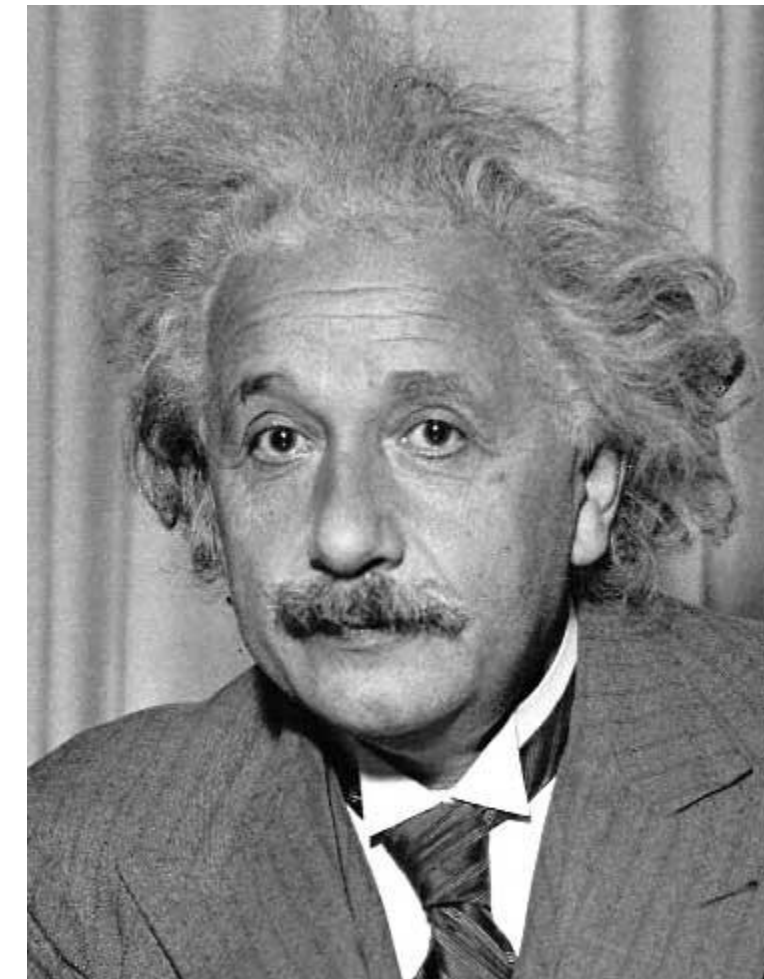
模板匹配

- 目标：在图像中查找
- 主要挑战：两个图像块之间什么是良好的相似度或距离度量？



Template matching

- Goal: find  in image
- Main challenge: What is a good similarity or distance measure between two patches?



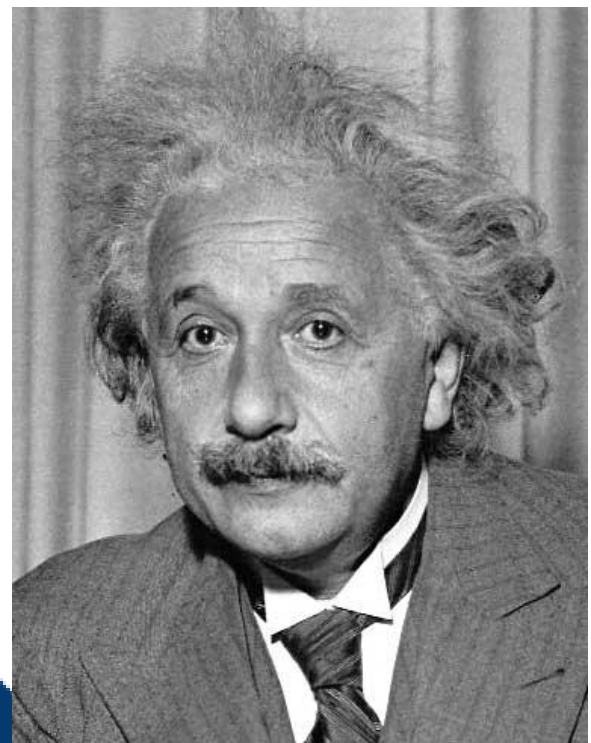
使用滤波器进行匹配

- 目标：在图像中查找
- 方法 0：使用眼睛图像块对图像进行滤波

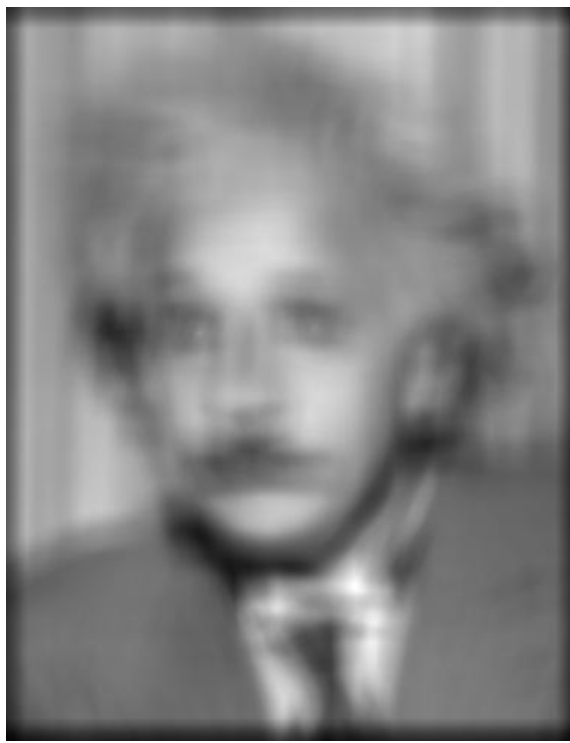
$$h[m,n] = \sum_{k,l} f[k,l] \text{ im}[m+k,n+l]$$

im = 图
像 f = 滤波器

出了问题？



输入



滤波后的图像

亮度偏置：一般明亮的区域会产生高响应，无论它们看起来是否像眼睛，因为大数相乘会得到很大的和。这就是为什么它看起来像原始人脸的一幅明亮而模糊的版本。

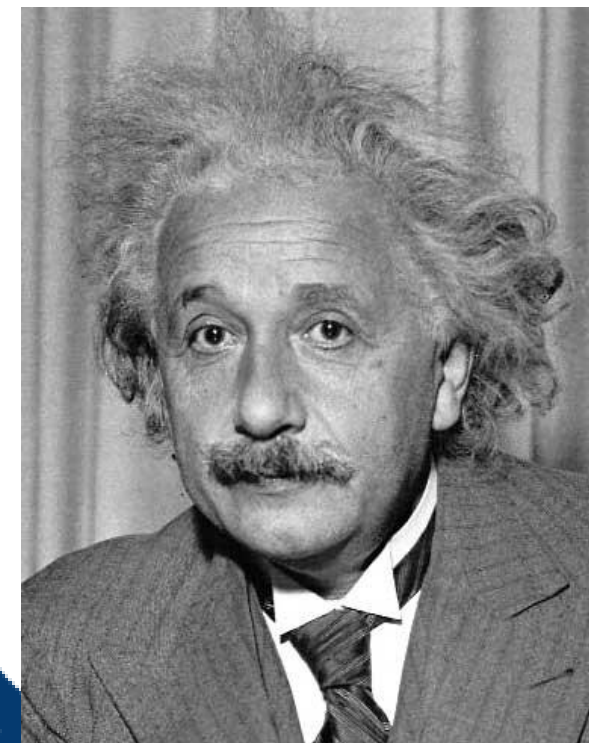
Matching with filters

- Goal: find  in image
- Method 0: filter the image with eye patch

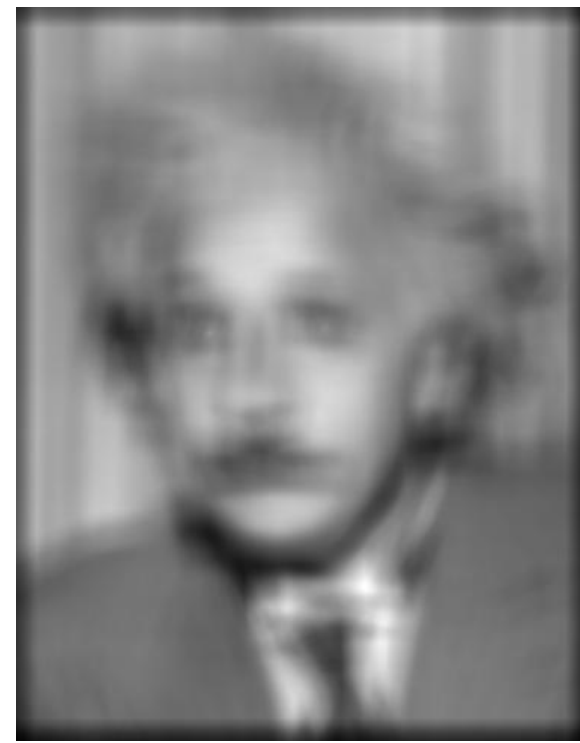
$$h[m,n] = \sum_{k,l} f[k,l] \text{ im}[m+k,n+l]$$

im = image
f = filter

What went wrong?



Input



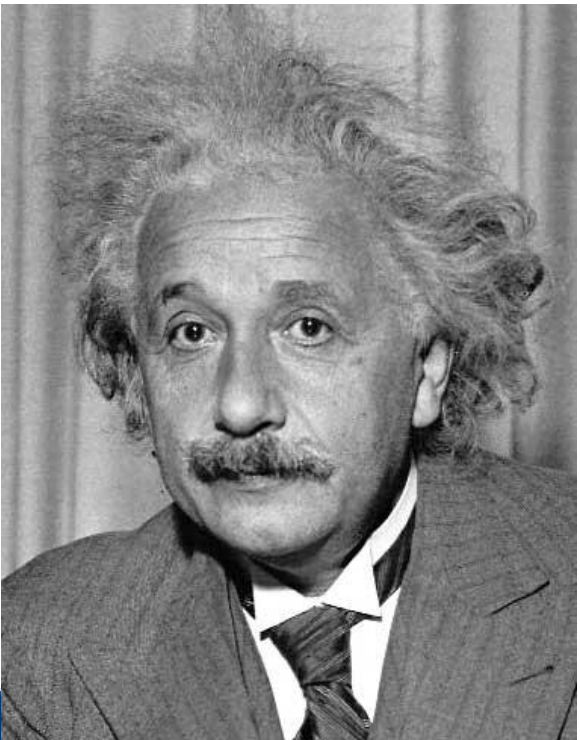
Filtered Image

Brightness Bias: Areas that are generally bright will produce a high response regardless of whether they look like an eye, because large numbers multiplied by large numbers yield a large sum. This is why it looks like a bright, blurry version of the original face.

使用滤波器进行匹配

- 目标：在 图像中查找
- 方法1：用以下滤波器对图像进行滤波：零均值眼睛

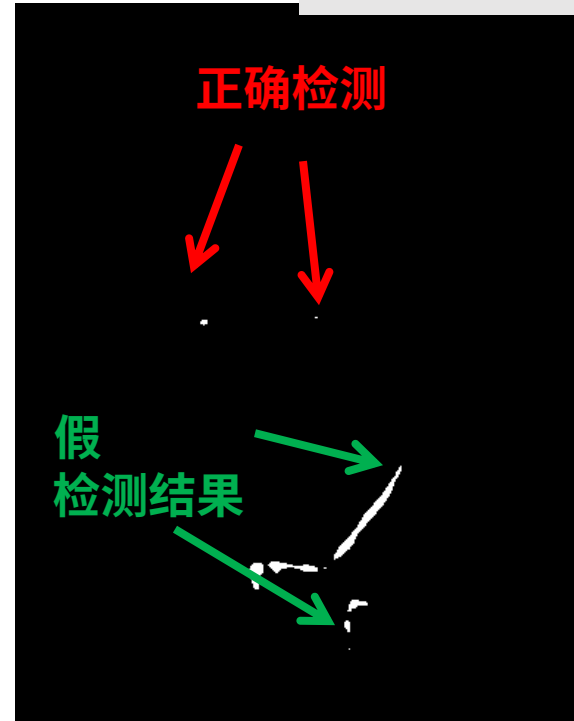
$$h[m,n] = \sum_{k,l} (f[k,l] - \bar{f}) \underset{\substack{\text{mean} \quad \text{滤波器 } f \text{ 的}}}{im[m+k,n+l]}$$



输入



滤波后图像（缩放）



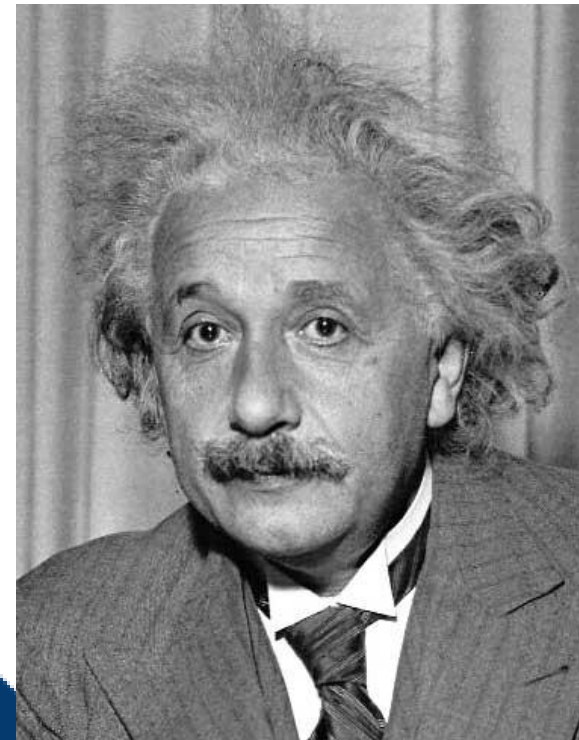
阈值化图像

- **模式优先于亮度**：通过减去平均强度 ($\bar{f}$)，滤波器只有在明暗的相对模式匹配时才会给出强烈的正响应。
- **消除均匀区域**：在完全平坦且明亮的区域（例如额头）中，零均值滤波器的求和结果为零，从而实际上忽略该区域。
- **真实检测**：这使“阈值化图像”能够在眼睛位置精确显示清晰的小点，而不是在整个面部呈现发光斑块。

Matching with filters

- Goal: find  in image
- Method 1: filter the image with zero-mean eye

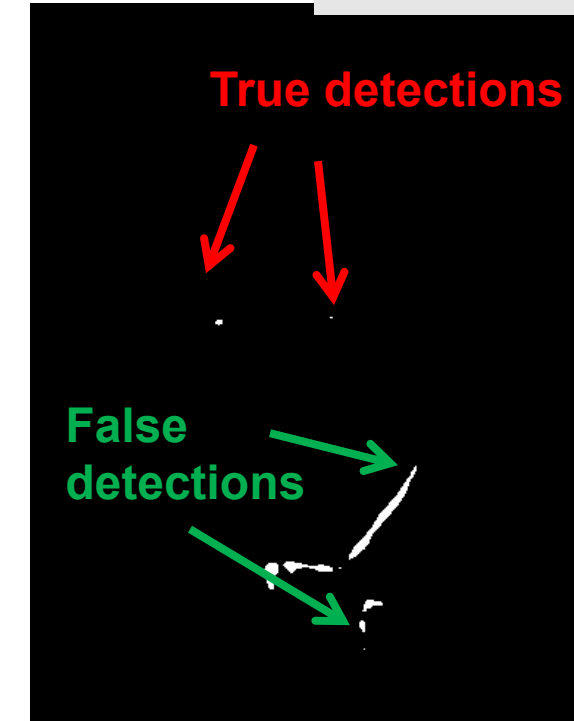
$$h[m,n] = \sum_{k,l} (f[k,l] - \bar{f}) \underset{\substack{\text{mean of filter } f}}{im[m+k,n+l]}$$



Input



Filtered Image (scaled)



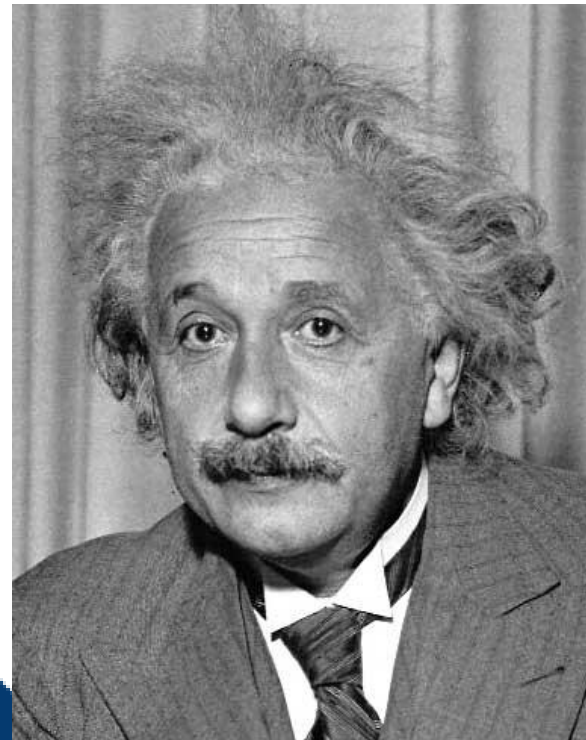
Thresholded Image

- **Pattern Over Brightness**: By subtracting the mean intensity ($\bar{f}$), the filter only gives a strong positive response when the *relative* patterns of light and dark match.
- **Canceling Uniform Areas**: In a perfectly flat, bright area (like a forehead), a zero-mean filter will sum to zero, effectively ignoring the area.
- **True Detections**: This allows the "Thresholded Image" to show sharp dots specifically at the eye locations, rather than glowing blobs across the whole face.

基于滤波器的匹配

- 目标：在图像中查找 
- 方法2：平方距离和（SSD）

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$



输入



1- sqrt(SSD)

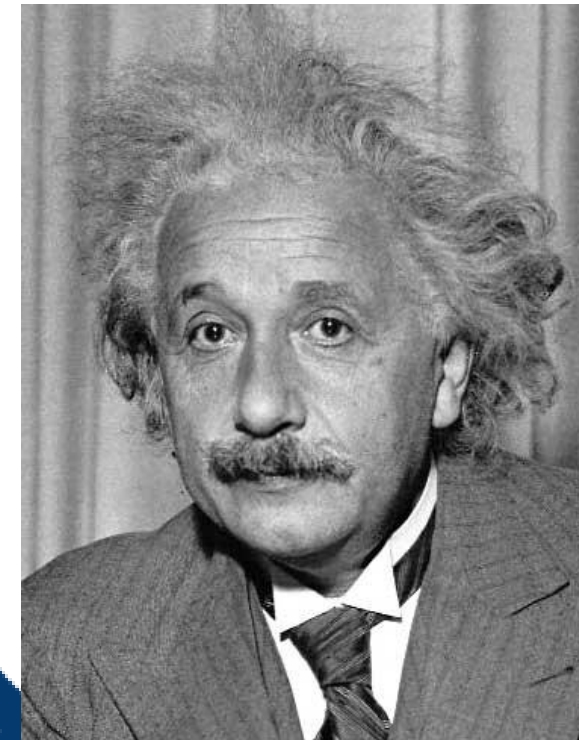


阈值化图像

Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

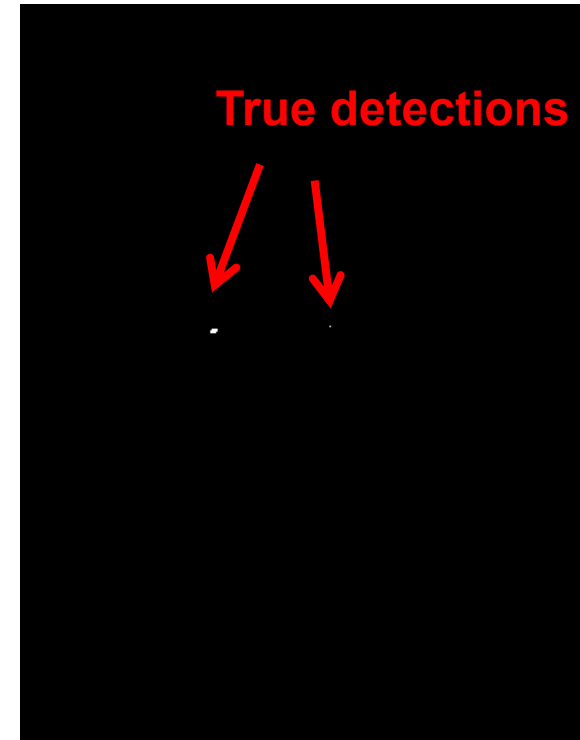
$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$



Input



1- sqrt(SSD)



Thresholded Image

使用滤波器进行匹配

SSD 是否可以用线性滤波器实现?

滤波要快得多!

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

$$h[m,n] = \sum_{k,l} (f[k,l]^2 - 2 \cdot im[m+k,n+l] \cdot f[k,l] + im[m+k,n+l]^2)$$

$$h[m,n] = \sum_{k,l} f[k,l]^2 - 2 \sum_{k,l} im[m+k,n+l] \cdot f[k,l] + \sum_{k,l} im[m+k,n+l]^2$$

$$h = \sum_{k,l} f[k,l]^2 - 2 \text{filter}(im, f) + \text{filter}(im.^2, \text{ones}(f.\text{shape}))$$

常数

线性滤波器

逐元素平方 f, 然后求和
使用大小为 f 的全 1 核

Matching with filters

Can SSD be implemented with linear filters?

Filtering is much faster!

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

$$h[m,n] = \sum_{k,l} (f[k,l]^2 - 2 \cdot im[m+k,n+l] \cdot f[k,l] + im[m+k,n+l]^2)$$

$$h[m,n] = \sum_{k,l} f[k,l]^2 - 2 \sum_{k,l} im[m+k,n+l] \cdot f[k,l] + \sum_{k,l} im[m+k,n+l]^2$$

$$h = \sum_{k,l} f[k,l]^2 - 2 \text{filter}(im, f) + \text{filter}(im.^2, \text{ones}(f.\text{shape}))$$

constant

linear filter

Element-wise square f, then sum
with ones kernel of size f

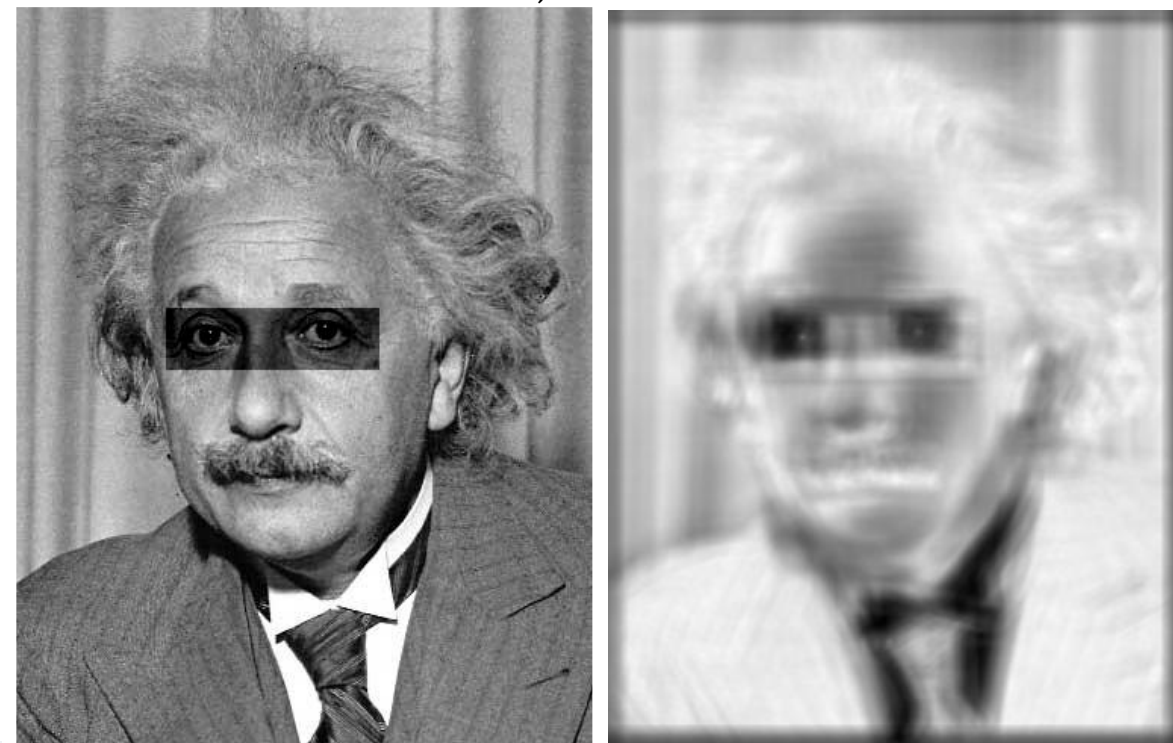
使用滤波器进行匹配

- 目标：在图中查找
- 方法2：平方距离和 (SSD)

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

SSD 的潜在缺点是什么？

对强度偏移敏感



输入

1- sqrt(SSD)

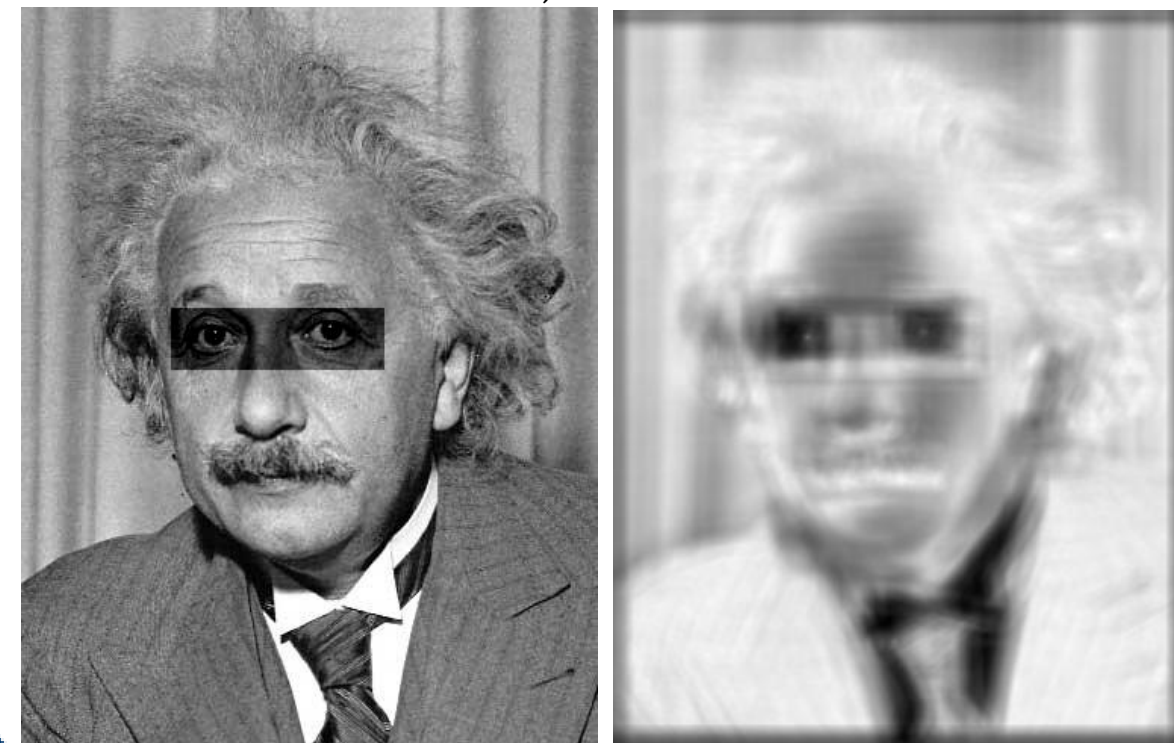
Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

What's the potential downside of SSD?

Sensitive to intensity shift



Input

1- sqrt(SSD)

使用滤波器进行匹配

- 目标: 在像中查找
- 方法3: 归一化互相关

$$h[m, n] = \frac{\sum (f - \bar{f})(im - \overline{im})}{\sqrt{\sum (f - \bar{f})^2 \sum (im - \overline{im})^2}}$$

余弦相似度!

$$h[m, n] = \frac{\sum_{k,l} (f[k, l] - \bar{f})(im[m + k, n + l] - \overline{im_{m,n}})}{(\sum_{k,l} (f[k, l] - \bar{f})^2 \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}})^2)^{0.5}}$$

模板均值 图像块均值
↓ ↓

(除以模板与图像块的标准差之积)

Python: `cv2.matchTemplate(im, template, cv2.TM_CCOEFF_NORMED)`



Matching with filters

- Goal: find  in image
- Method 3: Normalized cross-correlation

$$h[m, n] = \frac{\sum (f - \bar{f})(im - \overline{im})}{\sqrt{\sum (f - \bar{f})^2 \sum (im - \overline{im})^2}}$$

Cosine Similarity!

$$h[m, n] = \frac{\sum_{k,l} (f[k, l] - \bar{f})(im[m + k, n + l] - \overline{im_{m,n}})}{(\sum_{k,l} (f[k, l] - \bar{f})^2 \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}})^2)^{0.5}}$$

mean template mean image patch
↓ ↓

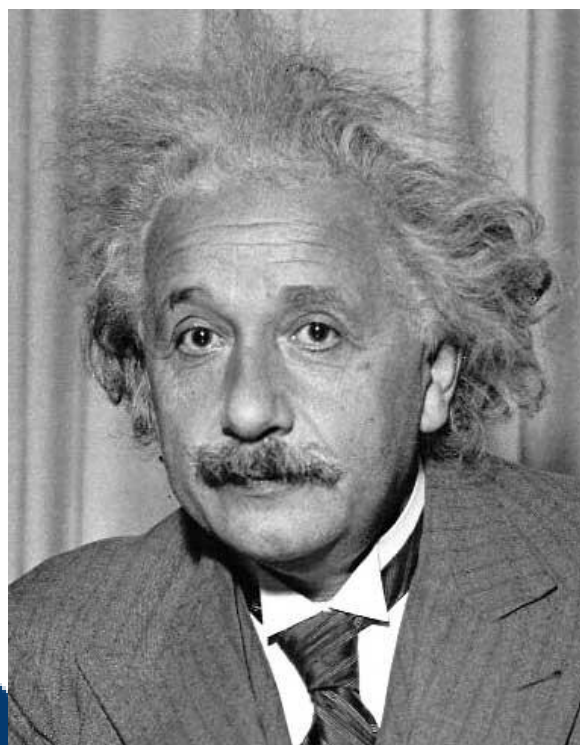
(divide by product of standard deviations of template and image patch)

Python: `cv2.matchTemplate(im, template, cv2.TM_CCOEFF_NORMED)`



使用滤波器进行匹配

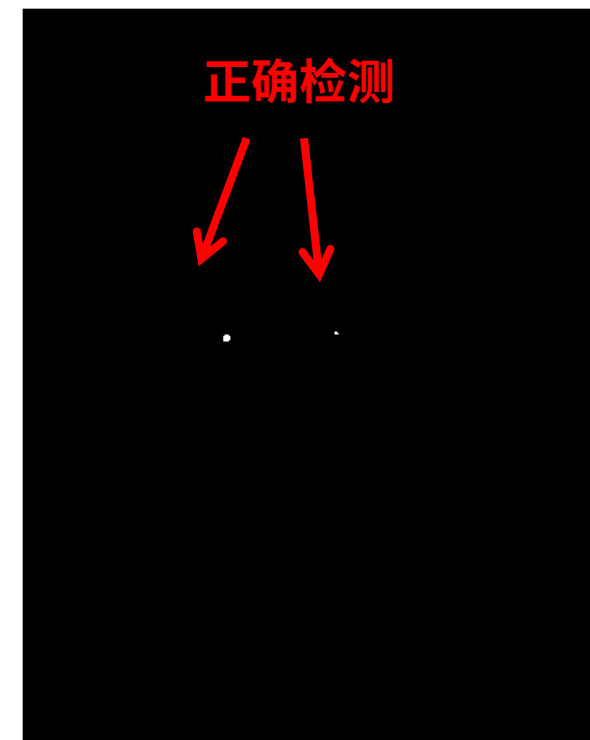
- 目标：在图像中查找
- 方法3：归一化互相关



输入



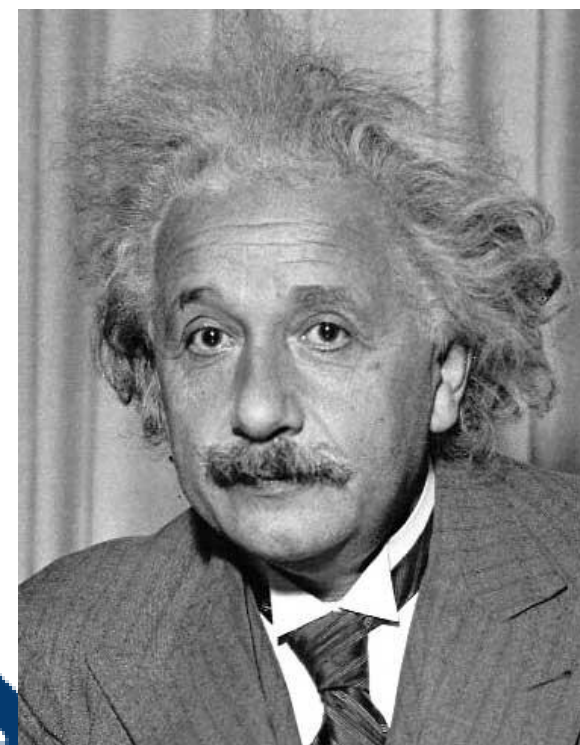
归一化互相关



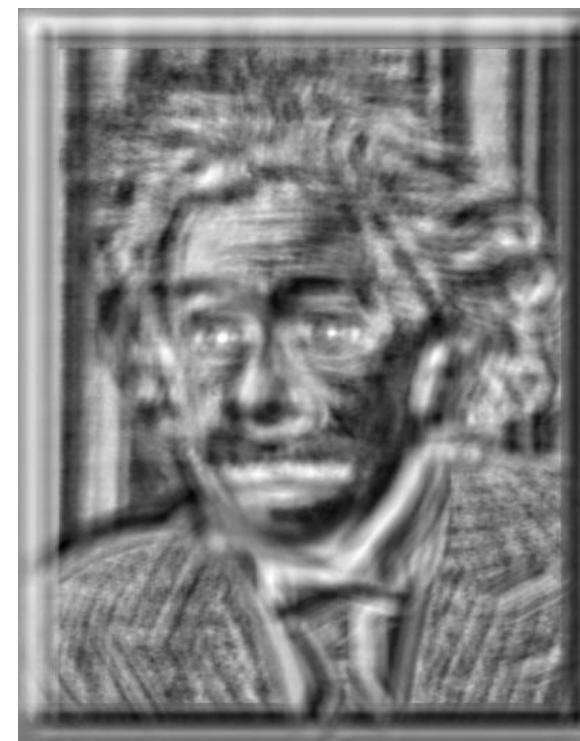
阈值化图像

Matching with filters

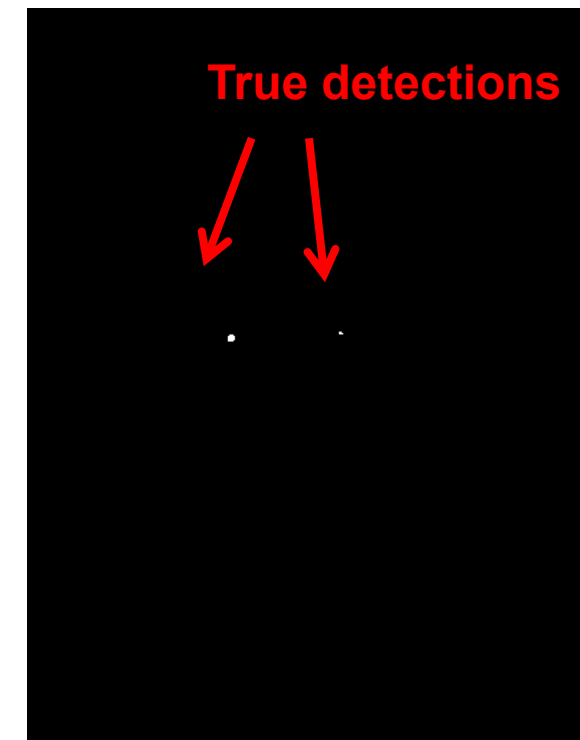
- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



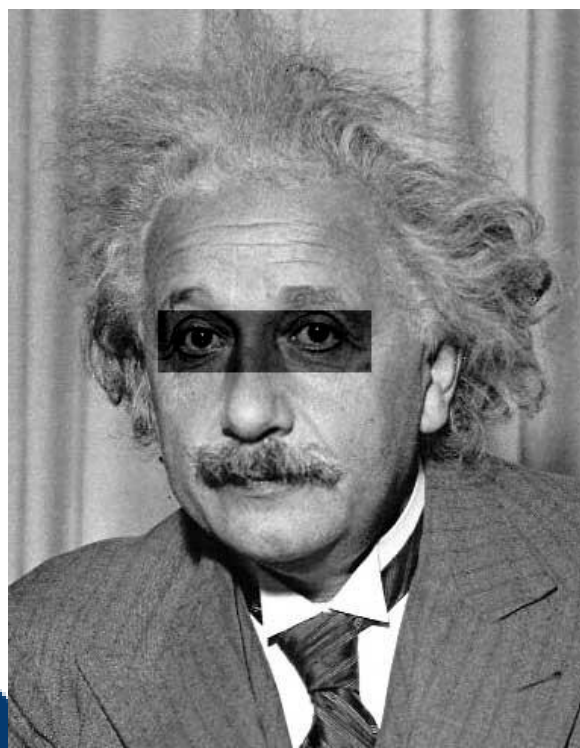
Normalized X-Correlation



Thresholded image

基于滤波器的匹配

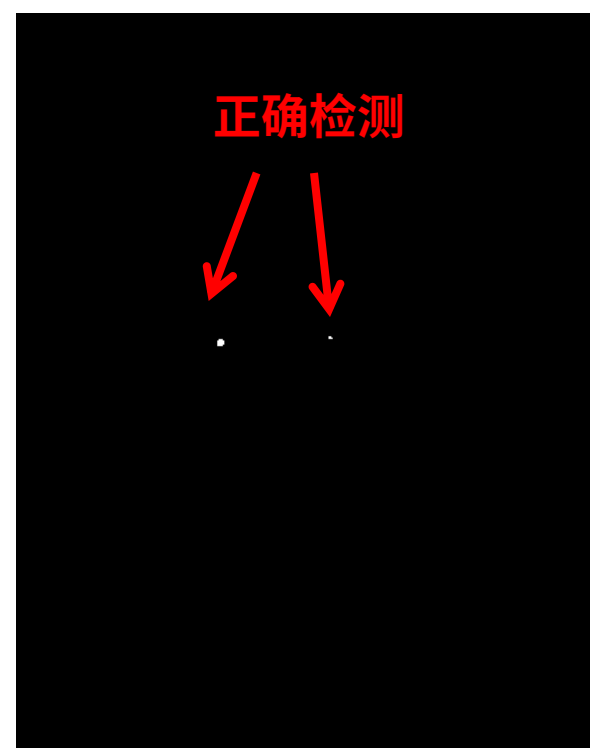
- 目标：在图像中查找 
- 方法 3：归一化互相关



输入



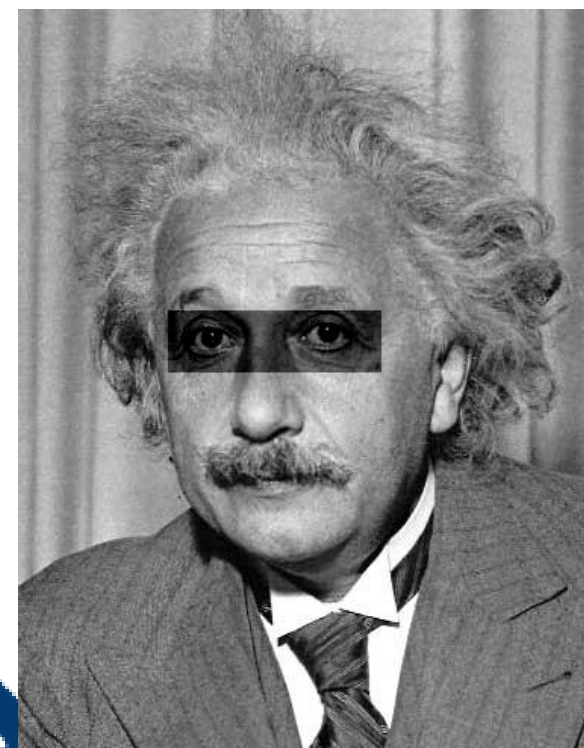
归一化互相关



阈值化图像

Matching with filters

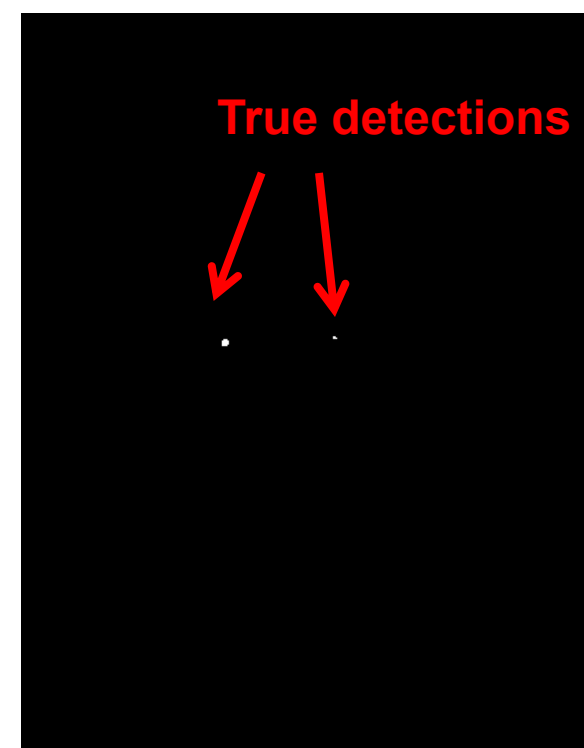
- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



Normalized X-Correlation



Thresholded Image

问：如果我们想找到更大或更小的眼睛怎么办？

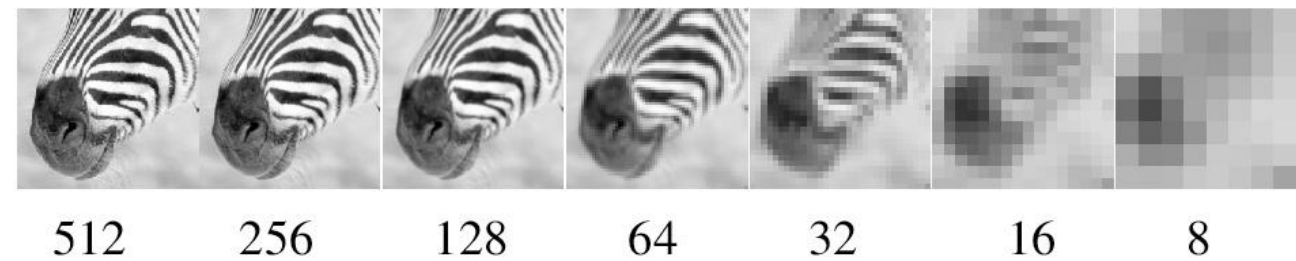
A：图像金字塔

Q: What if we want to find larger or smaller eyes?

A: Image Pyramid

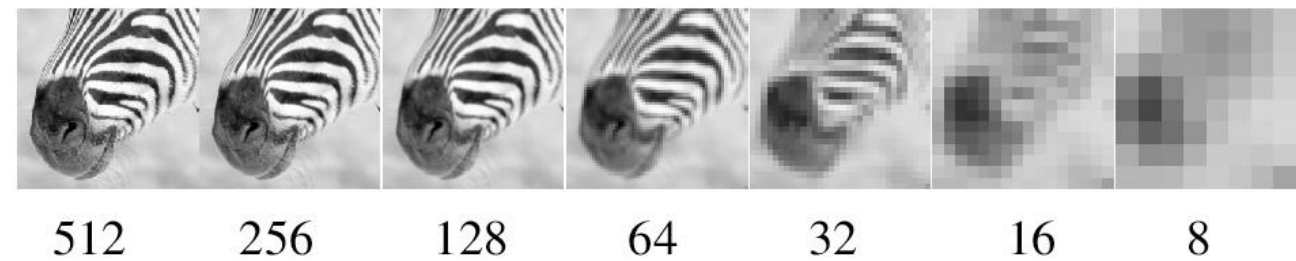


高斯金字塔



来源: Forsyth

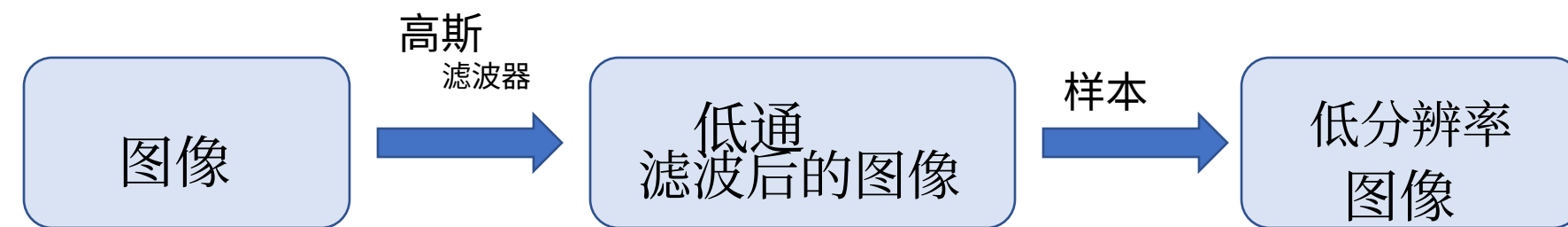
Gaussian pyramid



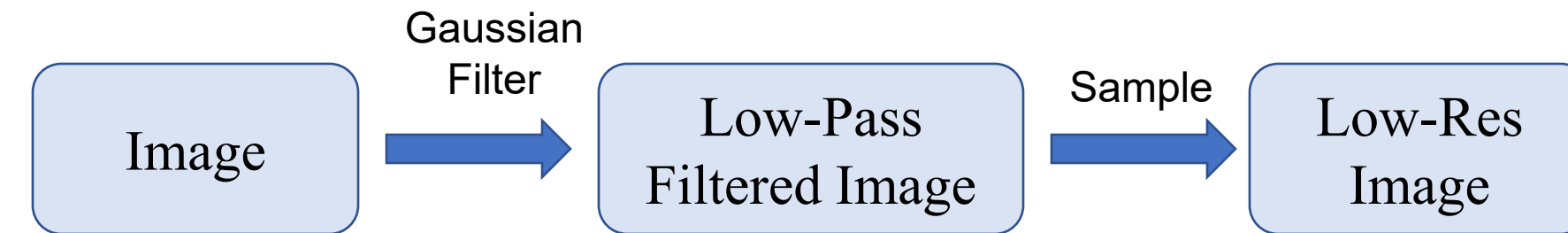
Source: Forsyth



采样回顾



Review of Sampling



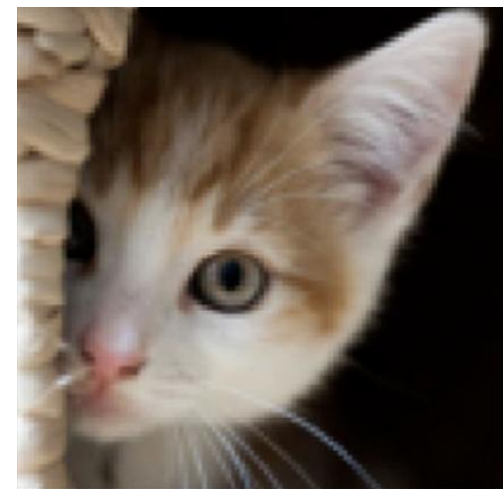
为什么需要低通滤波器？



高分辨率图像



直接下采样



使用低通滤波器

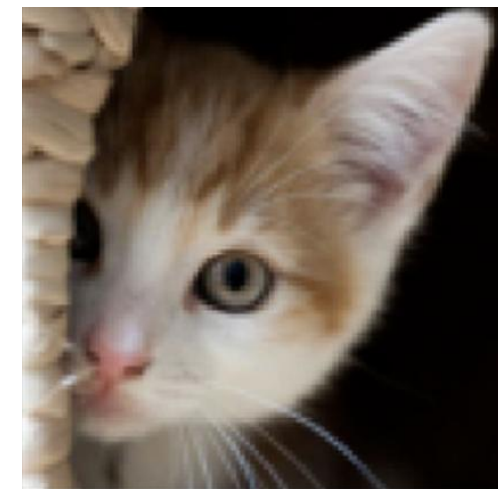
Why low-pass filter is needed?



High resolution image



Direct down-sampling



With low-pass filter



还有高斯金字塔的其他优势吗？

加速

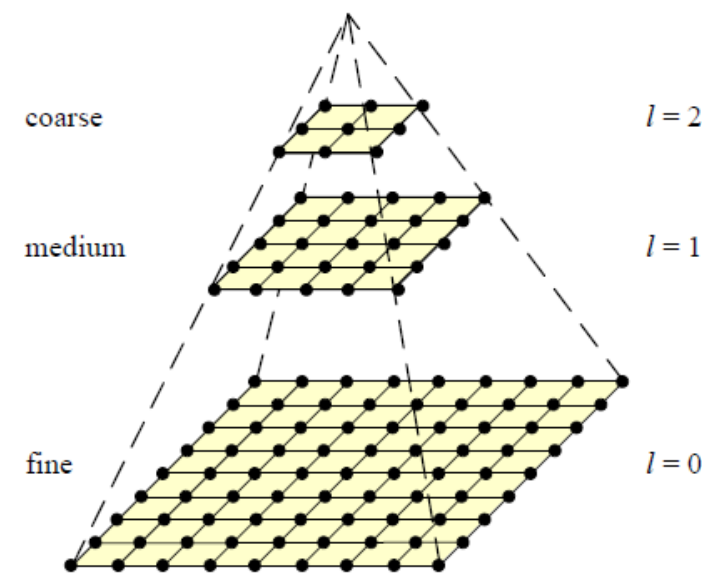
Any other benefits of Gaussian pyramid?

Speed up



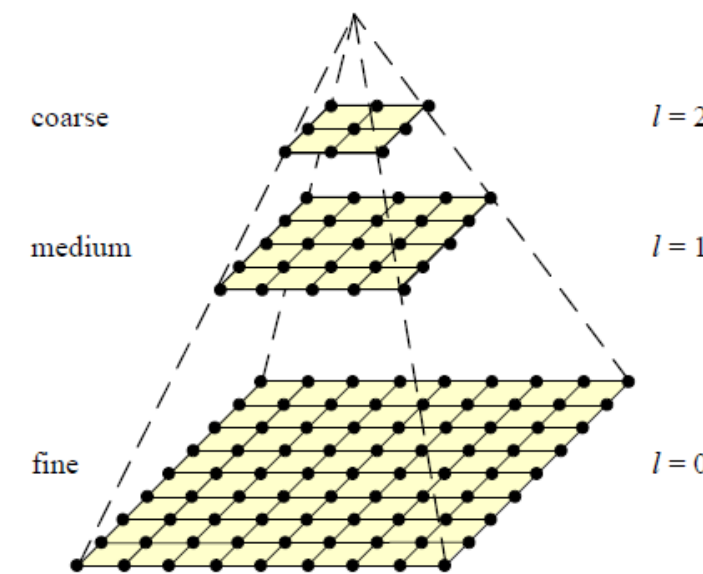
由粗到细的图像配准

1. 计算高斯金字塔
2. 在粗层级进行对齐
 - 找到最小 SSD 位置。
3. 在更细的层级上依次对齐
 - 在以较粗尺度确定的位置为中心的小范围内搜索（例如 5x5）

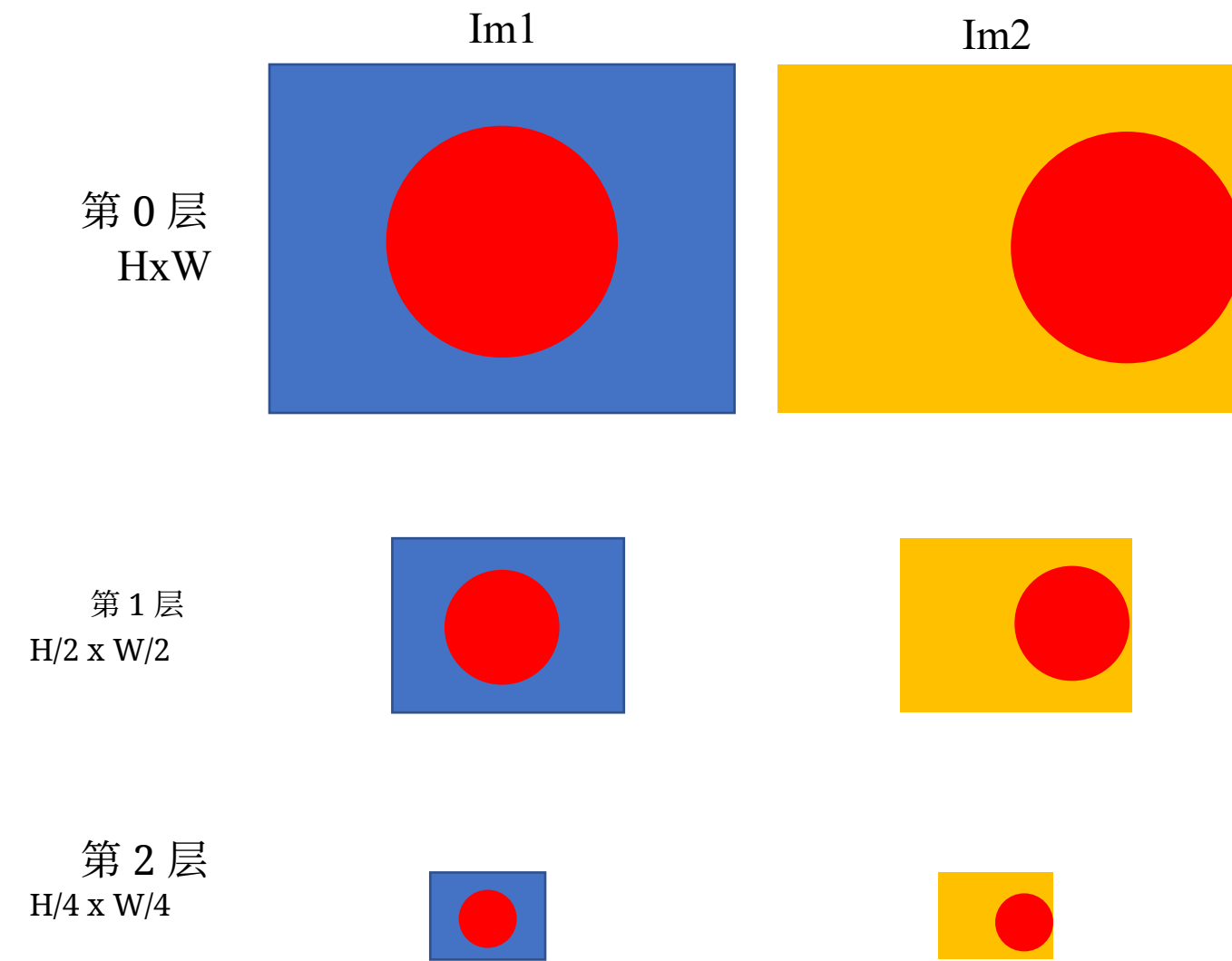


Coarse-to-fine Image Registration

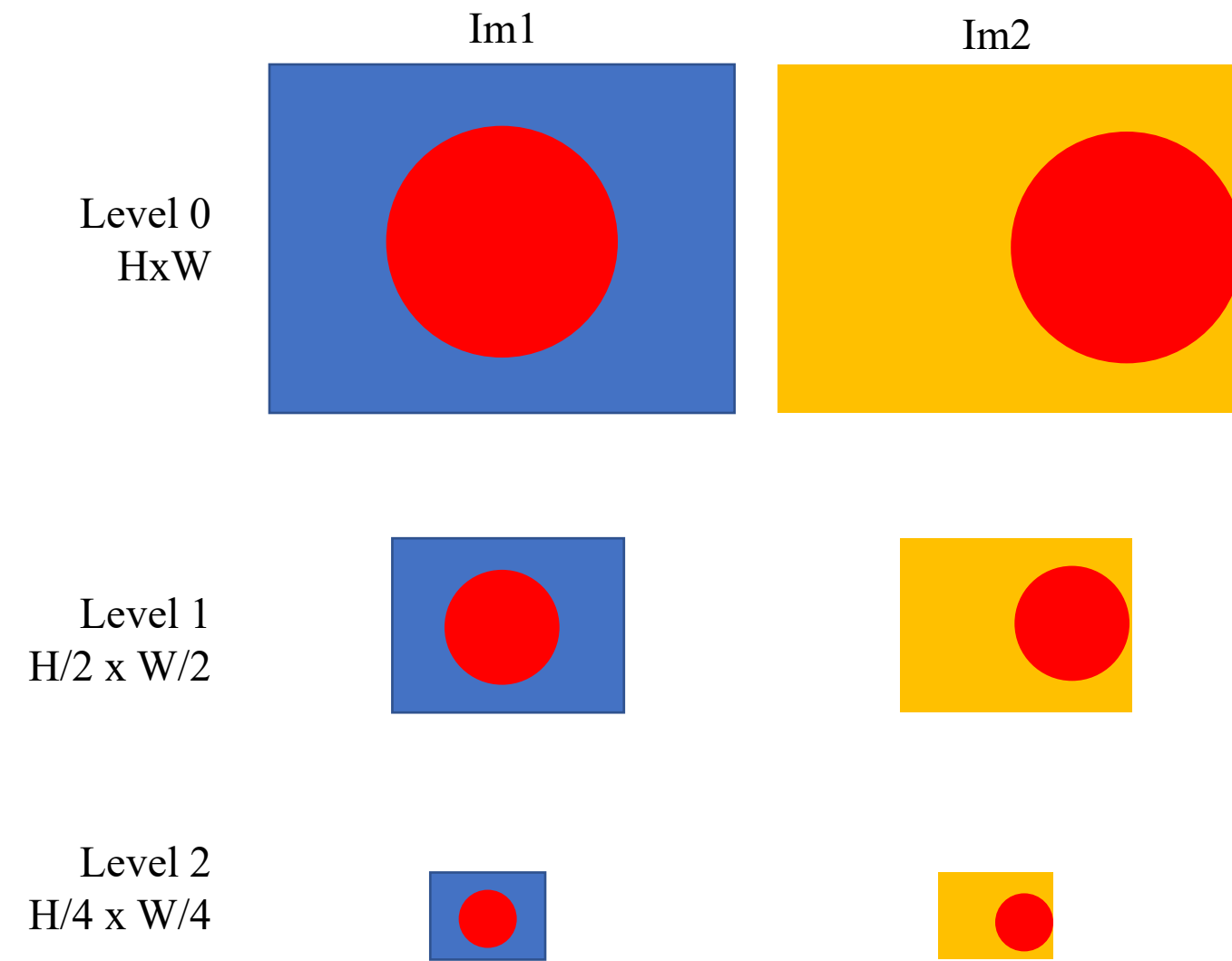
1. Compute Gaussian pyramid
2. Align at coarse level
 - Find minimum SSD position
3. Successively align at finer levels
 - Search small range (e.g., 5x5) centered around position determined at coarser scale



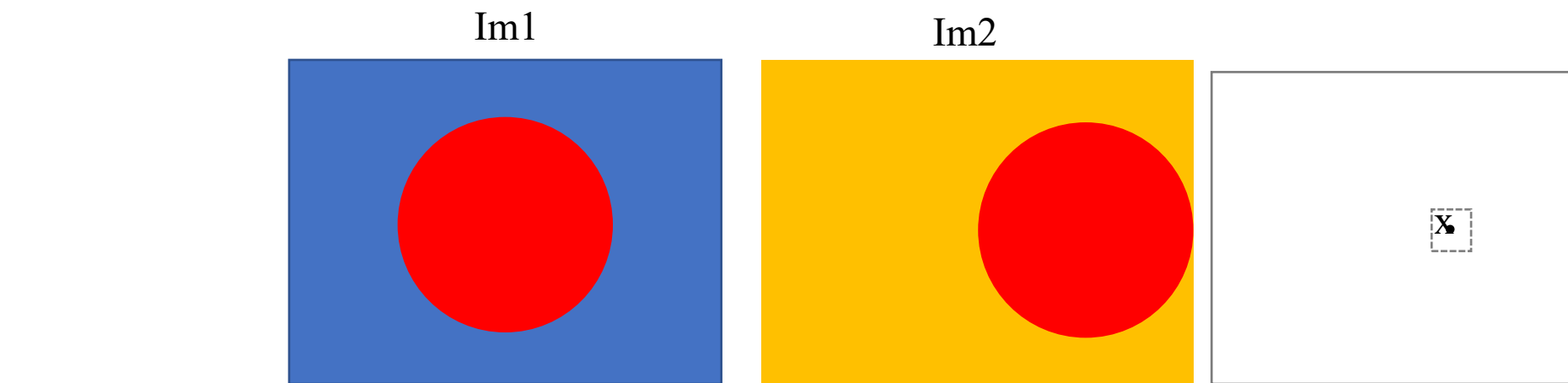
由粗到细的图像配准



Coarse-to-fine Image Registration



由粗到精的图像配准



$$tx_0, ty_0 = \operatorname{argmin}_{\substack{tx \in 2 \cdot tx_1 + [-s..s], \\ ty \in 2 \cdot ty_1 + [-s..s]}} \operatorname{SSD}(im1_0, \operatorname{translate}(im2_0, tx, ty))$$

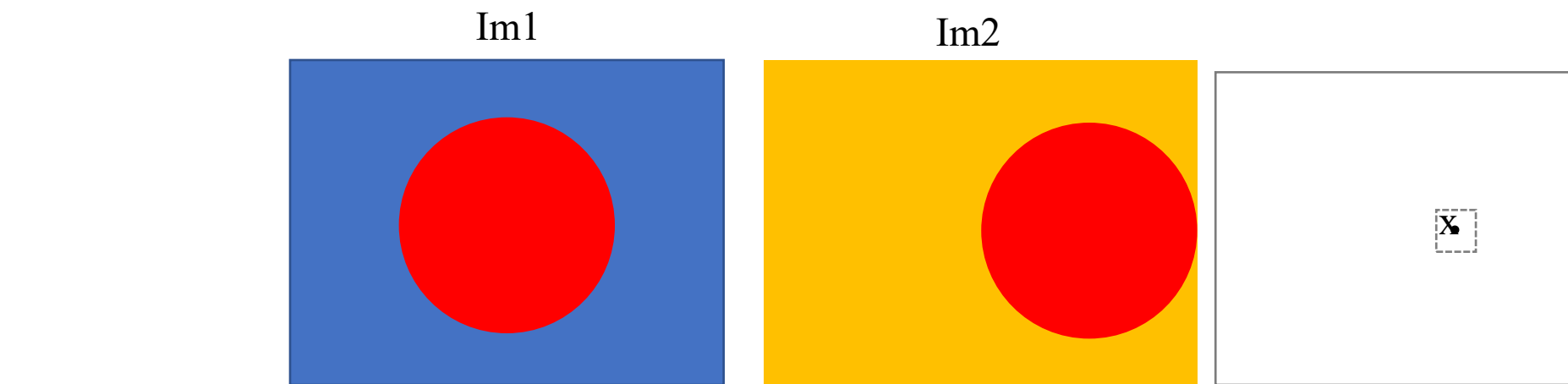


$$tx_1, ty_1 = \operatorname{argmin}_{\substack{tx \in 2 \cdot tx_2 + [-s..s], \\ ty \in 2 \cdot ty_2 + [-s..s]}} \operatorname{SSD}(im1_1, \operatorname{translate}(im2_1, tx, ty))$$



$$tx_2, ty_2 = \operatorname{argmin}_{\substack{tx \in [-\frac{W}{8}..\frac{W}{8}], \\ ty \in [-\frac{H}{8}..\frac{H}{8}]}} \operatorname{SSD}(im1_2, \operatorname{translate}(im2_2, tx, ty))$$

Coarse-to-fine Image Registration



$$tx_0, ty_0 = \operatorname{argmin}_{\substack{tx \in 2 \cdot tx_1 + [-s..s], \\ ty \in 2 \cdot ty_1 + [-s..s]}} \operatorname{SSD}(im1_0, \operatorname{translate}(im2_0, tx, ty))$$



$$tx_1, ty_1 = \operatorname{argmin}_{\substack{tx \in 2 \cdot tx_2 + [-s..s], \\ ty \in 2 \cdot ty_2 + [-s..s]}} \operatorname{SSD}(im1_1, \operatorname{translate}(im2_1, tx, ty))$$

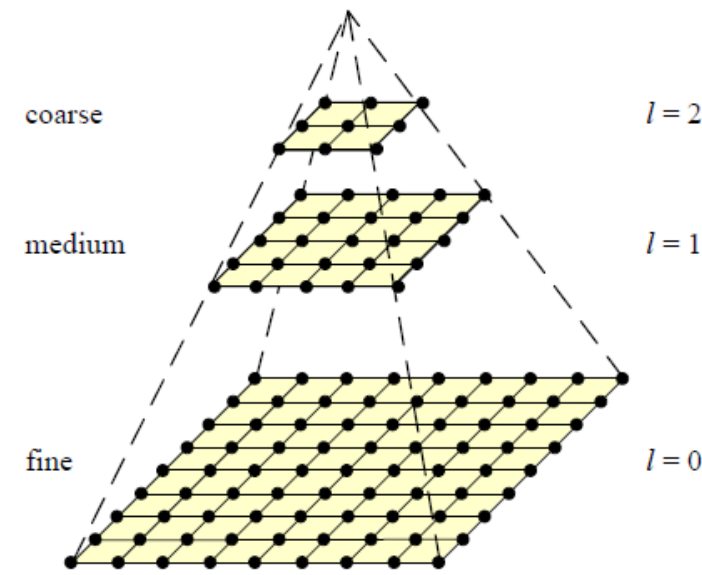


$$tx_2, ty_2 = \operatorname{argmin}_{\substack{tx \in \left[-\frac{W}{8}..\frac{W}{8}\right], \\ ty \in \left[-\frac{H}{8}..\frac{H}{8}\right]}} \operatorname{SSD}(im1_2, \operatorname{translate}(im2_2, tx, ty))$$



粗到细图像配准

1. 计算高斯金字塔
2. 在粗层级进行对齐
 - 寻找 SSD 最小的位置
3. 依次在更精细层级进行对齐
 - 在以较粗尺度确定的位置为中心的小范围内搜索（例如 5x5）



为什么这样会更快？

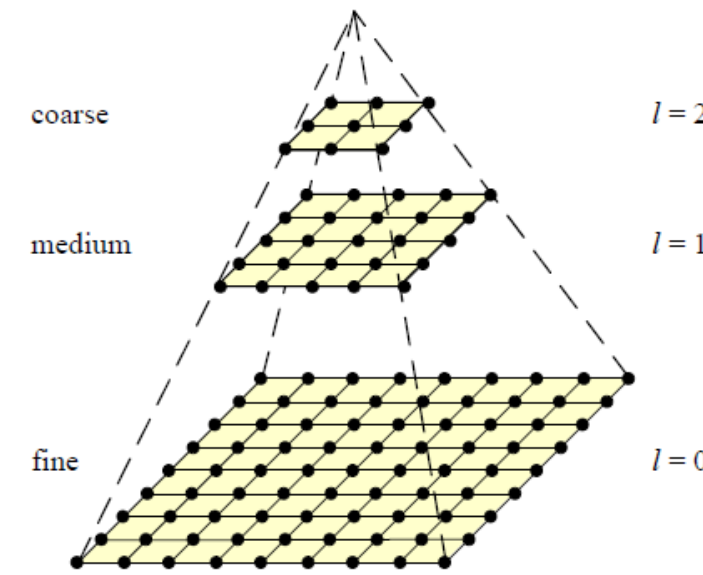
我们能保证得到相同的结果吗？

并非总是如此.....当你的模板包含许多重要的高频信息时，它可能会失败。
当你的图像带有噪声时，它可能表现更好。



Coarse-to-fine Image Registration

1. Compute Gaussian pyramid
2. Align at coarse level
 - Find minimum SSD position
3. Successively align at finer levels
 - Search small range (e.g., 5x5) centered around position determined at coarser scale



Why is this faster?

Are we guaranteed to get the same result?

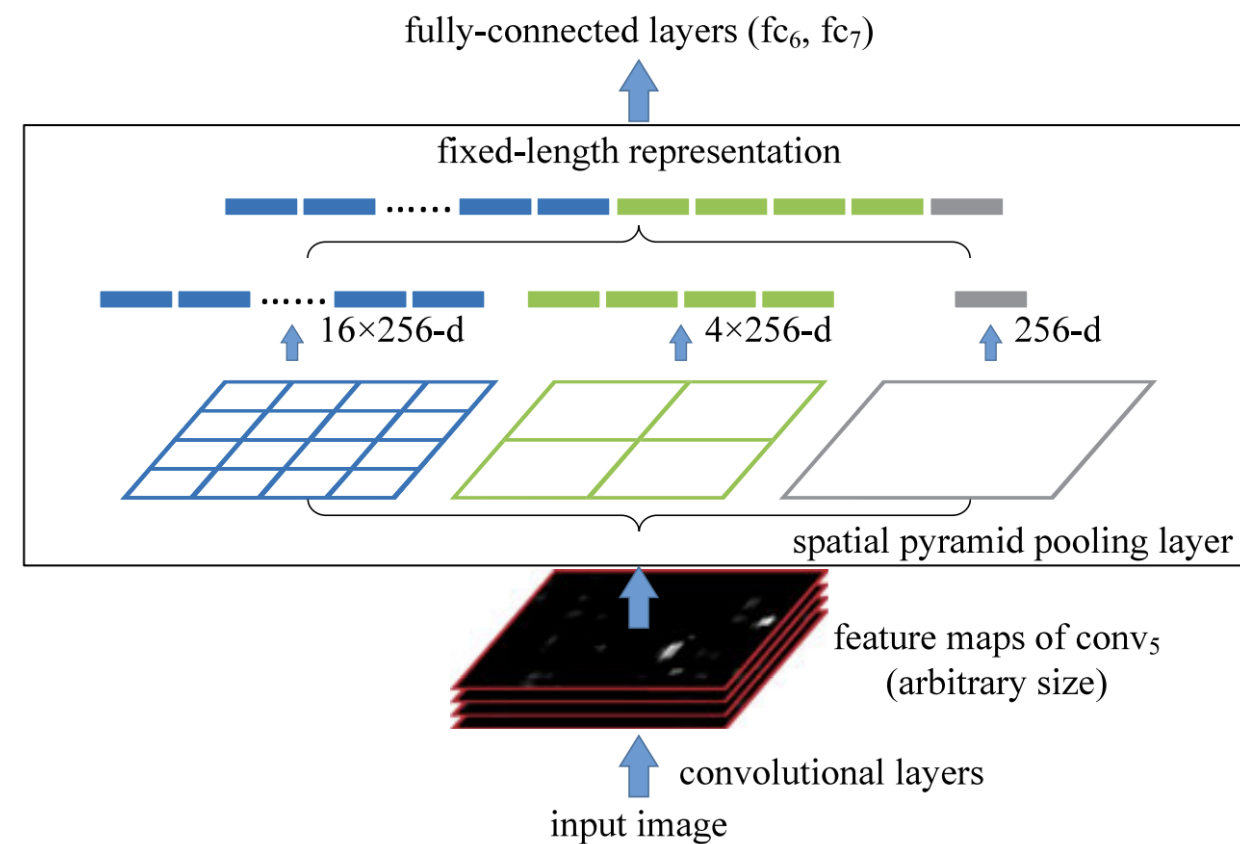
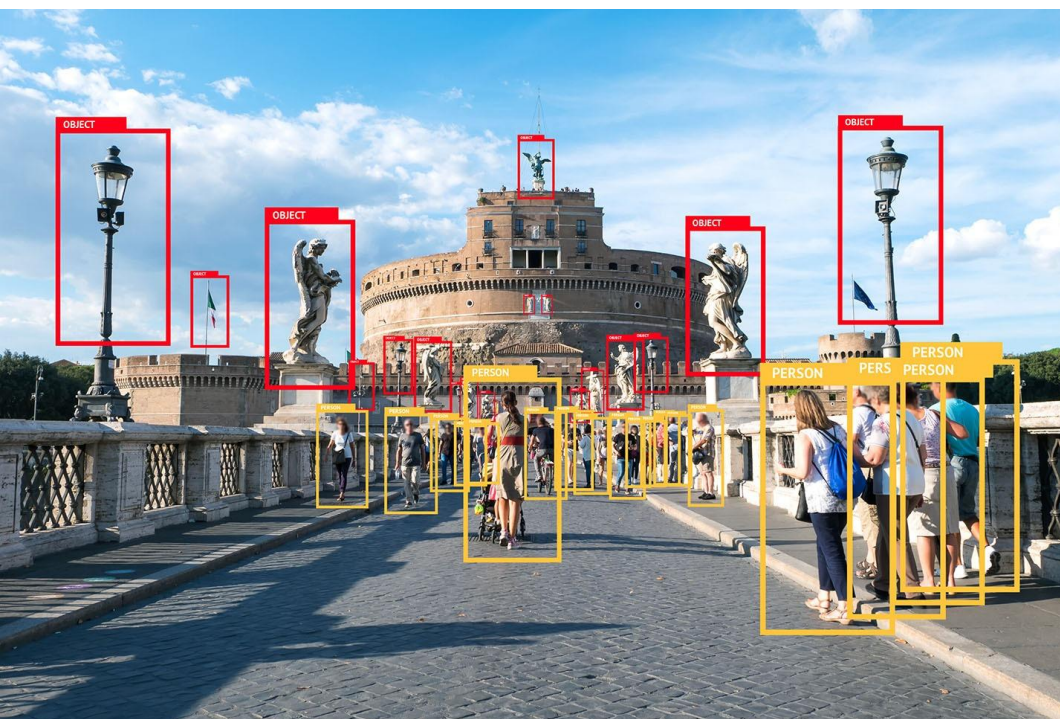
Not all the time...

It may fail when your template has many important high-frequency information.
It may perform better when your image has noise.



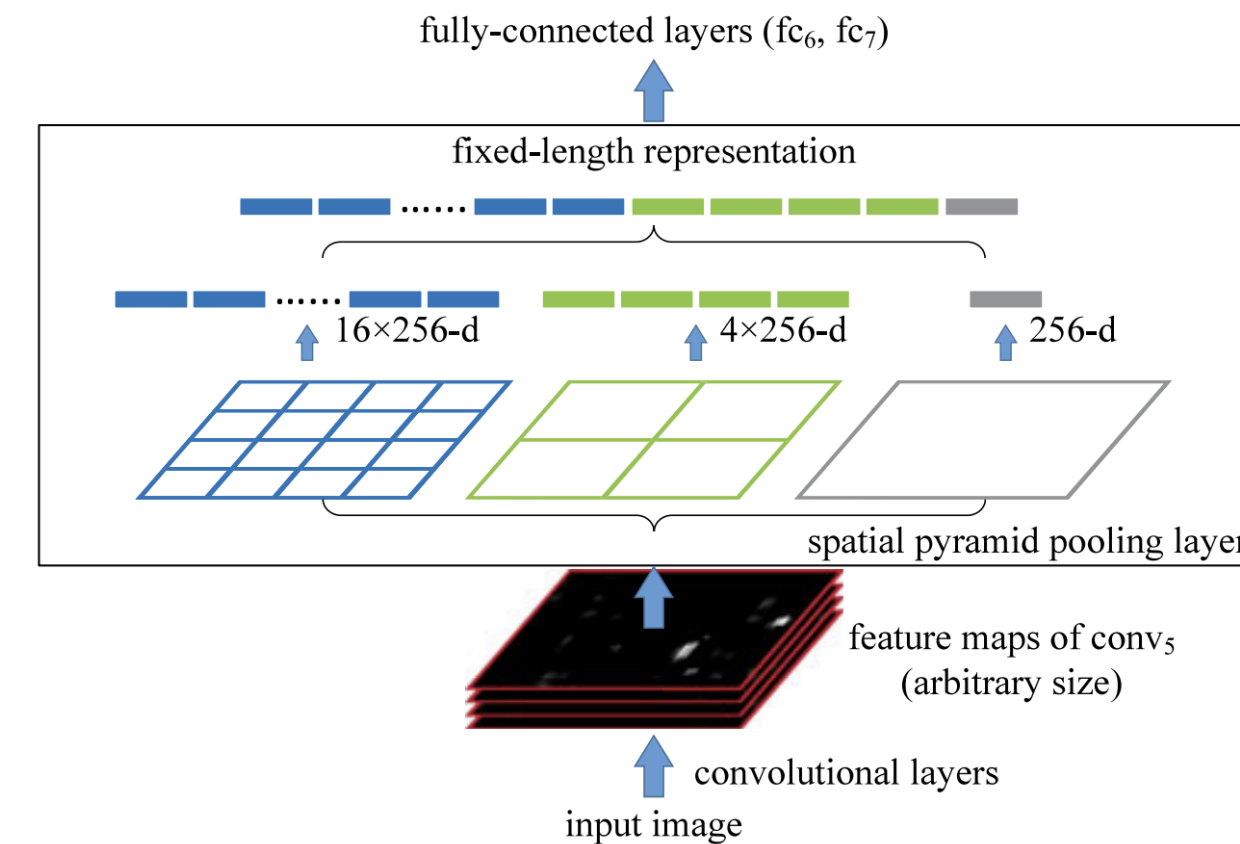
关于图像金字塔的更多内容

目标检测与空间金字塔池化



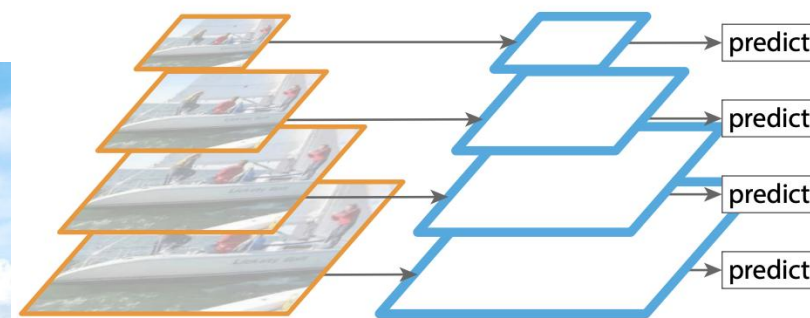
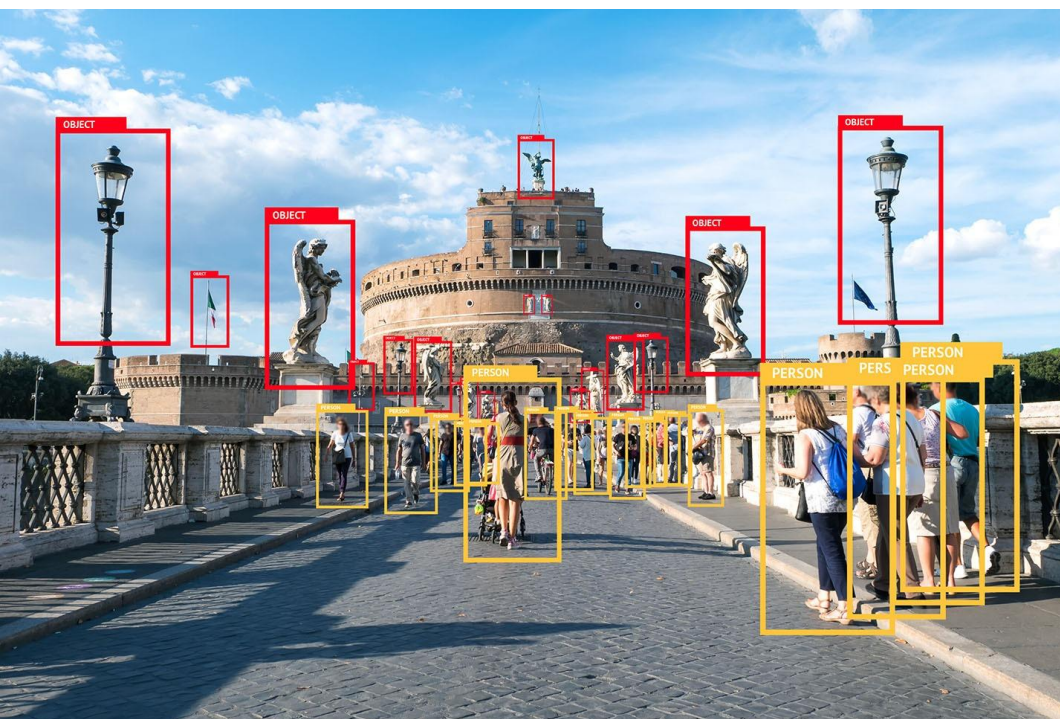
More about image pyramid

Object detection and spatial pyramid pooling

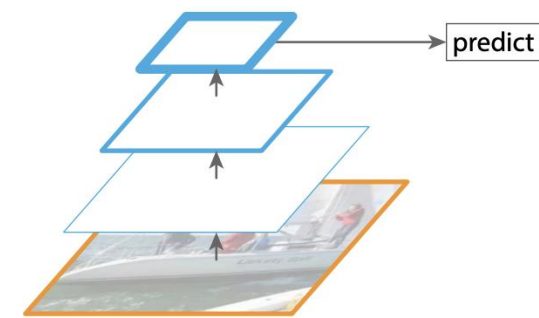


关于图像金字塔的更多内容

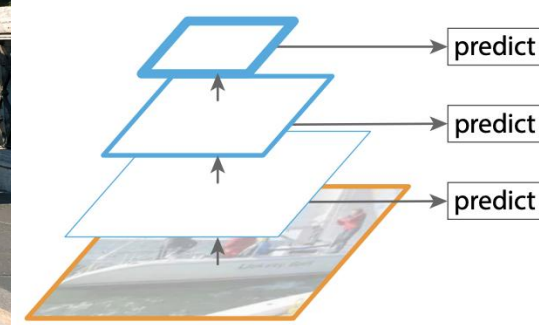
目标检测与特征金字塔网络



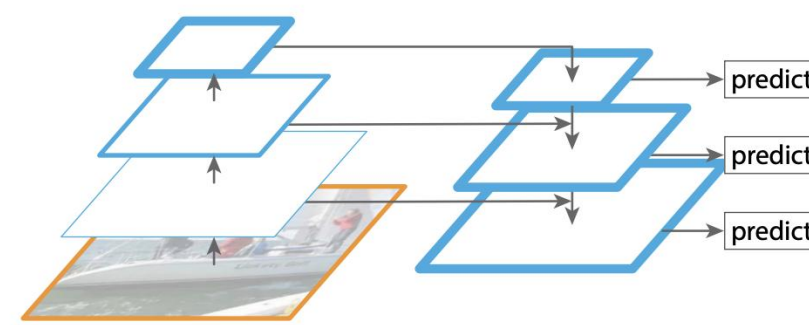
(a) Featurized image pyramid



(b) Single feature map



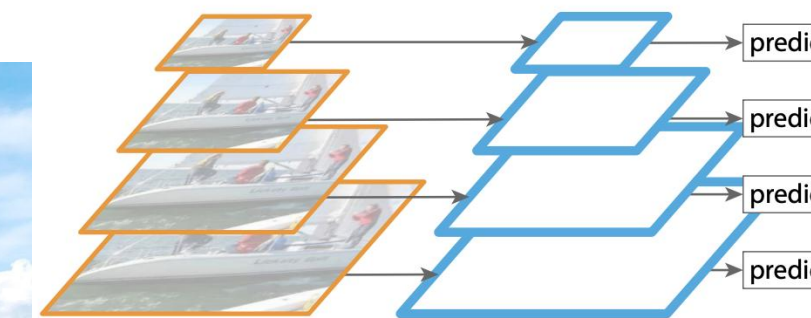
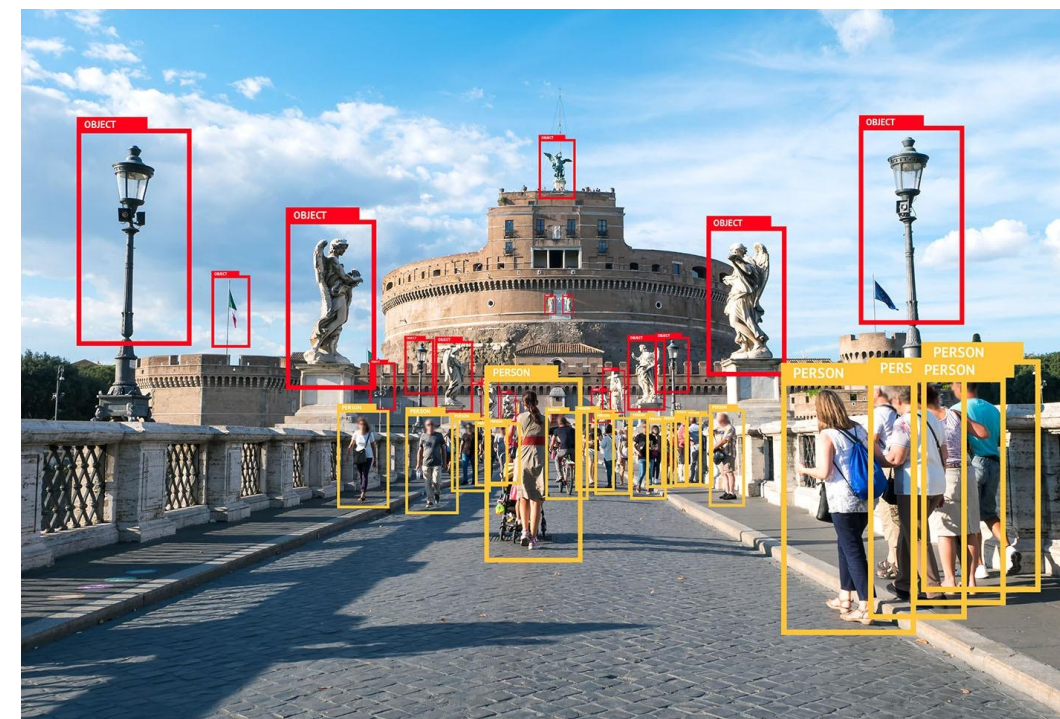
(c) Pyramidal feature hierarchy



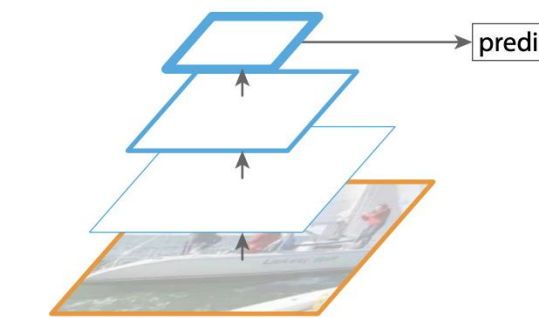
(d) Feature Pyramid Network

More about image pyramid

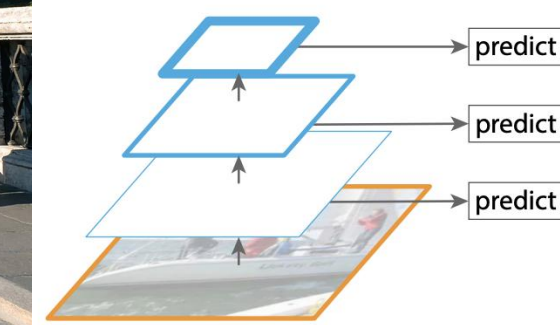
Object detection and feature pyramid networks



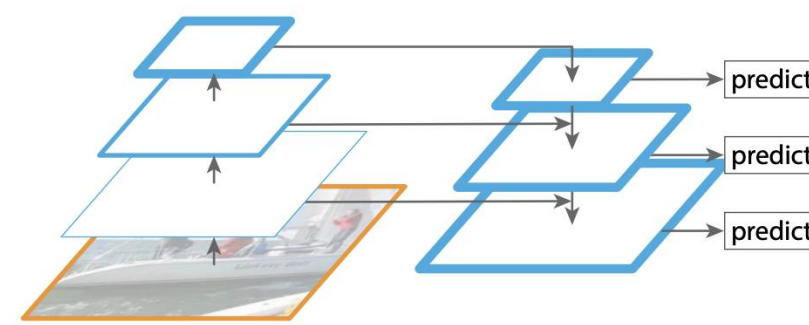
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy

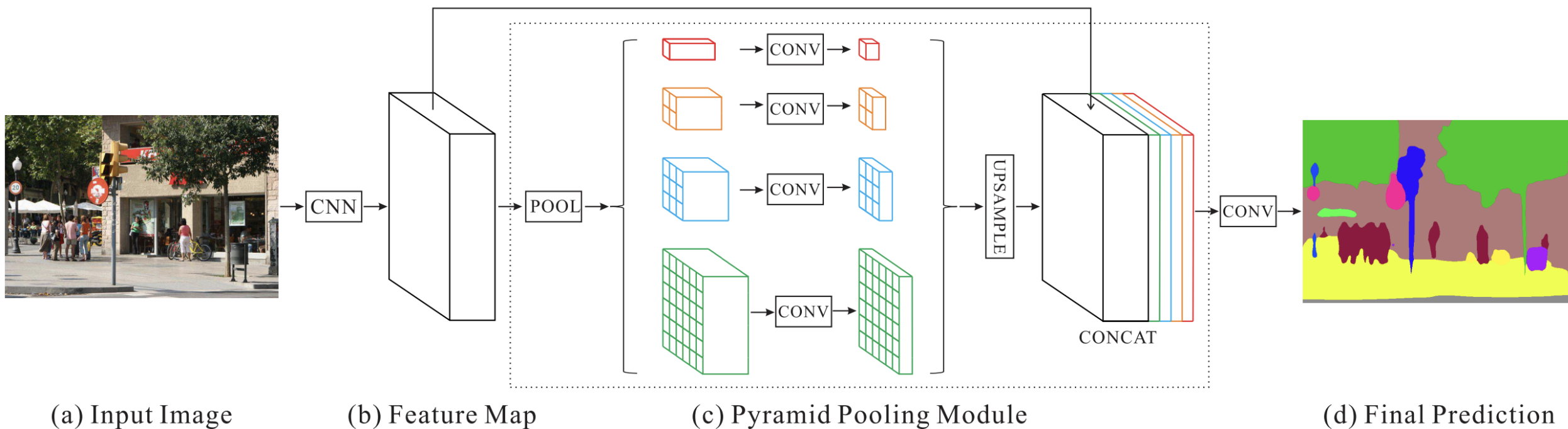


(d) Feature Pyramid Network



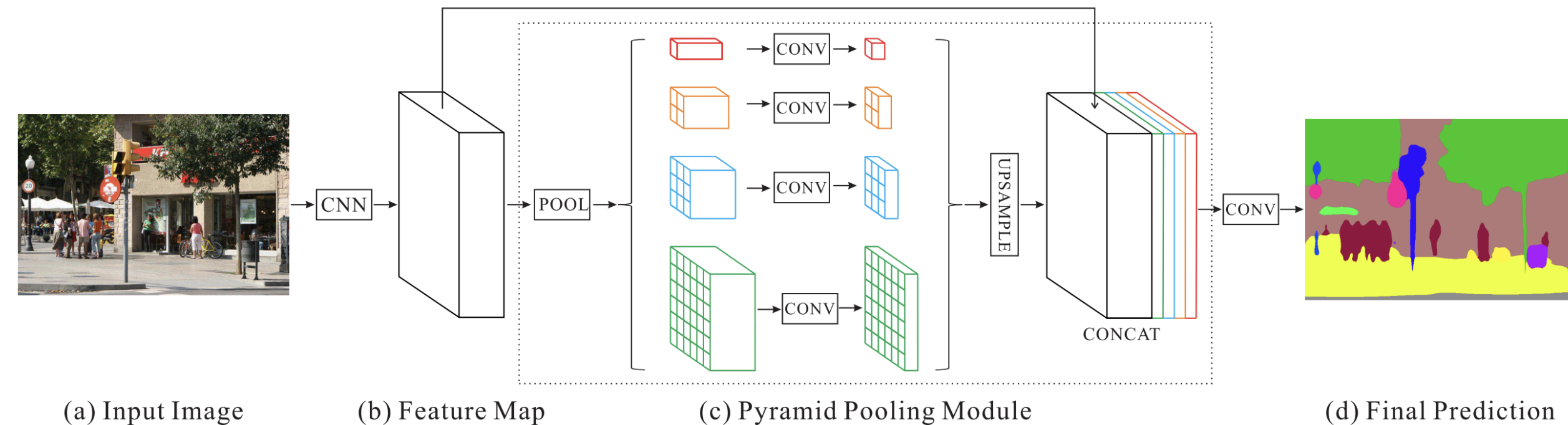
关于图像金字塔的更多内容

图像分割与图像金字塔

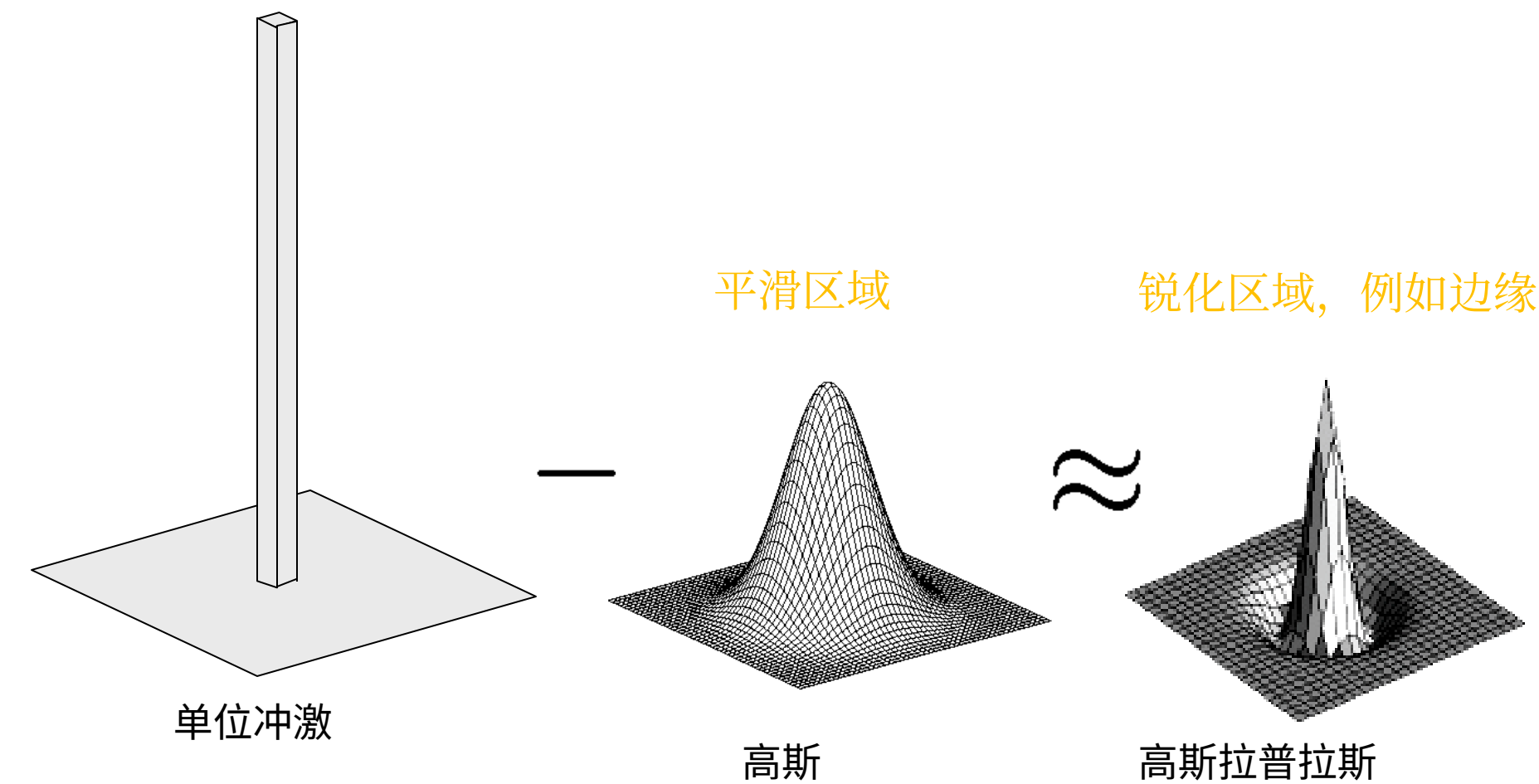


More about image pyramid

Image segmentation and image pyramid

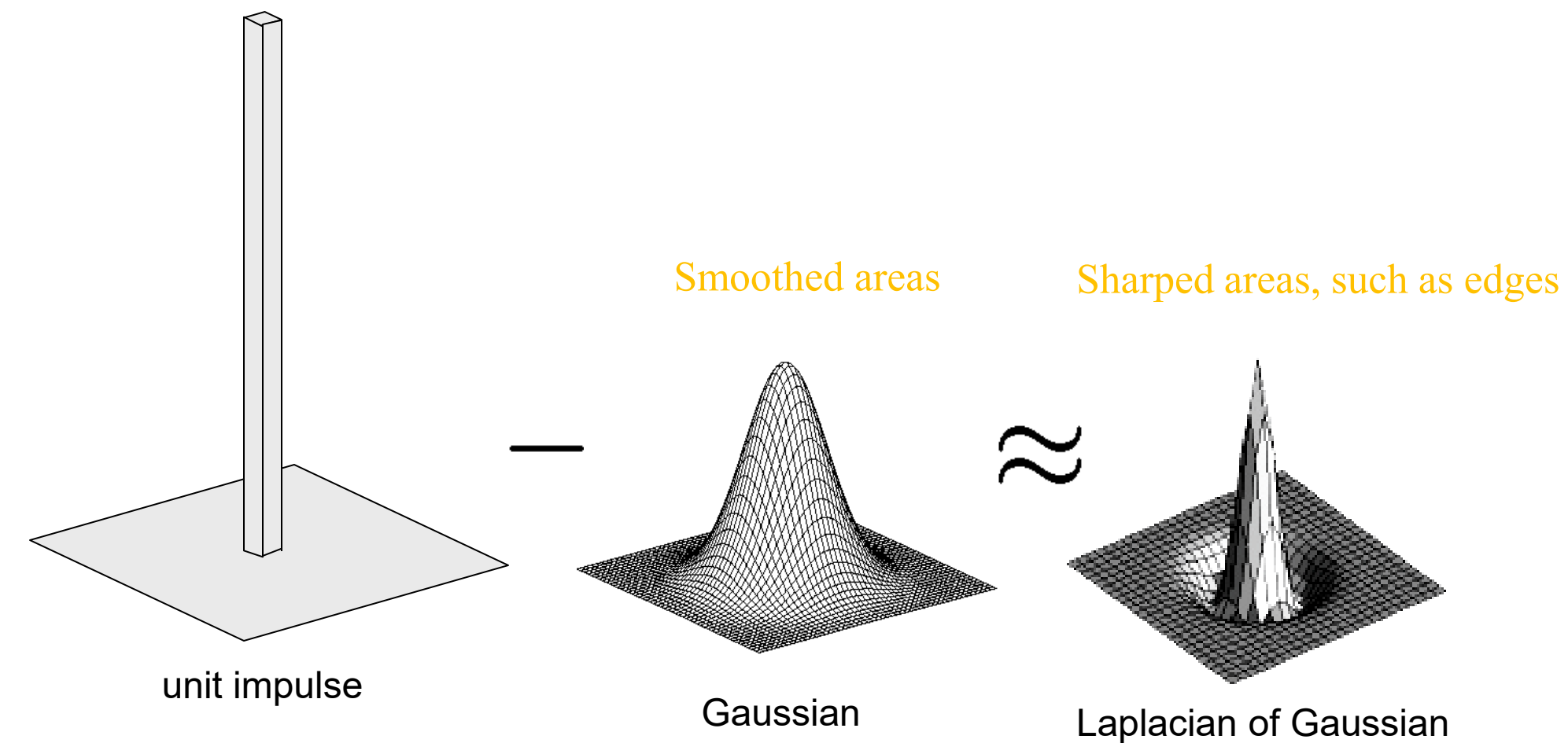


拉普拉斯滤波器



来源: Lazebnik

Laplacian filter

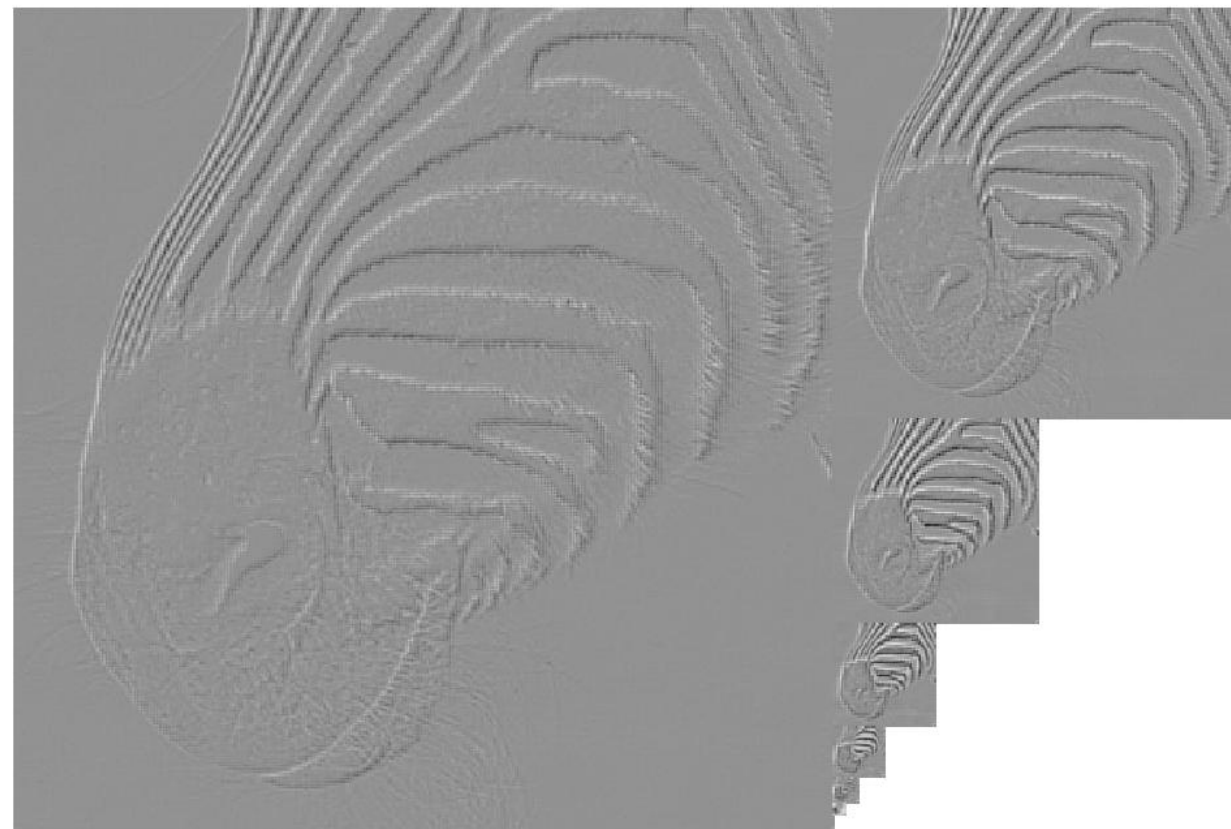


Source: Lazebnik

拉普拉斯金字塔



512 256 128 64 32 16 8

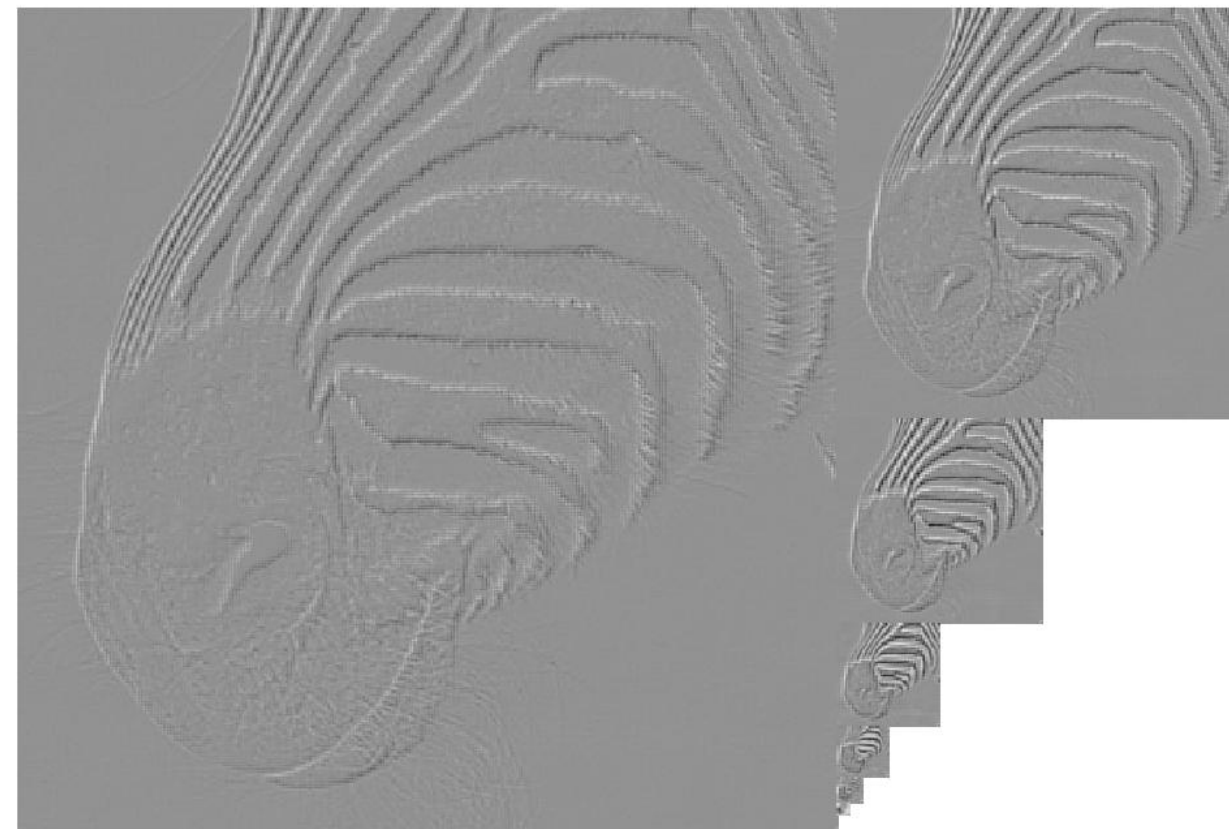


来源: Forsyth

Laplacian pyramid



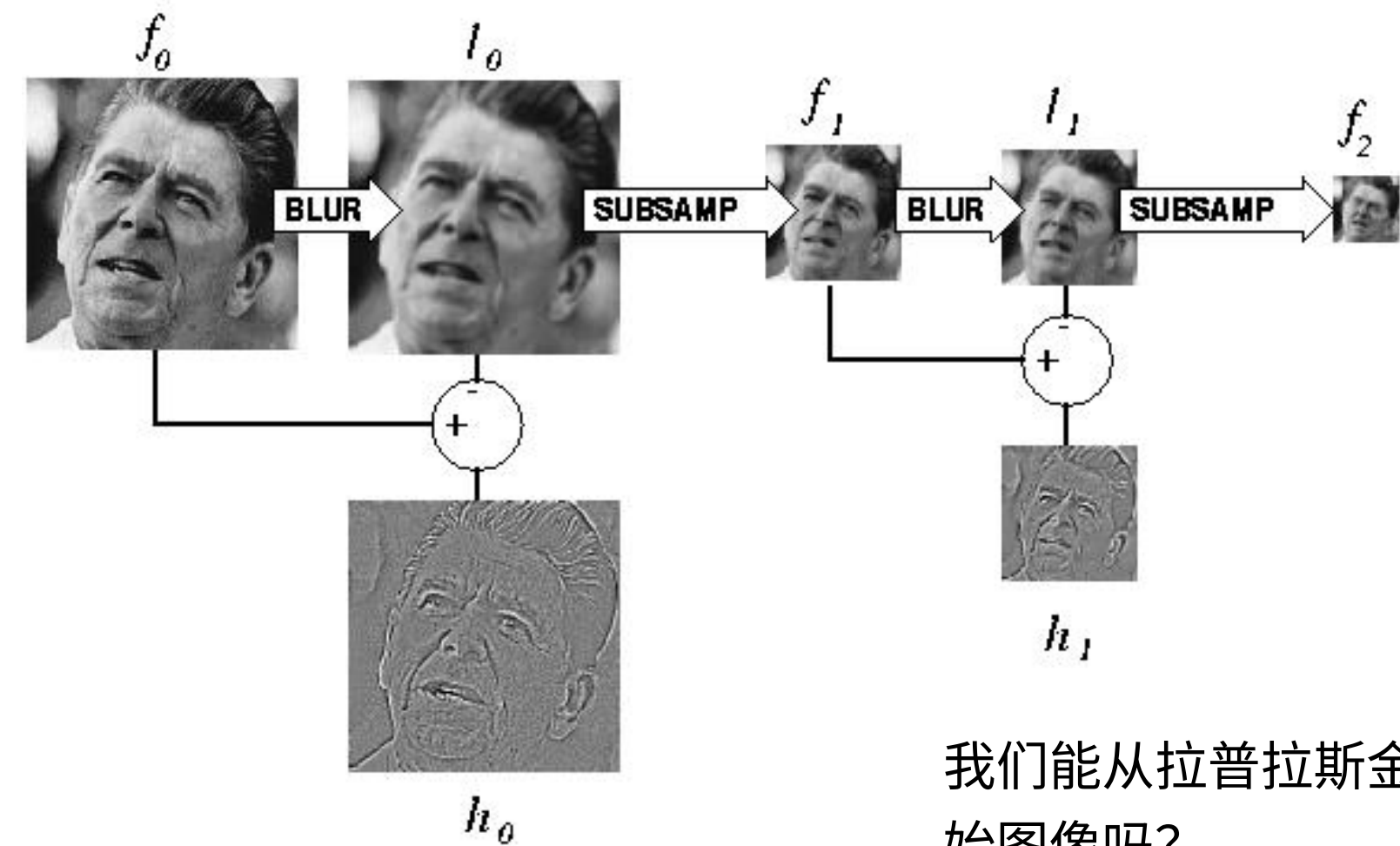
512 256 128 64 32 16 8



Source: Forsyth

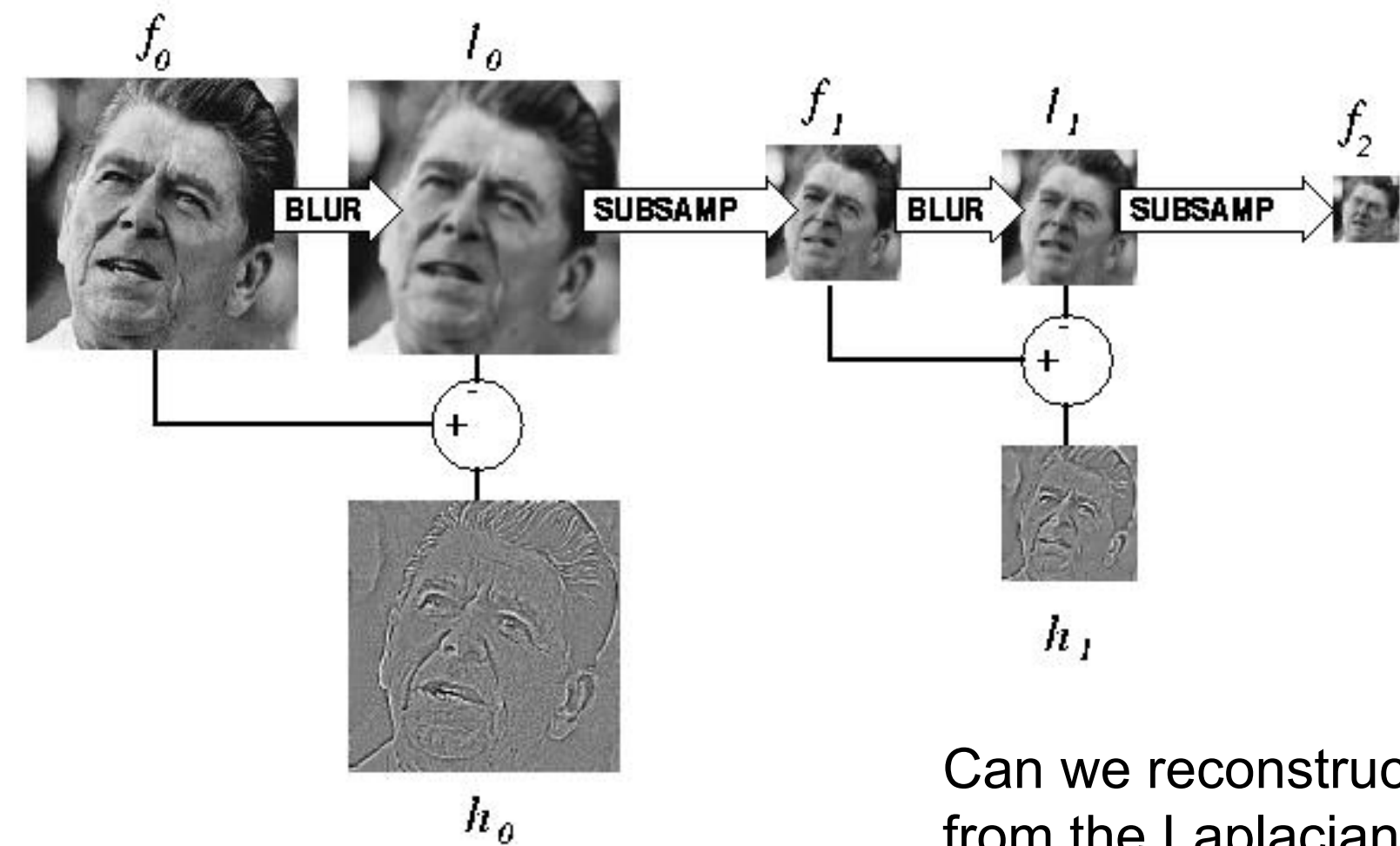


计算高斯/拉普拉斯金字塔



我们能从拉普拉斯金字塔重建原始图像吗？

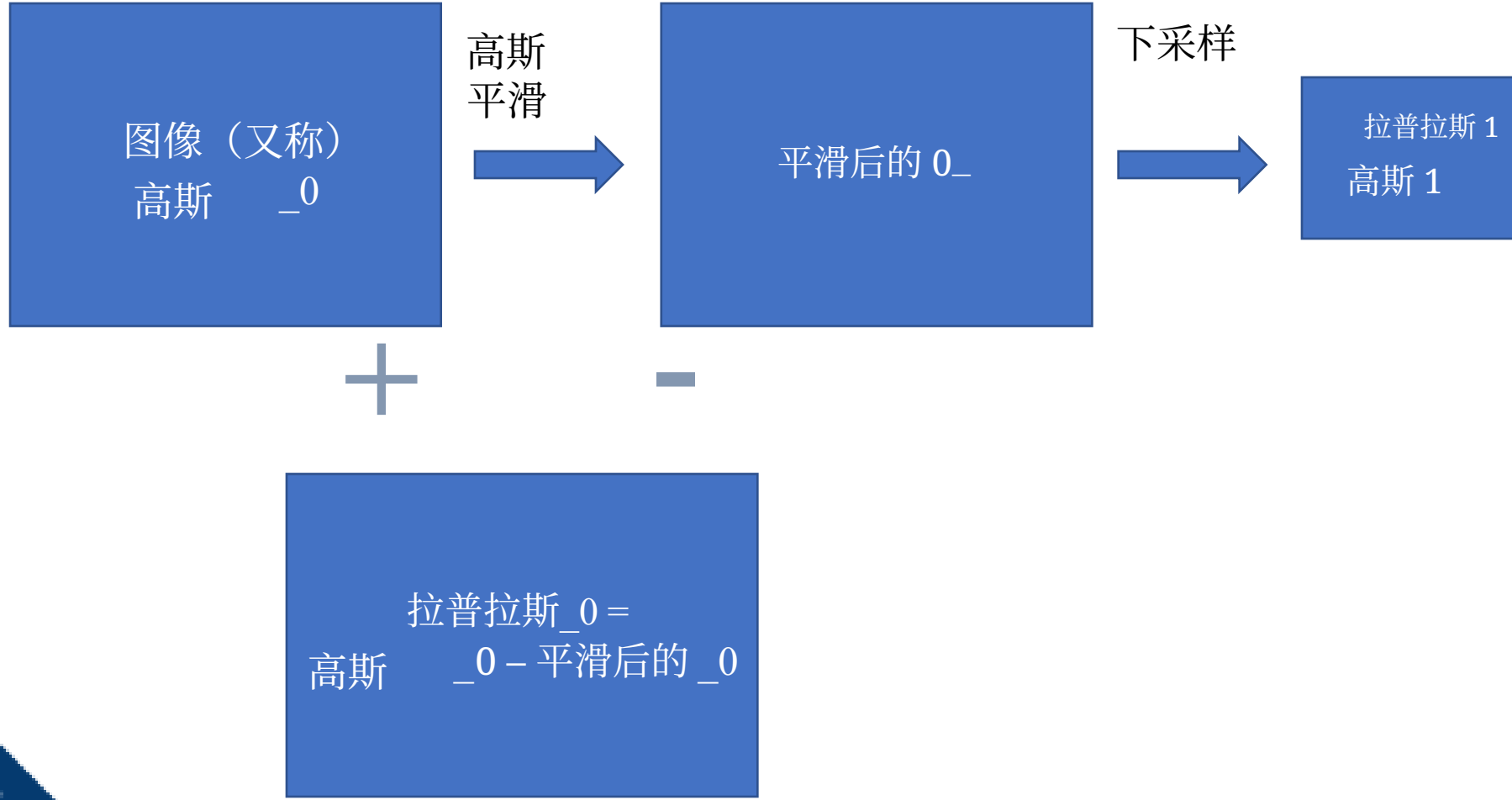
Computing Gaussian/Laplacian Pyramid



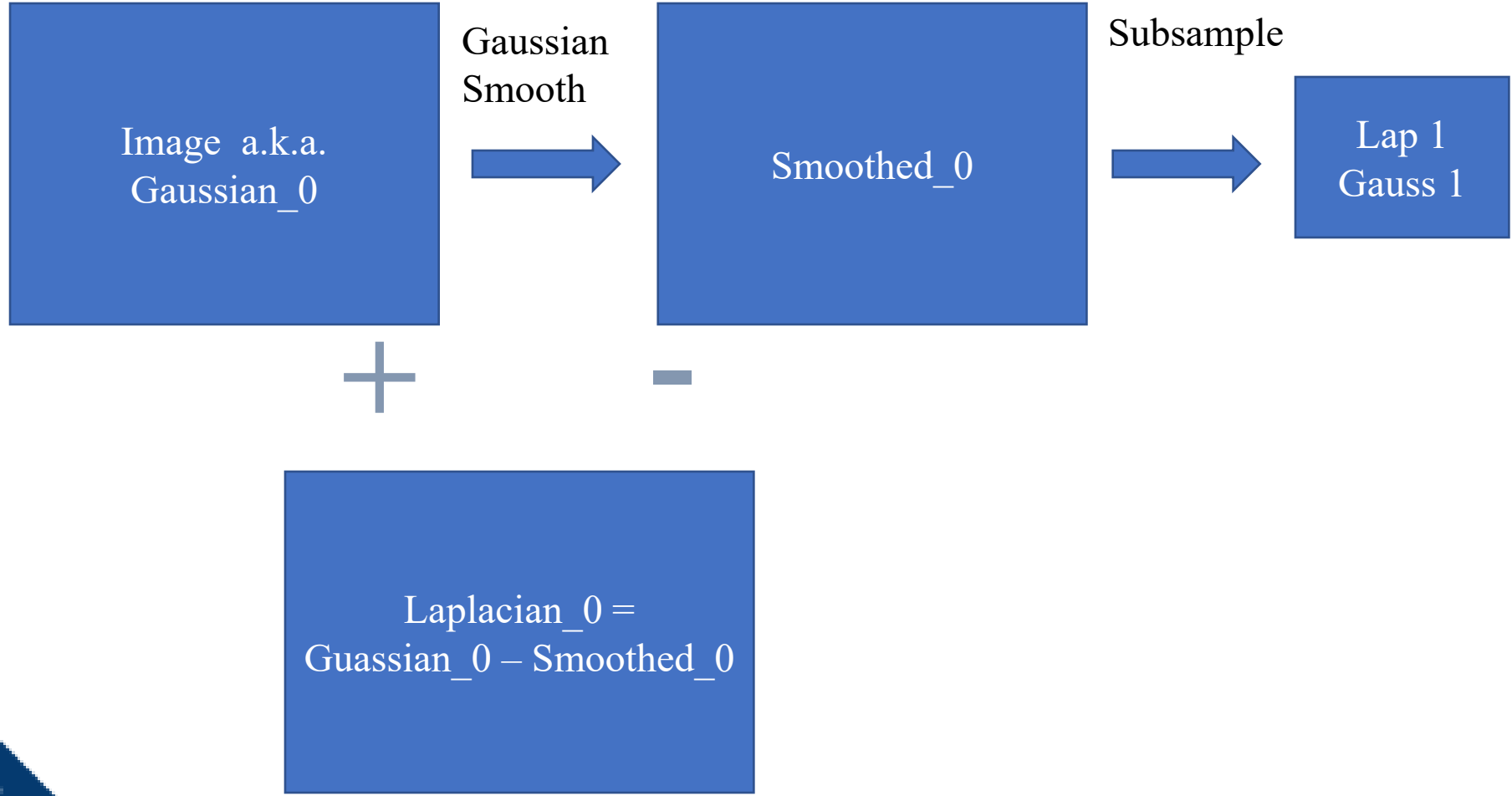
Can we reconstruct the original from the Laplacian pyramid?



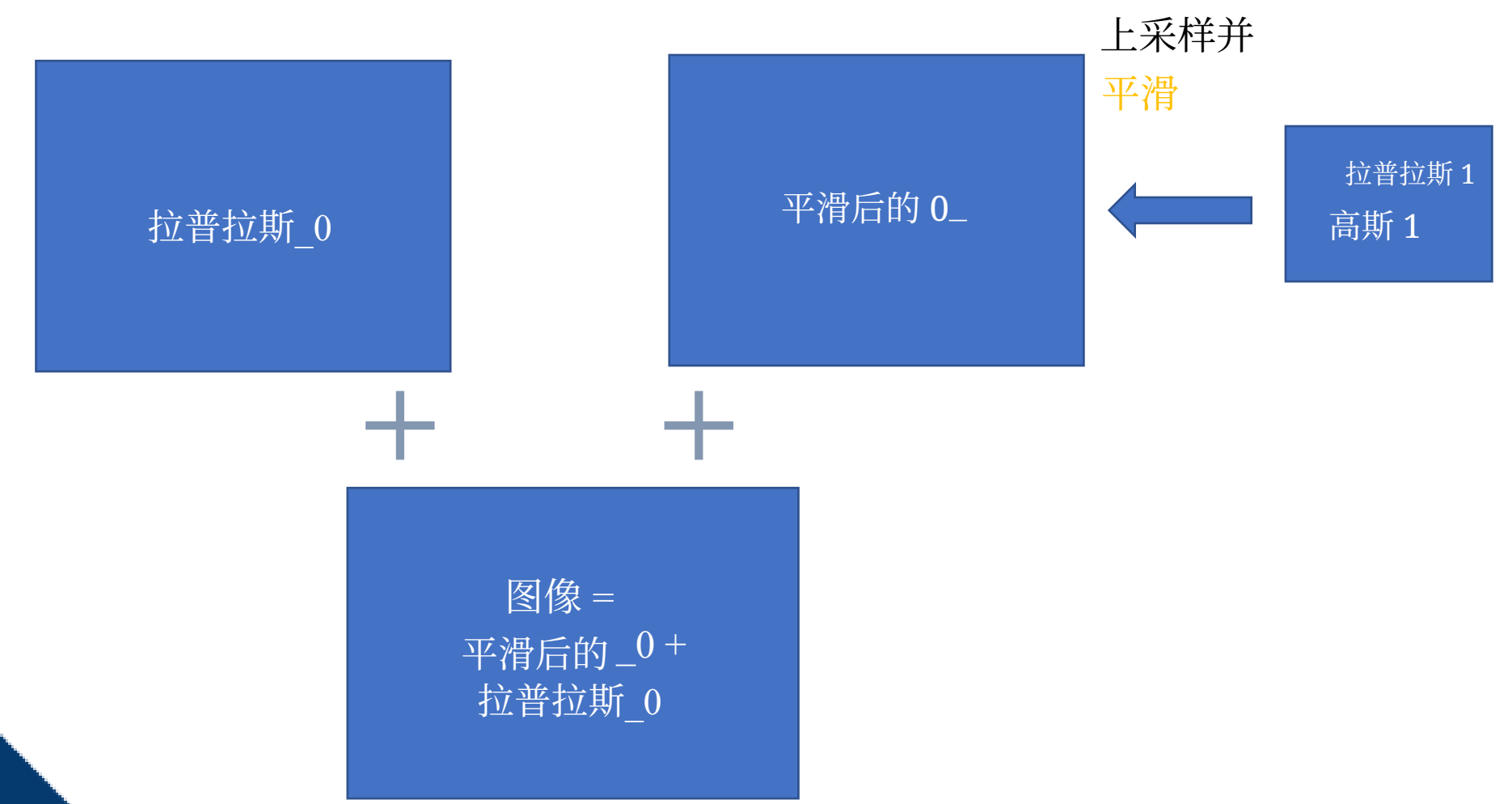
创建2级拉普拉斯金字塔



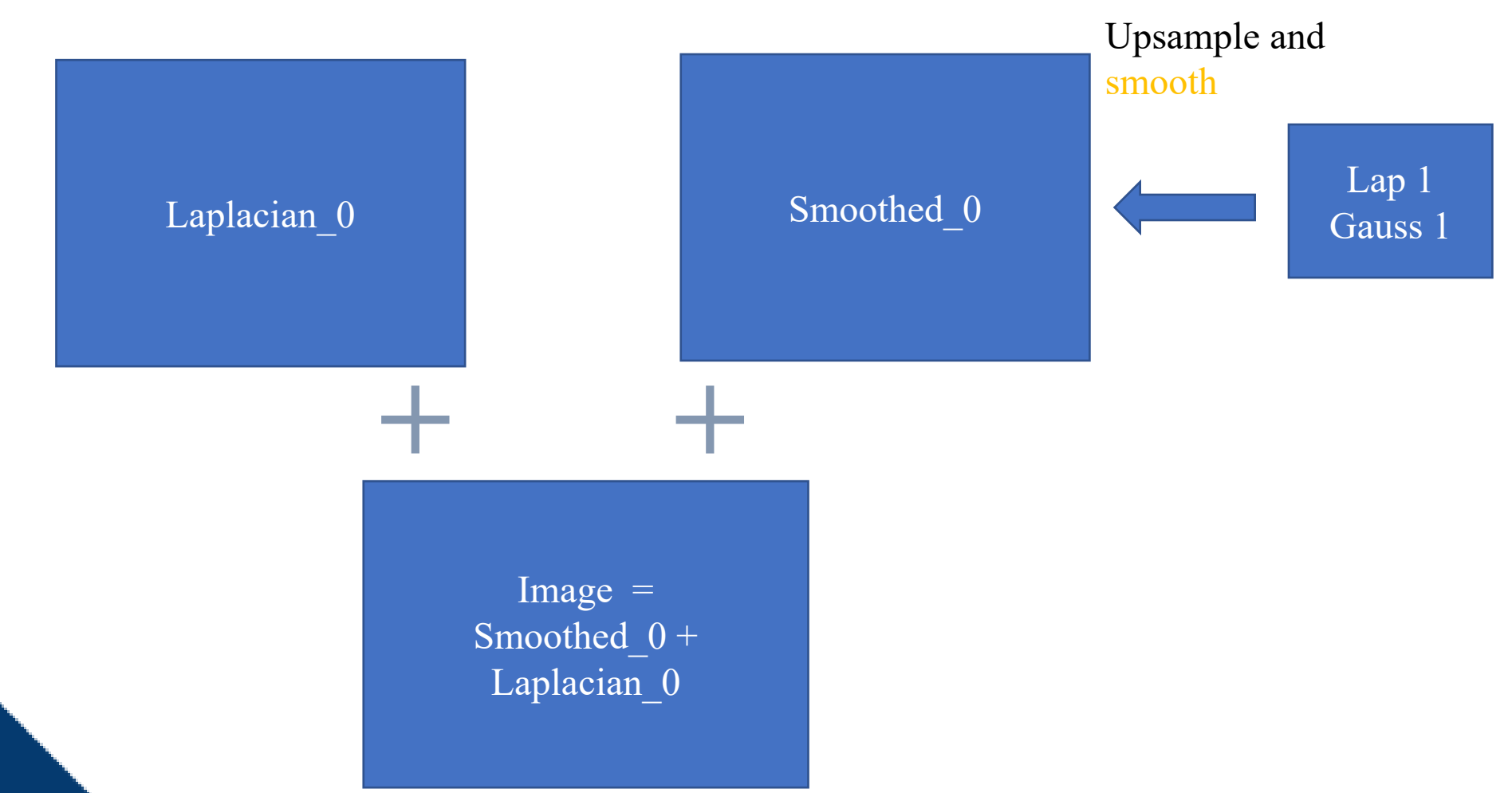
Creating a 2-level Laplacian pyramid



从拉普拉斯金字塔重建图像

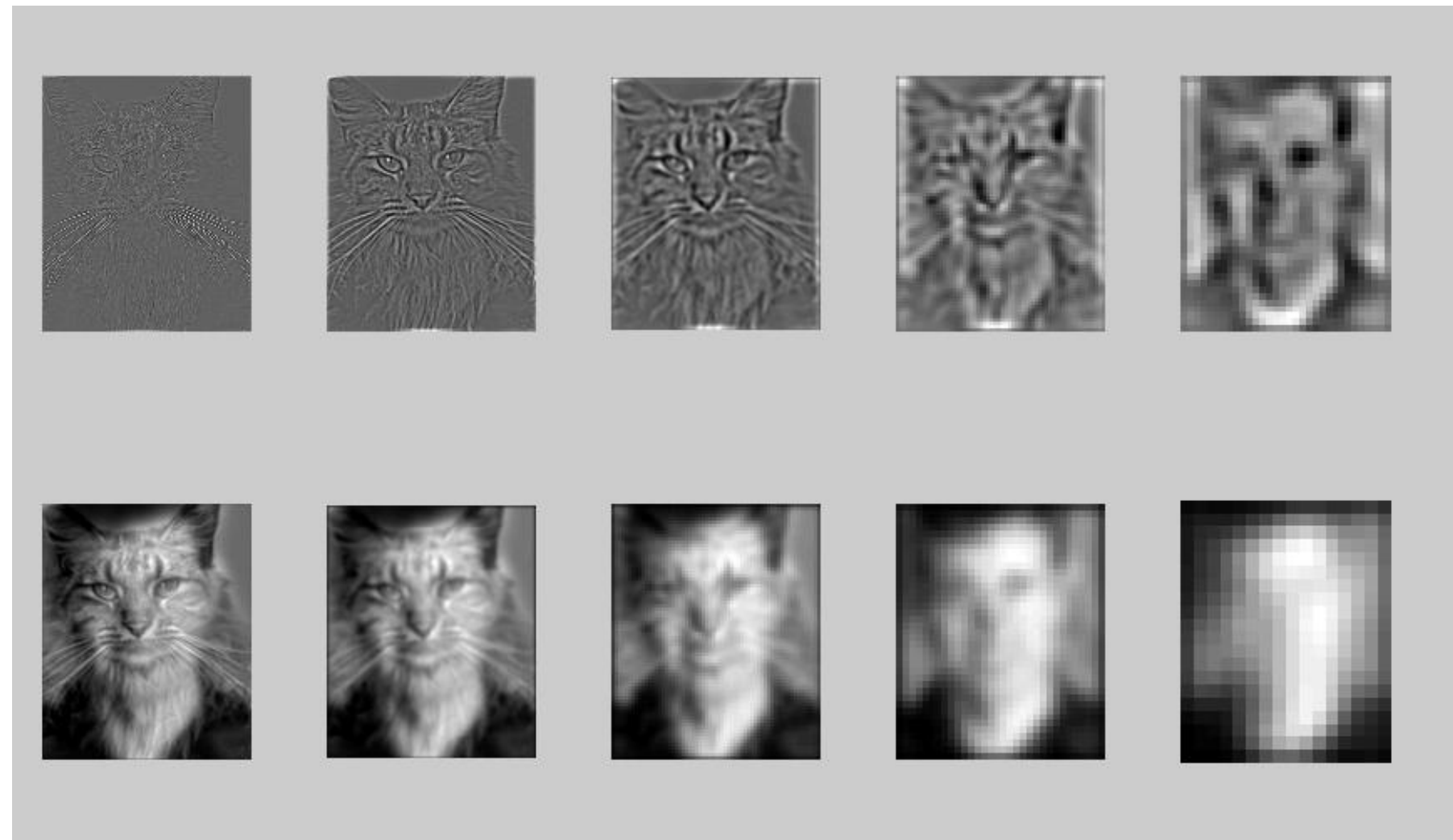


Reconstructing the image from Laplacian pyramid



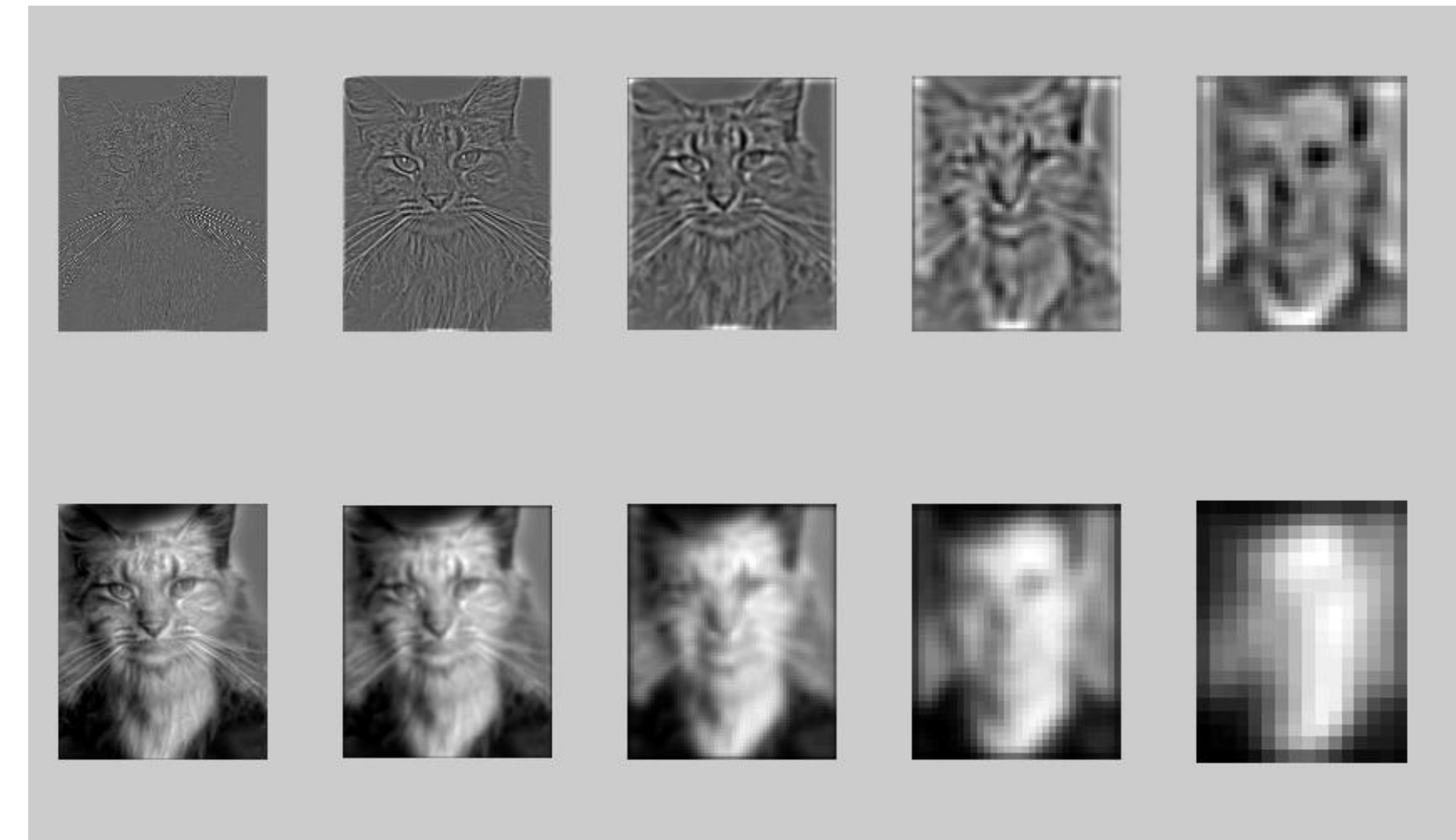
拉普拉斯金字塔中的混合图像

高频 → 低频

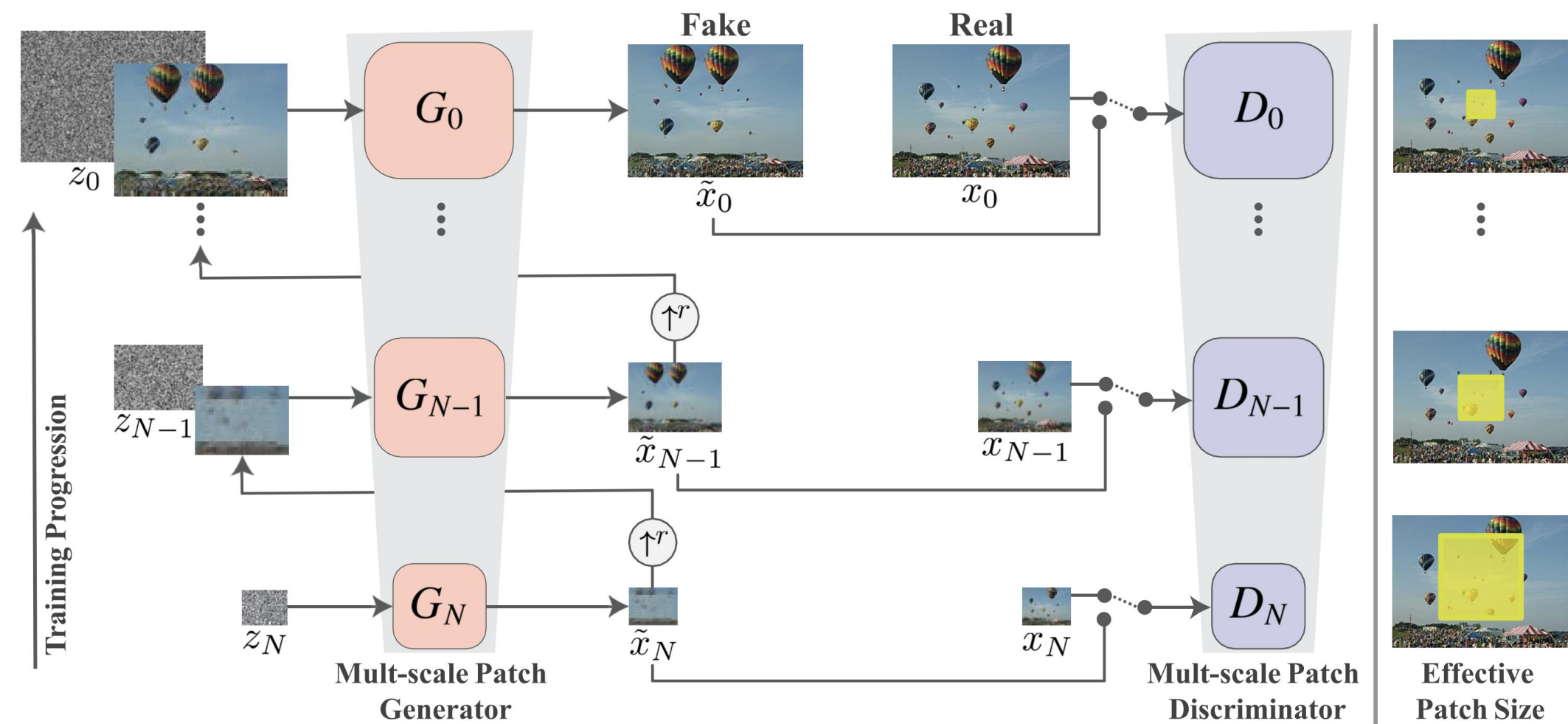


Hybrid Image in Laplacian Pyramid

High frequency → Low frequency

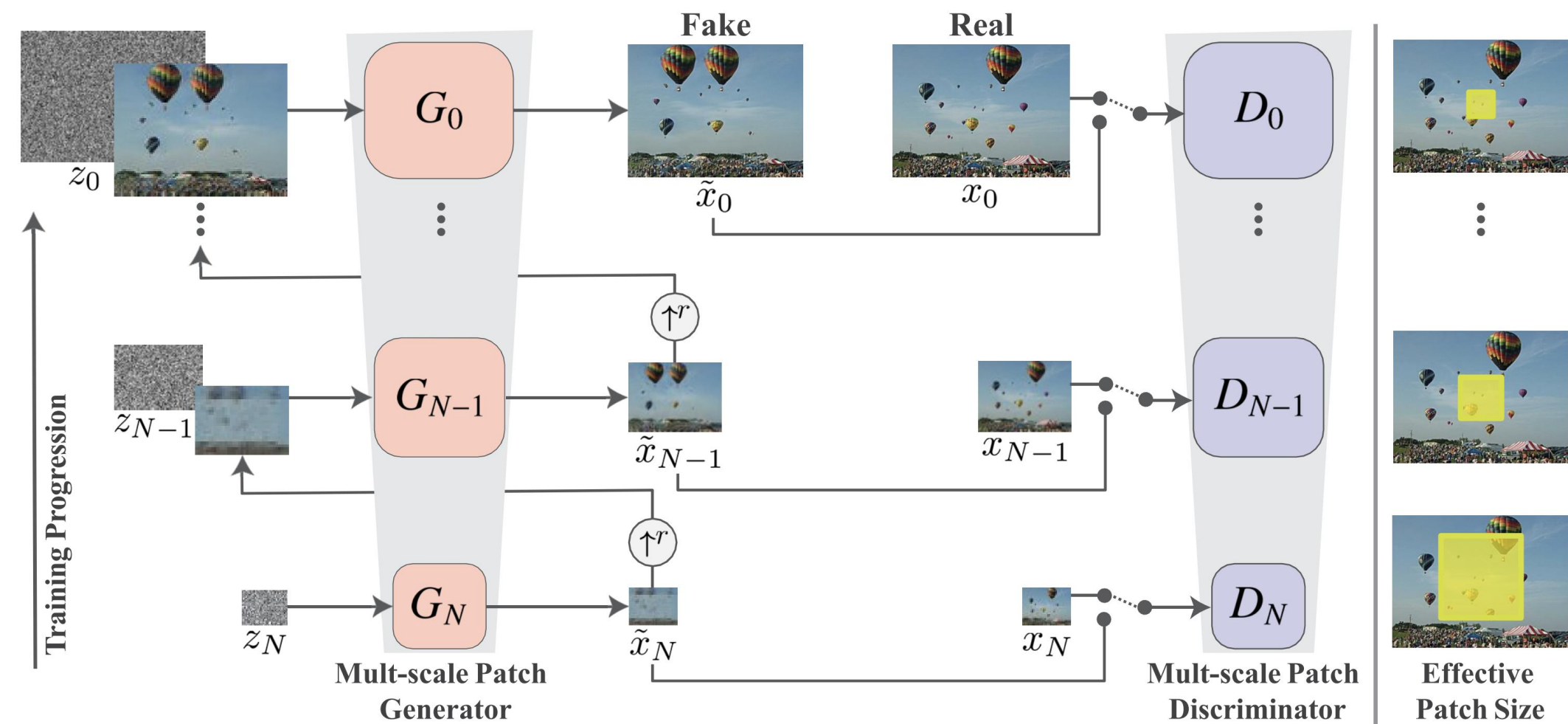


关于拉普拉斯金字塔的类似思路



<https://youtu.be/mdAcPe74tZI?t=3191>

Similar idea about laplacian pyramid



<https://youtu.be/mdAcPe74tZI?t=3191>

要点总结

- 图像滤波器可用于
 - 去噪
 - 图像增强
 - 模板匹配
 - ...
- 高斯金字塔支持跨不同分辨率进行匹配并提升速度
- 通过进行微小的修改，高斯金字塔就变成了拉普拉斯金字塔，这在图像合成应用中非常有用。



匿名课堂反馈

Take-home messages

- Image filters can be used for
 - Denoising
 - Image enhancement
 - Template matching
 - ...
- Gaussian pyramid enables matching across different resolutions and improves the speed
- By making small modification, Gaussian pyramid becomes Laplacian pyramid, which is quite useful image synthesis applications.



**Anonymous
Class Feedback**

